

PYTHON FOR GEN AI COMPLETE GUIDE



A MUST HAVE FOR ANYONE BUILDING
GEN AI APPS USING PYTHON

— @DATASCIENCEBRAIN —

Table of Contents

Phase 1 - Python Core for AI - PAGE 10

Module 01 - Python Foundations

1. Variables and data types - str, int, float, bool, None
2. Strings and fstrings for prompt assembly
3. Control flow - if/elif/else, for loops, while loops
4. Functions - parameters, defaults, type hints, docstrings
5. *args and **kwargs
6. Scope, closures, and the LEGB rule
7. Exception handling - try/except, custom exception classes, retry patterns

Module 02 - Data Structures & Comprehensions

1. Lists - creation, slicing, sorting, common methods
2. Dictionaries - access, update, iteration, nested dicts, defaultdict
3. Sets - deduplication, intersection, union, difference
4. Tuples - immutability, unpacking, NamedTuple
5. List, dict, and set comprehensions
6. Generators and the yield keyword

Module 03 - OOP & Modules

1. Classes - attributes, methods, **init**, **repr**, **len**
2. Inheritance and abstract base classes (ABC)
3. Decorators - functools.wraps, caching, logging, parametrised decorators
4. Dataclasses - autogenerated methods, field(), **post_init**, properties
5. Module and package structure

6. Recommended project layout with pyproject.toml

Module 04 - File I/O & APIs

1. Text files with pathlib
2. JSON - reading, writing, nested structures
3. CSV - DictReader, DictWriter, aggregation
4. Async/await with asyncio
5. Concurrent HTTP requests with httpx
6. Environment variables and secrets with pythondotenv

Module 05 - Python for Data

1. NumPy arrays - dtypes, shape, reshape, indexing
 2. Broadcasting and vectorised operations
 3. Cosine similarity and topk search from scratch
 4. Pandas DataFrames - creation, selection, filtering
 5. Data cleaning - fillna, dropna, computed columns
 6. GroupBy and aggregation
 7. Mini project: endtoend model evaluation pipeline
-

Phase 2 - LLM APIs & Prompting - PAGE 51

Module 06 - OpenAI & Anthropic APIs

1. Setting up clients and loading API keys
2. The Anthropic SDK - basic calls, system prompts, multiturn conversations
3. The OpenAI SDK - chat completions, the same interface pattern
4. Streaming responses with text_stream and write_stream
5. Tool calling - defining schemas, executing functions, returning results (both providers)
6. Vision - image inputs via URL and base64

7. Token counting and cost estimation
8. Building a provideragnostic BaseLLMClient abstraction

Module 07 - Prompt Engineering

1. Why system prompts matter - weak vs strong examples
2. Fewshot prompting - teaching format and task through examples
3. Chainofthought - zeroshot CoT and structured XMLtagged reasoning
4. Structured output - promptenforced JSON and OpenAI json_object mode
5. PromptTemplate dataclass with autodetected variable validation
6. Systematic prompt evaluation across a test set

Module 08 - Embeddings & Semantic Search

1. What embeddings are and why they work
 2. Generating embeddings with OpenAI and Voyage AI
 3. Cosine similarity - pairwise matrices, normalisation
 4. Inmemory VectorStore with semantic search
 5. Sentenceaware chunking with overlap
 6. Retrieval evaluation - Precision@k, Recall@k, MRR
 7. Metadatafiltered search
-

Phase 3 - GenAI Libraries - PAGE 101

Module 09 - LangChain

1. ChatOpenAI and ChatAnthropic - the unified interface
2. ChatPromptTemplate and MessagesPlaceholder
3. LCEL - the pipe operator for composing chains
4. RunnableParallel - running chains side by side
5. Conditional routing with RunnableLambda
6. Structured output with Pydantic and with_structured_output

7. RecursiveCharacterTextSplitter for document chunking
8. RunnableWithMessageHistory for conversational memory
9. Endtoend RAG chain with a Chroma vector store

Module 10 - LlamaIndex

1. Core concepts - Document, Node, Index, Query Engine, Retriever
2. Global Settings configuration
3. VectorStoreIndex - build, query, inspect source nodes
4. SummaryIndex for documentwide synthesis
5. Standalone VectorIndexRetriever used independently
6. Persistent storage with StorageContext
7. Custom query pipeline with ResponseMode options

Module 11 - Vector Databases

1. ChromaDB - inmemory and persistent clients, explicit embeddings, metadata filtering, update and delete
2. Pinecone - serverless index creation, upsert, query, metadata filter
3. Weaviate - collection schema, hybrid BM25 + vector search, metadata filter
4. Decision guide: when to use each database

Module 12 - Hugging Face Transformers

1. The Pipeline API - text classification, NER, text generation, summarisation
2. AutoTokenizer - encoding, decoding, batch encoding, token counting
3. AutoModelForCausalLM - direct inference with generate()
4. LoRA finetuning with PEFT - LoraConfig, get_peft_model, Trainer
5. Saving and loading adapters separately from the base model

Module 13 - LangGraph & Agents

1. StateGraph with TypedDict state and Annotated reducers
2. Nodes, edges, and conditional edges

3. ReAct agent - calculator and costlookup tools, full tool loop
4. Routing with stop conditions and iteration limits
5. Humanintheloop with interrupt_before and MemorySaver

Module 14 - CrewAI

1. Agent - role, goal, backstory, tools, allow_delegation
 2. Custom BaseTool with Pydantic input schemas
 3. Task - description, expected_output, context dependencies
 4. Process.sequential - tasks run in order with context passing
 5. Process.hierarchical - manager agent orchestrates workers
-

Phase 4 - RAG & Memory Systems - PAGE 166

Module 15 - RAG Fundamentals

1. The threestage RAG architecture - indexing, retrieval, generation
2. Common failure modes and how to diagnose them
3. Chunking strategies - fixedsize, recursive character, sentenceaware
4. Choosing a chunking strategy by document type
5. RAGIndexer - batched embedding, deduplication, persistent ChromaDB
6. RAGRetriever - cosine distance conversion, metadata prefiltering
7. RAGPromptBuilder - context budgeting, source labels
8. LLMasjudge reranking - twostage retrievethenrerank
9. Endtoend RAGPipeline class

Module 16 - Advanced RAG Patterns

1. HyDE - generating a hypothetical answer document for retrieval
2. Selfquery - LLM extracts metadata filters from natural language
3. Converting parsed filters to ChromaDB where clauses
4. Reciprocal Rank Fusion - merging ranked lists without score normalisation

5. Hybrid search - BM25 + dense with a tunable alpha parameter
6. Multihop retrieval - iterative queries guided by LLM followup decisions
7. RAG evaluation framework - faithfulness, answer relevance, context recall

Module 17 - Memory & State

1. The four memory types and when to use each
 2. WindowMemory - rolling turnpair buffer with token estimation
 3. SummaryMemory - LLM compression of overflow turns
 4. EntityMemory - structured fact extraction via JSONmode LLM
 5. PersistentSession - SQLite tables for messages, entities, and summaries
 6. VectorMemory - embedding all turns, retrieving by semantic relevance
 7. Memory strategy selection framework
-

Phase 5 - Deployment & Production - PAGE 231

Module 18 - Building AI Apps with Streamlit

1. How Streamlit's execution model works - reruns and session state
2. Minimal chat app - `st.chat_message`, `st.chat_input`, `st.write_stream`
3. `@st.cache_resource` for API clients and vector stores
4. RAG chat app with file upload, chunking, indexing, and context display
5. Multipage app structure - `pages/`, `components/`, `utils/`
`config.toml` for theming and `secrets.toml` for API keys
6. `@st.cache_data` with TTL for serialisable data
7. Progress indicators for longrunning operations
8. Deploying to Streamlit Cloud

Module 19 - FastAPI for GenAI Backends

1. Lifespan - startup and shutdown with `asynccontextmanager`
2. Pydantic request and response models with `field_validator`

3. CORS middleware and request timing middleware
4. Nonstreaming chat endpoint with Depends for auth
5. SSE streaming with StreamingResponse and correct headers
6. RAG query and document indexing endpoints
7. BackgroundTasks for nonblocking logging
8. Tokenbucket rate limiter with asyncio.Lock
9. Health and metrics endpoints
10. Running with uvicorn and gunicorn
11. TestClient for unit tests

Module 20 - Evaluation & Observability

1. JSONFormatter for structured singleline JSON logs
 2. Request logging middleware
 3. Tracer with parentchild spans and perspan attributes
 4. LLMasjudge - faithfulness, answer relevance, hallucination detection, conciseness
 5. RegressionSuite - saving baselines, comparing current scores, flagging drops
 6. Streamlit metrics dashboard - KPIs, latency distribution, eval scores, cost trend
 7. Production readiness checklist
-

Phase 6 - Portfolio Projects - PAGE 283

Project 01 - AI Research Assistant

1. PDF and text ingestion with pdfplumber
2. Paragraphaware chunking with overlap
3. Persistent ChromaDB index with deduplication
4. Perfile metadata filtering in retrieval

5. Streaming answers with inline [Source N] citation markers
6. Citation extraction and formatted source list
7. Streamlit UI with sidebar document management

Project 02 - Autonomous AI Agent

1. LangGraph ReAct graph - llm node, tools node, conditional routing
2. Five tools: calculator, python_repl, web_search, get_datetime, save_note
3. Safe code execution via subprocess with timeout
4. Persistent MemorySaver keyed by session_id
5. FastAPI backend streaming toolcall and done events as SSE
6. Streamlit frontend consuming the event stream in real time

Project 03 - MultiAgent Content Studio

1. Four CrewAI agents: Researcher, Writer, Editor, SEO Specialist
2. Three custom BaseTool implementations: WordCountTool, SEOAnalyserTool, ResearchSummaryTool
3. Sequential process with task context dependencies
4. FastAPI async job queue with BackgroundTasks
5. SSE progress streaming - one event per pipeline stage
6. Streamlit UI with live step indicators and article download

Phase 1 - Python Core for AI

Runtime: Python 3.12 · NumPy 2.4 · Pandas 3.0

Install: `pip install numpy pandas python-dotenv`

Module 01 - Python Foundations

Variables · Types · Control Flow · Functions · Exceptions

1.1 Why Python for AI

Python is the default language of the AI industry. Every major framework - PyTorch, TensorFlow, LangChain, Hugging Face, LlamaIndex - exposes a Python API. Its readable syntax, vast ecosystem, and interactive tooling (Jupyter, IPython) make it uniquely suited for data-heavy, experimental work.

Work through every example in a Python REPL or Jupyter notebook. The muscle memory of typing the code matters more than reading it.

1.2 Variables and Data Types

Python is dynamically typed - you do not declare a type; Python infers it at runtime. Use type hints (PEP 484) for readability and IDE support, especially in functions.

```
# Core scalar types
model_name: str = "claude-sonnet-4-5"
temperature: float = 0.7
max_tokens: int = 1024
is_streaming: bool = True

# Check types at runtime
print(type(model_name))    # <class 'str'>
print(type(temperature))   # <class 'float'>

# None - the absence of a value
```



```
response = None
print(response is None)    # True
```

Strings

Strings are immutable sequences of Unicode characters. The f-string is the most useful string tool for AI work - it assembles prompts cleanly and efficiently.

```
user_input    = "Explain transformers in simple terms"
system_prompt = "You are a helpful AI tutor."

# f-strings - the preferred way to assemble prompts
full_prompt = f"System:{system_prompt}\nUser:{user_input}"
print(full_prompt)

# Common string methods
print(user_input.upper())          # EXPLAIN TRANSFORMER
S...
print(user_input.split())          # ['Explain', 'transf
ormers', ...]
print(user_input.replace("simple", "plain"))
print(len(user_input))             # 35

# Multi-line strings - useful for long system prompts
prompt = """
You are an expert data scientist.
Answer concisely in bullet points.
"""
print(prompt.strip())
```

Numbers

```
# Integer arithmetic
tokens_used   = 450
tokens_limit  = 1024
```

```

remaining    = tokens_limit - tokens_used    # 574

# Float arithmetic
cost_per_token = 0.000003
total_cost     = tokens_used * cost_per_token
print(f"Cost: ${total_cost:.6f}")

# Integer division and modulo
batches = tokens_used // 100    # 4
leftover = tokens_used % 100    # 50

# Built-in math
import math
print(math.log2(512))    # 9.0 - useful in information theory
print(math.ceil(3.1))    # 4

```

Use underscores in large numeric literals for readability: `1_000_000` instead of `1000000`. Python ignores the underscores.

1.3 Control Flow

Python uses indentation - there are no braces. Consistent 4-space indentation is enforced by the community and most formatters (Black, Ruff).

if / elif / else

```

def classify_response_length(token_count: int) -> str:
    if token_count < 100:
        return "short"
    elif token_count < 500:
        return "medium"
    elif token_count < 2000:
        return "long"
    else:
        return "very long"

```

```

print(classify_response_length(80))    # short
print(classify_response_length(350))  # medium
print(classify_response_length(3000)) # very long

```

for Loops

```

models = ["gpt-4o", "claude-sonnet-4-5", "gemini-1.5-pro"]

# Basic iteration
for model in models:
    print(f"Checking:{model}")

# With index - use enumerate, not range(len(...))
for i, model in enumerate(models):
    print(f"{i + 1}.{model}")

# Iterate over key-value pairs in a dict
token_limits = {"gpt-4o": 128000, "claude-sonnet-4-5": 200000}
for model, limit in token_limits.items():
    print(f"{model}:{limit:,} tokens")

```

while Loops

```

import time

MAX_RETRIES = 3
attempt = 0

while attempt < MAX_RETRIES:
    attempt += 1
    print(f"Attempt{attempt}")
    if attempt == 2:
        print("Success!")
        break

```

```

        time.sleep(0.1)
    else:
        # Runs only if loop exhausted without break
        print("All retries failed")

```

The `for/while...else` construct is unique to Python. The `else` block runs only when the loop completes normally (no `break`). Use it for search loops where you need to handle “not found”.

1.4 Functions

Functions are the primary unit of reuse in Python. Write them early and often - they make prompt templates, API wrappers, and data transforms composable and testable.

Defining Functions

```

def build_prompt(system: str, user: str, temperature: float =
0.7) -> str:
    """Assemble a prompt string for an LLM API call.

    Args:
        system:      The system instruction.
        user:        The user message.
        temperature: Sampling temperature (0.0 – 2.0).

    Returns:
        Formatted prompt string.
    """
    return f"[System]\n{system}\n\n[User]\n{user}"

result = build_prompt(
    system="You are a concise AI assistant.",
    user="What is backpropagation?",

```

```
)  
print(result)
```

args and **kwargs

```
# *args - variable positional arguments  
def log_messages(*messages: str) -> None:  
    for msg in messages:  
        print(f"[LOG]{msg}")  
  
log_messages("Starting", "Loading model", "Done")  
  
# **kwargs - variable keyword arguments  
def create_api_payload(model: str, **kwargs) -> dict:  
    payload = {"model": model}  
    payload.update(kwargs)  
    return payload  
  
payload = create_api_payload(  
    "claude-sonnet-4-5",  
    max_tokens=1024,  
    temperature=0.3,  
    stream=True,  
)  
print(payload)  
# {'model': 'claude-sonnet-4-5', 'max_tokens': 1024, 'tempera  
ture': 0.3, 'stream': True}
```

Lambda Functions

```
responses = [  
    {"model": "gpt-4o", "tokens": 540},  
    {"model": "claude-sonnet-4-5", "tokens": 310},  
    {"model": "gemini-1.5-pro", "tokens": 820},  
]
```

```
# Sort by token count ascending
sorted_responses = sorted(responses, key=lambda r: r["tokens"])
for r in sorted_responses:
    print(f"{r['model']}: {r['tokens']} tokens")
```

1.5 Scope and Closures

Python uses LEGB scoping: **L**ocal → **E**nclosing → **G**lobal → **B**uilt-in. Understanding scope prevents the most common bugs when working with configuration objects and shared state in AI pipelines.

```
API_KEY = "sk-test-xxx"    # module-level (global)

def get_client():
    base_url = "https://api.anthropic.com"    # local
    return f"Client({base_url}, key={API_KEY[:6]}...)"

print(get_client())

# Closure - a function that remembers its enclosing scope
def make_counter(start: int = 0):
    count = [start]    # mutable container so inner function can mutate it
    def increment():
        count[0] += 1
        return count[0]
    return increment

token_counter = make_counter()
print(token_counter())    # 1
print(token_counter())    # 2
print(token_counter())    # 3
```

1.6 Exception Handling

AI API calls fail. Rate limits hit, networks time out, model responses are malformed. Defensive exception handling is not optional - it is required for production code.

```
import time
from typing import Optional

class APIError(Exception):
    """Raised when an AI API returns an error response."""
    def __init__(self, message: str, status_code: int):
        super().__init__(message)
        self.status_code = status_code

def call_api_with_retry(
    prompt: str,
    max_retries: int = 3,
    backoff_seconds: float = 2.0,
) -> Optional[str]:
    """Call a mock API with exponential backoff on failure.
    """
    for attempt in range(1, max_retries + 1):
        try:
            # Simulate API call - replace with real client call

            if attempt < 3:
                raise APIError("Rate limit exceeded", 429)
            return f"Response to:{prompt}"

        except APIError as e:
            if e.status_code == 429 and attempt < max_retries:
                wait = backoff_seconds ** attempt
                print(f"Rate limited. Retrying in {wait:.1f}s...")
                time.sleep(0.01) # shortened for demo
```

```

        else:
            raise
    except Exception as e:
        print(f"Unexpected error:{e}")
        raise

result = call_api_with_retry("What is a neural network?")
print(result)

```

Always create specific exception subclasses for your own errors (`APIError` , `ValidationError`). Catching bare `Exception` hides bugs; catching your own types lets you handle known failure modes gracefully.

1.7 Module 01 Exercises

1. Write a function `token_cost(tokens: int, model: str) -> float` that returns the estimated cost using a dict of costs per 1K tokens inside the function. Raise a `ValueError` if the model is unknown.
2. Build a `retry` decorator (using `functools.wraps`) that retries a function up to `n` times on any exception, with a fixed sleep between attempts. Test it with a function that fails the first two times.
3. Write a `temperature_label(t: float) -> str` function that maps 0.0–0.3 to `"precise"` , 0.3–0.7 to `"balanced"` , 0.7–1.0 to `"creative"` , and raises `ValueError` outside 0.0–1.0.
4. Parse the string `"128000 tokens, 0.005 USD per 1K"` and extract the token count as an `int` and the cost as a `float` using only string methods (no regex).

Module 02 - Data Structures & Comprehensions

Lists · Dicts · Sets · Tuples · Comprehensions · Generators

2.1 Lists

Lists are ordered, mutable sequences. In AI code they carry conversation histories, token arrays, search results, and batches of documents.


```

# Building a conversation history
conversation = []

conversation.append({"role": "user",      "content": "Hello"})
conversation.append({"role": "assistant", "content": "Hi! How can I help?"})
conversation.append({"role": "user",      "content": "Explain RAG."})

print(len(conversation))    # 3
print(conversation[0])      # first message
print(conversation[-1])     # last message
print(conversation[1:3])    # slice: messages 1 and 2

# Useful list methods
scores = [0.91, 0.76, 0.88, 0.65, 0.95]
scores.sort(reverse=True)   # in-place sort
print(scores)               # [0.95, 0.91, 0.88, 0.76, 0.65]
print(max(scores), min(scores))

# Remove by value vs by index
scores.remove(0.76)         # remove first occurrence of this value
popped = scores.pop()       # remove and return the last element
print(popped, scores)

```

2.2 Dictionaries

Dictionaries are the workhorse of Python AI code. API requests, model configs, retrieval results, and metadata are all dicts. Python 3.7+ guarantees insertion order.

```

# Building an LLM API payload
payload = {
    "model": "claude-sonnet-4-5",
    "max_tokens": 1024,
    "temperature": 0.7,
    "messages": [
        {"role": "user", "content": "What is attention in transformers?"}
    ],
}

# Access
print(payload["model"])           # claude-sonnet-4-5
print(payload.get("top_p", 1.0))  # default 1.0 if key missing

# Update
payload["temperature"] = 0.3
payload.update({"stream": True, "top_k": 40})

# Iteration
for key, value in payload.items():
    if not isinstance(value, list):
        print(f"{key}:{value}")

# Membership check
print("stream" in payload)        # True
print("top_p" in payload)         # False

```

Nested Dicts and Default Values

```

from collections import defaultdict

# Track token usage per model

```

```
usage: dict[str, dict[str, int]] = defaultdict(lambda: {"input": 0, "output": 0})

usage["claude-sonnet-4-5"]["input"] += 350
usage["claude-sonnet-4-5"]["output"] += 210
usage["gpt-4o"]["input"] += 420
usage["gpt-4o"]["output"] += 180

for model, counts in usage.items():
    total = counts["input"] + counts["output"]
    print(f"{model}:{total} total tokens")
```

2.3 Sets

Sets are unordered collections of unique items. Use them to deduplicate retrieved documents, track visited URLs, or compute overlap between keyword sets.

```
# Deduplication
retrieved_doc_ids = ["doc_3", "doc_1", "doc_3", "doc_7", "doc_1"]
unique_ids = set(retrieved_doc_ids)
print(unique_ids)    # {'doc_1', 'doc_3', 'doc_7'}

# Set operations - useful for keyword and topic analysis
gpt4_topics = {"coding", "math", "reasoning", "vision"}
claude_topics = {"coding", "writing", "reasoning", "safety"}

both = gpt4_topics & claude_topics    # intersection
either = gpt4_topics | claude_topics  # union
gpt_only = gpt4_topics - claude_topics # difference

print("Both: ", both)
print("Either: ", either)
print("GPT only:", gpt_only)
```

2.4 Tuples

Tuples are immutable sequences. Use them for data that must not change: coordinate pairs, function return values carrying multiple items, or dictionary keys.

```
# Return multiple values from a function
def parse_model_string(model_id: str) -> tuple[str, str, str]:
    """Parse 'provider/model-name:version' into parts."""
    provider, rest = model_id.split("/")
    if ":" in rest:
        name, version = rest.split(":")
    else:
        name, version = rest, "latest"
    return provider, name, version

provider, name, version = parse_model_string("anthropic/claude-sonnet-4-5:20241022")
print(f"Provider:{provider}, Model:{name}, Version:{version}")

# Named tuples - self-documenting tuples
from typing import NamedTuple

class EmbeddingResult(NamedTuple):
    text: str
    vector: list[float]
    model: str

result = EmbeddingResult(
    text="Hello world",
    vector=[0.12, -0.34, 0.89],
    model="text-embedding-3-small",
)
print(result.text, result.model)
```

2.5 Comprehensions

Comprehensions are concise, readable, and fast (implemented in C). They replace explicit `for` loops when the goal is to build a new sequence from an existing one.

List Comprehensions

```
messages = [
    {"role": "user",      "content": "What is RAG?"},
    {"role": "assistant", "content": "RAG stands for Retrieval-Augmented Generation."},
    {"role": "user",      "content": "Give an example."},
]

# Extract only user messages
user_messages = [m["content"] for m in messages if m["role"] == "user"]
print(user_messages)

# Word count per message
word_counts = [len(m["content"].split()) for m in messages]
print(word_counts)    # [3, 7, 3]

# Flatten a nested list
keywords = [["RAG", "retrieval"], ["LLM", "embedding"], ["vector"]]
flat      = [kw for group in keywords for kw in group]
print(flat)
```

Dict and Set Comprehensions

```
models          = ["gpt-4o", "claude-sonnet-4-5", "gemini-1.5-pro"]
context_windows = [128_000, 200_000, 1_000_000]

# Dict comprehension
```

```

model_context = {m: c for m, c in zip(models, context_windows)}
print(model_context)

# Filter to models with > 150K context
large_context = {m: c for m, c in model_context.items() if c > 150_000}
print(large_context)

# Set comprehension - unique word lengths
sentence = "the quick brown fox jumps over the lazy dog"
unique_lengths = {len(w) for w in sentence.split()}
print(sorted(unique_lengths)) # [2, 3, 4, 5]

```

2.6 Generators

Generators produce values lazily - one at a time, on demand. They are essential when processing large document collections, streaming token outputs, or chunking text without loading everything into memory.

```

from typing import Generator

def chunk_text(
    text: str,
    chunk_size: int = 500,
    overlap: int = 50,
) -> Generator[str, None, None]:
    """Yield overlapping text chunks for embedding/RAG pipelines."""
    start = 0
    while start < len(text):
        end = min(start + chunk_size, len(text))
        yield text[start:end]
        start += chunk_size - overlap

```

```

# Load a long document (simulated)
document = "Python is a versatile language. " * 30

# Process chunks without loading all into memory at once
chunk_count = 0
for chunk in chunk_text(document, chunk_size=100, overlap=20):
    chunk_count += 1
    # In real code: embed the chunk and store in vector DB

print(f"Processed{chunk_count} chunks")

# Generator expression - like a list comprehension but lazy
sizes = (len(chunk) for chunk in chunk_text(document, 100, 20))
print(f"Max chunk size:{max(sizes)}")

```

A list comprehension `[f(x) for x in items]` builds the full list in memory immediately. A generator expression `(f(x) for x in items)` computes each value only when requested. For 10 million documents, this difference is decisive.

2.7 Module 02 Exercises

1. Given a list of API response dicts (each with `"tokens"` and `"latency_ms"` keys), write a comprehension that returns only responses where latency is under 500ms, sorted by token count ascending.
2. Build a function `conversation_stats(messages: list[dict]) -> dict` that returns a dict with keys `"total_messages"`, `"user_turns"`, `"assistant_turns"`, and `"avg_words_per_message"`.
3. Write a generator `batch_items(items, batch_size)` that yields lists of `batch_size` items from any iterable. Handle the last partial batch correctly.
4. Use set operations to find models that appear in both a `fast_models` list and a `cheap_models` list - models that are both fast and cheap.

Module 03 - OOP & Modules

Classes · Inheritance · Decorators · Dataclasses · Packaging

3.1 Classes and Objects

Object-Oriented Programming lets you model AI components - clients, memory stores, prompt templates, agents - as self-contained objects with state and behaviour. This is how every major AI library is built.

```
from typing import Optional

class ConversationHistory:
    """Manages a rolling window of messages for LLM context."""

    def __init__(self, max_turns: int = 10, system_prompt: str = ""):
        self.system_prompt = system_prompt
        self.max_turns = max_turns
        self._messages: list[dict] = []

    def add(self, role: str, content: str) -> None:
        """Add a message and trim to max_turns."""
        if role not in ("user", "assistant"):
            raise ValueError(f"Invalid role:{role!r}")
        self._messages.append({"role": role, "content": content})

        if len(self._messages) > self.max_turns * 2:
            self._messages = self._messages[-(self.max_turns * 2):]

    def to_api_payload(self) -> list[dict]:
        """Return messages formatted for an LLM API call."""
        return list(self._messages)
```



```

def clear(self) -> None:
    self._messages = []

def __len__(self) -> int:
    return len(self._messages)

def __repr__(self) -> str:
    return f"ConversationHistory(turns={len(self._messages)}, max={self.max_turns})"

# Usage
history = ConversationHistory(max_turns=5, system_prompt="Be concise.")
history.add("user", "What is an embedding?")
history.add("assistant", "An embedding is a vector representation of data.")
history.add("user", "Give a use case.")
print(history)          # ConversationHistory(turns=3, max=5)
print(len(history))     # 3

```

3.2 Inheritance

Inheritance lets you specialise behaviour without duplicating code. In AI libraries this appears everywhere: a base LLM client extended by provider-specific clients, a base retriever extended by vector/BM25/hybrid retrievers.

```

from abc import ABC, abstractmethod

class BaseLLMClient(ABC):
    """Abstract base class for LLM provider clients."""

    def __init__(self, api_key: str, model: str):
        self.api_key = api_key
        self.model = model

```

```

@abstractmethod
def complete(self, messages: list[dict], **kwargs) -> str:
    """Send messages and return the assistant's reply."""

def count_words(self, text: str) -> int:
    """Shared utility - word count estimate."""
    return len(text.split())

class MockAnthropicClient(BaseLLMClient):
    """Simulates an Anthropic API client for testing."""

    def complete(self, messages: list[dict], **kwargs) -> str:
        last_user = next(
            m["content"] for m in reversed(messages) if m["role"] == "user"
        )
        return f"[Mock Anthropic] Echo:{last_user}"

class MockOpenAIClient(BaseLLMClient):
    """Simulates an OpenAI API client for testing."""

    def complete(self, messages: list[dict], **kwargs) -> str:
        return f"[Mock OpenAI] Received{len(messages)} messages."

# Polymorphism - same interface, different implementation
clients: list[BaseLLMClient] = [
    MockAnthropicClient("key-ant", "claude-sonnet-4-5"),
    MockOpenAIClient("key-oai", "gpt-4o"),

```

```
]

msgs = [{"role": "user", "content": "Hello!"}]
for client in clients:
    print(client.complete(msgs))
```

3.3 Decorators

Decorators wrap a function to add behaviour - logging, caching, retry, rate limiting - without modifying the function's body. They are used extensively in FastAPI, LangChain, and Python testing.

```
import functools
import time

def log_call(func):
    """Decorator: log function name and return value."""
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f">>> Calling{func.__name__}")
        result = func(*args, **kwargs)
        print(f"<<<{func.__name__} returned:{result!r}")
        return result
    return wrapper

def cache_result(func):
    """Simple in-memory cache (no expiry)."""
    _cache: dict = {}
    @functools.wraps(func)
    def wrapper(*args):
        if args not in _cache:
            _cache[args] = func(*args)
        return _cache[args]
    return wrapper
```

```

@log_call
@cache_result
def get_embedding(text: str) -> list[float]:
    """Simulate an embedding API call (cached)."""
    time.sleep(0.01) # simulate latency
    return [hash(text) % 100 / 100.0, 0.42, 0.87]

# First call: logs + computes
e1 = get_embedding("What is RAG?")
# Second call: logs but returns from cache instantly
e2 = get_embedding("What is RAG?")
print(e1 == e2) # True

```

Parametrised Decorators

```

import functools
import time

def retry(max_attempts: int = 3, delay: float = 0.1):
    """Parametrised retry decorator."""
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            last_error = None
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    last_error = e
                    if attempt < max_attempts:
                        time.sleep(delay)
            raise last_error
        return wrapper
    return decorator

```

```
@retry(max_attempts=3, delay=0.05)
def flaky_api_call(prompt: str) -> str:
    import random
    if random.random() < 0.6: # fails 60% of the time
        raise ConnectionError("Simulated network error")
    return f"Response to:{prompt}"
```

3.4 Dataclasses

The `dataclasses` module (Python 3.7+) eliminates boilerplate for classes that primarily hold data. It auto-generates `__init__`, `__repr__`, and `__eq__`. Combined with type hints, dataclasses make configuration objects readable and safe.

```
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class LLMConfig:
    """Configuration for a single LLM API call."""
    model: str
    temperature: float = 0.7
    max_tokens: int = 1024
    top_p: float = 1.0
    stop_sequences: list[str] = field(default_factory=list)
    system_prompt: Optional[str] = None

    def __post_init__(self):
        if not 0.0 <= self.temperature <= 2.0:
            raise ValueError(f"temperature must be 0-2, got{self.temperature}")
        if self.max_tokens < 1:
            raise ValueError("max_tokens must be >= 1")

    @property
    def as_dict(self) -> dict:
```

```

        """Return dict suitable for passing to an API client.
        """
        d = {
            "model": self.model,
            "temperature": self.temperature,
            "max_tokens": self.max_tokens,
            "top_p": self.top_p,
        }
        if self.stop_sequences:
            d["stop_sequences"] = self.stop_sequences
        if self.system_prompt:
            d["system"] = self.system_prompt
        return d

    cfg = LLMConfig(
        model="claude-sonnet-4-5",
        temperature=0.3,
        system_prompt="You are a concise Python tutor.",
    )
    print(cfg)
    print(cfg.as_dict)

```

3.5 Modules and Packages

As your AI project grows, organise code into modules (single `.py` files) and packages (directories with `__init__.py`). A well-structured package makes imports clean and code reusable across projects.

Recommended Project Layout

```

my_ai_project/
  src/
    my_ai_project/
      __init__.py
      config.py          ← LLMConfig and settings
      clients/
        __init__.py
        anthropic.py     ← AnthropicClient

```

```

    openai.py      ← OpenAIClient
    retrieval/
        __init__.py
        chunker.py    ← chunk_text generator
        embedder.py   ← embed_texts function
    pipeline.py    ← main RAG pipeline
tests/
    test_config.py
    test_chunker.py
pyproject.toml
README.md

```

Importing Correctly

```

# Absolute imports - preferred
from my_ai_project.config import LLMConfig
from my_ai_project.clients.anthropic import AnthropicClient

# Import specific names
from my_ai_project.retrieval.chunker import chunk_text

# Import module as alias
import my_ai_project.retrieval.embedder as embedder
vectors = embedder.embed_texts(["hello", "world"])

```

Use `pyproject.toml` (PEP 517/518) for all new projects - not `setup.py`. The standard tool chain is: `hatch` or `poetry` for packaging, `ruff` for linting, `mypy` for type checking, and `pytest` for tests.

3.6 Module 03 Exercises

1. Implement a `RateLimiter` class with a method `check_and_wait()` that ensures no more than `N` calls per minute, sleeping as needed. Test it by calling it 5 times rapidly.
2. Create a `PromptTemplate` dataclass with a `template` string field and a `render(**kwargs)` method that fills in placeholders using `str.format_map`. Add validation that all placeholders are provided.

3. Write a `@retry(max_attempts=3, delay=0.1)` parametrised decorator - the syntax is `@retry(max_attempts=3)`. Test it on a function that raises on the first two calls.
4. Organise the `ConversationHistory` class from section 3.1 and the `LLMConfig` dataclass from section 3.4 into a package structure with proper `__init__.py` exports.

Module 04 - File I/O & APIs

JSON · CSV · Async/await · httpx · Environment variables

4.1 Working with Files

AI pipelines constantly read and write data: prompt templates, model outputs, evaluation results, embeddings. The `pathlib` module is the modern, preferred way to work with file paths.

Text Files

```
import pathlib

# pathlib is preferred over os.path in Python 3.6+
data_dir = pathlib.Path("data")
data_dir.mkdir(exist_ok=True)

# Write a prompt template
template = """You are a{role}.
Answer the following question concisely.

Question:{question}"""

template_file = data_dir / "qa_prompt.txt"
template_file.write_text(template, encoding="utf-8")

# Read it back and fill placeholders
loaded = template_file.read_text(encoding="utf-8")
filled = loaded.format(role="Python tutor", question="What is
```



```

a generator?")
print(filled)

# List all .txt files in a directory
for f in data_dir.glob("*.txt"):
    print(f.name, f.stat().st_size, "bytes")

```

JSON

```

import json
import pathlib

# Save model evaluation results
results = {
    "model": "claude-sonnet-4-5",
    "benchmark": "MMLU",
    "scores": {"science": 0.91, "math": 0.88, "history": 0.85},
    "total_samples": 14042,
    "timestamp": "2025-01-15T09:30:00Z",
}

out = pathlib.Path("results.json")
out.write_text(json.dumps(results, indent=2), encoding="utf-8")

# Read back and use
data = json.loads(out.read_text(encoding="utf-8"))
print(f"Model: {data['model']}")
avg = sum(data["scores"].values()) / len(data["scores"])
print(f"Average score: {avg:.2%}")

```

CSV

```

import csv
import pathlib

# Write evaluation results as CSV
rows = [
    {"prompt": "What is RAG?", "model": "claude-sonnet-4-5", "tokens": 312, "latency_ms": 410},
    {"prompt": "Explain embeddings", "model": "claude-sonnet-4-5", "tokens": 498, "latency_ms": 580},
    {"prompt": "What is RAG?", "model": "gpt-4o", "tokens": 287, "latency_ms": 360},
]

csv_file = pathlib.Path("eval.csv")
with csv_file.open("w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["prompt", "model", "tokens", "latency_ms"])
    writer.writeheader()
    writer.writerows(rows)

# Read and aggregate by model
from collections import defaultdict
model_latencies: dict[str, list[float]] = defaultdict(list)

with csv_file.open(encoding="utf-8") as f:
    for row in csv.DictReader(f):
        model_latencies[row["model"]].append(float(row["latency_ms"]))

for model, latencies in model_latencies.items():
    avg = sum(latencies) / len(latencies)
    print(f"{model}: avg latency{avg:.0f}ms")

```

4.2 Async/Await

When you need to call multiple APIs concurrently - comparing model outputs, embedding a batch of documents, or fetching data while the LLM responds - use `asyncio`. The `httpx` library is the modern async-capable HTTP client.

```
import asyncio
import httpx

async def fetch_json(client: httpx.AsyncClient, url: str) -> dict:
    """Fetch a URL asynchronously and return JSON."""
    response = await client.get(url, timeout=10.0)
    response.raise_for_status()
    return response.json()

async def fetch_multiple(urls: list[str]) -> list[dict]:
    """Fetch all URLs concurrently - much faster than sequential."""
    async with httpx.AsyncClient() as client:
        tasks = [fetch_json(client, url) for url in urls]
        results = await asyncio.gather(*tasks, return_exceptions=True)
    return [r for r in results if not isinstance(r, Exception)]

async def main():
    # JSONPlaceholder - a free public test API
    urls = [
        f"https://jsonplaceholder.typicode.com/posts/{i}"
        for i in range(1, 4)
    ]
    posts = await fetch_multiple(urls)
    for post in posts:
        print(f"Post{post['id']}: {post['title'][:40]}")

asyncio.run(main())
```

Use `asyncio` + `httpx` when making multiple independent API calls in a single handler. Use synchronous `requests` for simple sequential scripts or CLI tools.

Simulating LLM API Calls

```
import asyncio

async def call_model_mock(model: str, prompt: str) -> dict:
    """Simulate calling an LLM API (replace with real client
    call)."""
    await asyncio.sleep(0.05) # simulate network latency
    return {
        "model": model,
        "response": f"[{model}] Answer to:{prompt[:30]}",
        "tokens": 42,
    }

async def compare_models(prompt: str, models: list[str]) -> list[dict]:
    """Call multiple models in parallel and return all responses."""
    tasks = [call_model_mock(m, prompt) for m in models]
    results = await asyncio.gather(*tasks)
    return list(results)

responses = asyncio.run(compare_models(
    "What is RAG?",
    ["claude-sonnet-4-5", "gpt-4o", "gemini-1.5-pro"],
))
for r in responses:
    print(f"{r['model']}:{r['response']}")
```

4.3 Environment Variables and Secrets

Never hardcode API keys. Use environment variables, loaded via `python-dotenv` in development.

```
# Create a .env file in your project root:
# ANTHROPIC_API_KEY=sk-ant-...
# OPENAI_API_KEY=sk-...
```

```
# Add to .gitignore immediately:
echo ".env" >> .gitignore
```

```
import os
from dotenv import load_dotenv

load_dotenv()  # loads .env into os.environ

def get_api_key(provider: str) -> str:
    """Retrieve an API key from the environment."""
    key_map = {
        "anthropic": "ANTHROPIC_API_KEY",
        "openai":     "OPENAI_API_KEY",
        "google":     "GOOGLE_API_KEY",
    }
    env_var = key_map.get(provider.lower())
    if not env_var:
        raise ValueError(f"Unknown provider:{provider}")
    key = os.getenv(env_var)
    if not key:
        raise EnvironmentError(
            f"{env_var} is not set. Add it to your .env file."
        )
    return key

# In practice:
# anthropic_key = get_api_key("anthropic")
# openai_key    = get_api_key("openai")
```

A leaked key can run up thousands of dollars in API costs within hours. Add `.env` to `.gitignore` before your first commit.

4.4 Module 04 Exercises

1. Write `save_conversation(history: list[dict], path: str)` and `load_conversation(path: str) -> list[dict]` that serialise and deserialise conversation history to/from a JSON file.
 2. Build an async function `compare_endpoints(urls: list[str])` that fetches all URLs concurrently with `httpx` and returns a list of `(url, status_code, response_time_ms)` tuples.
 3. Create a config loader that reads a JSON config file and merges it with environment variable overrides (env var values take precedence). Use `pathlib` for file operations.
 4. Write a CSV log writer class that appends a row each time an LLM is called, recording: timestamp, model, input_tokens, output_tokens, latency_ms. Make it thread-safe with a `threading.Lock`.
-

Module 05 - Python for Data

NumPy Arrays · Pandas DataFrames · Data Wrangling for ML

5.1 NumPy

NumPy is the foundation of numerical computing in Python. Embedding vectors, similarity matrices, and attention weight arrays are all NumPy arrays under the hood in nearly every AI library.

Arrays and Dtypes

```
import numpy as np

# Creating arrays
scores = np.array([0.91, 0.76, 0.88, 0.65, 0.95], dtype=np.float32)
```

```

print(scores.dtype, scores.shape)    # float32 (5,)

# Zeros, ones, ranges
zeros    = np.zeros((3, 4))          # 3 rows, 4
cols of 0.0
rng_vals = np.arange(0, 1.0, 0.1)    # [0.0, 0.1,
..., 0.9]
linspace = np.linspace(0, 1, 5)      # [0.0, 0.25,
0.5, 0.75, 1.0]

# Random - use a seeded Generator for reproducibility
rng      = np.random.default_rng(seed=42)
mock_embedding = rng.standard_normal(1536) # 1536-dim like
text-embedding-3-small
print(f"Embedding shape:{mock_embedding.shape}, mean:{mock_em
bedding.mean():.4f}")

```

Shape, Reshape, Indexing

```

import numpy as np

# Simulate 4 document embeddings of dimension 8
rng      = np.random.default_rng(42)
embeddings = rng.standard_normal((4, 8))
print("Shape:", embeddings.shape)      # (4, 8)
print("First embedding:", embeddings[0])
print("First 3 dims of all docs:\n", embeddings[:, :3])

# Reshape
flat = embeddings.flatten()            # (32,)
back = flat.reshape(4, 8)             # (4, 8)

# Boolean indexing
similarity_scores = np.array([0.91, 0.43, 0.78, 0.55])
above_threshold   = embeddings[similarity_scores > 0.7]

```

```
print(f"Docs above 0.7 similarity:{above_threshold.shape
[0]}")
```

Cosine Similarity - The Core of Semantic Search

```
import numpy as np

def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    """Compute cosine similarity between two vectors."""
    norm_a = np.linalg.norm(a)
    norm_b = np.linalg.norm(b)
    if norm_a == 0 or norm_b == 0:
        return 0.0
    return float(np.dot(a, b) / (norm_a * norm_b))

def top_k_similar(
    query: np.ndarray,
    corpus: np.ndarray,
    k: int = 3,
) -> list[tuple[int, float]]:
    """Return indices and scores of the k most similar vector
    s."""
    # Normalise corpus rows
    norms = np.linalg.norm(corpus, axis=1, keepdims=True)
    norms = np.where(norms == 0, 1, norms)
    normed = corpus / norms

    # Normalise query
    q_norm = np.linalg.norm(query)
    q_normed = query / (q_norm if q_norm > 0 else 1)

    # Matrix-vector multiply → cosine similarities
    sims = normed @ q_normed
    top_idx = np.argsort(sims)[::-1][:k]
    return [(int(i), float(sims[i])) for i in top_idx]
```



```

rng      = np.random.default_rng(42)
corpus   = rng.standard_normal((10, 8))
query    = rng.standard_normal(8)

results  = top_k_similar(query, corpus, k=3)
for idx, score in results:
    print(f"Doc{idx}: similarity = {score:.4f}")

```

This `top_k_similar` function is the core of a vector search engine. Vectorised NumPy operations on a corpus of 10,000 documents take milliseconds - a Python loop over the same data would take seconds.

5.2 Pandas

Pandas provides the DataFrame - a labelled, 2D table built on NumPy. It is the standard tool for loading, cleaning, exploring, and transforming tabular data before feeding it to AI models.

Creating and Inspecting DataFrames

```

import pandas as pd

data = [
    {"model": "gpt-4o",          "provider": "OpenAI",      "context_k": 128, "cost_input": 2.50},
    {"model": "claude-sonnet-4-5", "provider": "Anthropic", "context_k": 200, "cost_input": 3.00},
    {"model": "gemini-1.5-pro",   "provider": "Google",     "context_k": 1000, "cost_input": 1.25},
    {"model": "llama-3.1-70b",    "provider": "Meta",       "context_k": 128, "cost_input": 0.00},
]

df = pd.DataFrame(data)
print(df.shape)      # (4, 4)

```

```
print(df.dtypes)
print(df.head())
print(df.describe())
```

Selection and Filtering

```
import pandas as pd

df = pd.DataFrame([
    {"model": "gpt-4o",          "context_k": 128,  "cost_input": 2.50},
    {"model": "claude-sonnet-4-5", "context_k": 200,  "cost_input": 3.00},
    {"model": "gemini-1.5-pro",    "context_k": 1000, "cost_input": 1.25},
    {"model": "llama-3.1-70b",     "context_k": 128,  "cost_input": 0.00},
])

# Select column
print(df["model"].tolist())

# Filter rows
affordable = df[df["cost_input"] < 2.0]
print(affordable)

# Multiple conditions
big_and_cheap = df[(df["context_k"] >= 128) & (df["cost_input"] < 2.0)]
print(big_and_cheap[["model", "context_k", "cost_input"]])

# loc (label-based) vs iloc (position-based)
print(df.loc[0, "model"])    # 'gpt-4o'
print(df.iloc[0, 0])         # 'gpt-4o'
```

Cleaning and Transforming

```
import pandas as pd
import numpy as np

# Simulate a messy evaluation dataset
raw = pd.DataFrame({
    "prompt":      ["Q1", "Q2", "Q3", "Q4", "Q5"],
    "response":    ["OK", None, "Good", "Bad", "OK"],
    "score":       [0.9, None, 0.85, 0.3, 0.88],
    "latency_ms":  [410, 520, None, 390, 480],
})

# Inspect missing data
print(raw.isnull().sum())

# Fill missing values
raw["score"]      = raw["score"].fillna(raw["score"].mean())
raw["latency_ms"] = raw["latency_ms"].fillna(raw["latency_ms"].median())

# Drop rows still missing critical columns
clean = raw.dropna(subset=["response"]).copy()

# Add computed column
clean["pass"] = clean["score"] >= 0.7

print(clean)
print(f"Pass rate:{clean['pass'].mean():.0%}")
```

GroupBy and Aggregation

```
import pandas as pd

evals = pd.DataFrame({
```

```

        "model":      ["claude", "gpt-4o", "claude", "gpt-4o", "c
        laude", "gpt-4o"],
        "task":       ["qa", "qa", "summarise", "summarise", "cod
        e", "code"],
        "score":      [0.91, 0.88, 0.85, 0.82, 0.93, 0.90],
        "latency_ms": [420, 380, 610, 550, 340, 300],
    })

# Average score per model
print(evals.groupby("model")["score"].mean())

# Multiple aggregations
summary = evals.groupby("model").agg(
    avg_score      = ("score",      "mean"),
    avg_latency    = ("latency_ms", "mean"),
    num_tasks      = ("task",       "count"),
)
print(summary)

# Pivot table - model vs task
pivot = evals.pivot_table(
    values="score", index="model", columns="task", aggfunc="m
    ean"
)
print(pivot)

```

5.3 Mini Project - Model Evaluation Pipeline

This example wires together everything from Phase 1: functions, dataclasses, file I/O, NumPy, and Pandas into a minimal model evaluation script.

```

import numpy as np
import pandas as pd
from dataclasses import dataclass, field

```

```

@dataclass
class EvalResult:
    prompt: str
    model: str
    response: str
    latency_ms: float
    score: float = field(default=0.0)

def mock_llm_call(prompt: str, model: str) -> tuple[str, float]:
    """Simulate an LLM call. Returns (response, latency_ms)."""
    rng = np.random.default_rng(abs(hash(prompt + model))
% 2**31)
    latency = rng.uniform(300, 700)
    response = f"[{model}] Answer to:{prompt[:30]}"
    return response, latency

def score_response(response: str, expected_keywords: list[str]) -> float:
    """Simple keyword-based scorer (0.0 – 1.0)."""
    found = sum(1 for kw in expected_keywords if kw.lower() in response.lower())
    return found / len(expected_keywords) if expected_keywords else 0.0

# Run evaluation
prompts = [
    ("What is a transformer?", ["attention", "model"]),
    ("Define RAG", ["retrieval", "generation"]),
    ("What is fine-tuning?", ["training", "weights"]),
]
models = ["claude-sonnet-4-5", "gpt-4o"]

results: list[EvalResult] = []
for prompt, keywords in prompts:

```

```

    for model in models:
        response, latency = mock_llm_call(prompt, model)
        score = score_response(response, keywords)

        results.append(EvalResult(prompt, model, response, latency, score))

# Analyse with Pandas
df = pd.DataFrame([vars(r) for r in results])

summary = df.groupby("model").agg(
    avg_score = ("score", "mean"),
    avg_latency = ("latency_ms", "mean"),
).round(3)

print(summary)

# Save results
df.to_csv("/tmp/eval_results.csv", index=False)
print("Saved to /tmp/eval_results.csv")

```

5.4 Module 05 Exercises

1. Load a CSV of LLM benchmark scores (create one with at least 20 rows and 4 models). Compute: mean score per model, best-performing task per model, and a correlation between score and latency columns.
2. Write a function `normalise_embeddings(matrix: np.ndarray) -> np.ndarray` that L2-normalises each row. Verify that all row norms equal `1.0` after normalisation.
3. Implement a function that reads a folder of `.txt` files and returns a Pandas DataFrame with columns: `filename`, `char_count`, `word_count`, `sentence_count`. Sort by `word_count` descending.
4. Build a pairwise cosine similarity matrix for a small corpus of 5 strings using hash-based mock embeddings. Find and print the pair with the highest similarity.

Phase 1 Summary

What You Learned

Module	Core Concepts Covered
M01 - Foundations	Variables, types, f-strings, <code>for</code> / <code>while</code> loops, functions (<code>*args</code> / <code>**kwargs</code>), closures, exception handling with custom exception subclasses
M02 - Data Structures	Lists, dicts, sets, tuples, list/dict/set comprehensions, generators for memory-efficient iteration
M03 - OOP & Modules	Classes, inheritance, abstract base classes, <code>@functools.wraps</code> decorators, parametrised decorators, dataclasses with <code>__post_init__</code> , package layout with <code>pyproject.toml</code>
M04 - File I/O & APIs	<code>pathlib</code> , JSON, CSV, async/await with <code>asyncio</code> , concurrent HTTP with <code>httpx</code> , environment variables with <code>python-dotenv</code>
M05 - Python for Data	NumPy arrays, broadcasting, cosine similarity from scratch, Pandas DataFrames, groupby aggregation, missing data cleaning, end-to-end evaluation pipeline

Key Patterns to Memorise

- Always use `pathlib.Path` instead of string paths and `os.path`
- Use `@dataclass` for config objects - it eliminates boilerplate and enforces types
- Separate API keys into `.env` - never hardcode them in source files
- Prefer `enumerate(items)` over `range(len(items))` in `for` loops
- Use generators (`yield`) when processing large document collections
- Wrap external API calls with `try/except` and a retry loop
- Use `np.linalg.norm` and vectorised `@` for cosine similarity - never loop over elements
- Use `df.groupby().agg()` with named aggregations for readable summaries

What Is Next - Phase 2

Phase 2 - LLM APIs and Prompting - puts everything you have built to work. You will call real APIs (Anthropic and OpenAI SDKs), learn structured prompting patterns, implement streaming responses, use tool/function calling, and generate your first text embeddings with semantic search.

Install before Phase 2: `pip install anthropic openai python-dotenv`

You will need API keys for Anthropic and/or OpenAI. Both offer free trial credits.

Phase 2 - LLM APIs & Prompting

Libraries: `anthropic 0.97.0` · `openai 2.32.0` · `numpy 2.4.4` **Install:** `pip install anthropic openai python-dotenv numpy`

Module 06 - OpenAI & Anthropic APIs

Chat completions · Streaming · Vision · Tool calling · Token management

6.1 Setting Up

Both Anthropic and OpenAI follow the same basic pattern: install the SDK, load your API key from the environment, create a client, and call a method. Never put keys in source code.

Create a `.env` file in your project root:

```
ANTHROPIC_API_KEY=sk-ant-...  
OPENAI_API_KEY=sk-...
```

Then load it at the top of every script:

```
import os  
from dotenv import load_dotenv  
  
load_dotenv()  
  
ANTHROPIC_API_KEY = os.environ["ANTHROPIC_API_KEY"]  
OPENAI_API_KEY = os.environ["OPENAI_API_KEY"]
```

Add `.env` to `.gitignore` immediately. A leaked key can run up thousands of dollars within hours.

6.2 The Anthropic SDK

The `anthropic` package (current version 0.97.0) is the official Python client for Claude models.

Basic message call

```
import anthropic
import os
from dotenv import load_dotenv

load_dotenv()

client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

message = client.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "What is retrieval-augmented generation?"}
    ]
)

# Response text is in the first content block
print(message.content[0].text)

# Usage stats
print(f"Input tokens:{message.usage.input_tokens}")
print(f"Output tokens:{message.usage.output_tokens}")
```

System prompt

```
import anthropic, os
from dotenv import load_dotenv
```

```

load_dotenv()
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

message = client.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=512,
    system="You are a concise technical writer. Answer in plain English, no jargon.",
    messages=[
        {"role": "user", "content": "Explain what a vector database does."}
    ]
)

print(message.content[0].text)

```

Multi-turn conversation

The Anthropic API is stateless - you must send the full conversation history on every call. Build the history yourself:

```

import anthropic, os
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

def chat(system: str) -> None:
    """Simple interactive multi-turn chat loop."""
    history = []

    while True:
        user_input = input("You: ").strip()

```

```

        if user_input.lower() in ("exit", "quit"):
            break

        history.append({"role": "user", "content": user_input})

        response = client.messages.create(
            model="claude-sonnet-4-5",
            max_tokens=1024,
            system=system,
            messages=history,
        )

        assistant_text = response.content[0].text
        history.append({"role": "assistant", "content": assistant_text})

        print(f"Claude:{assistant_text}\n")

# chat(system="You are a helpful Python tutor.")

```

Streaming responses

Streaming lets you display tokens as they arrive instead of waiting for the full response - essential for chat UIs.

```

import anthropic, os
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

with client.messages.stream(
    model="claude-sonnet-4-5",
    max_tokens=512,

```

```

        messages=[{"role": "user", "content": "List 5 use cases f
or vector databases."}],
    ) as stream:
        for text in stream.text_stream:
            print(text, end="", flush=True)

print() # newline after stream ends

# Access final message and usage after stream completes
final = stream.get_final_message()
print(f"\nTotal tokens:{final.usage.input_tokens + final.usag
e.output_tokens}")

```

6.3 The OpenAI SDK

The `openai` package (current version 2.32.0) follows a nearly identical pattern.

Basic message call

```

from openai import OpenAI
import os
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

response = client.chat.completions.create(
    model="gpt-4o",
    max_tokens=1024,
    messages=[
        {"role": "system", "content": "You are a concise tech
nical assistant."},
        {"role": "user", "content": "What is the difference
between RAG and fine-tuning?"}
    ]
)

```

```
)

print(response.choices[0].message.content)
print(f"Tokens used:{response.usage.total_tokens}")
```

Streaming with OpenAI

```
from openai import OpenAI
import os
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

stream = client.chat.completions.create(
    model="gpt-4o",
    max_tokens=512,
    stream=True,
    messages=[{"role": "user", "content": "Explain embeddings
in 3 bullet points."}]
)

for chunk in stream:
    delta = chunk.choices[0].delta.content
    if delta:
        print(delta, end="", flush=True)

print()
```

6.4 Tool Calling (Function Calling)

Tool calling lets the model decide when to invoke a function you define. This is the foundation of agents: the model requests a tool, you execute it, return the result, and the model continues.

Anthropic tool calling

```
import anthropic, os, json
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

# 1. Define the tool schema
tools = [
    {
        "name": "get_model_info",
        "description": "Returns context window size and cost per 1K tokens for a given LLM.",
        "input_schema": {
            "type": "object",
            "properties": {
                "model_name": {
                    "type": "string",
                    "description": "The model identifier, e.g. 'gpt-4o' or 'claude-sonnet-4-5'."
                }
            },
            "required": ["model_name"]
        }
    }
]

# 2. The actual function the tool will call
def get_model_info(model_name: str) -> dict:
    db = {
        "claude-sonnet-4-5": {"context_k": 200, "cost_input": 3.00, "cost_output": 15.00},
        "gpt-4o": {"context_k": 128, "cost_input": 2.50, "cost_output": 10.00},
    }
```

```

        "gemini-1.5-pro": {"context_k": 1000, "cost_input":
1.25, "cost_output": 5.00},
    }
    return db.get(model_name, {"error": f"Unknown model:{mode
l_name}"})

```

```

# 3. First API call - model may return a tool_use block
response = client.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=1024,
    tools=tools,
    messages=[
        {"role": "user", "content": "How large is the context
window of claude-sonnet-4-5?"}
    ]
)

```

```

# 4. Check if model wants to use a tool
if response.stop_reason == "tool_use":
    tool_block = next(b for b in response.content if b.type ==
"tool_use")
    tool_name = tool_block.name
    tool_input = tool_block.input
    tool_use_id = tool_block.id

```

```

# 5. Execute the function
result = get_model_info(**tool_input)
print(f"Tool called:{tool_name}({tool_input})")
print(f"Tool result:{result}")

```

```

# 6. Send tool result back to the model
final = client.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=1024,
    tools=tools,
    messages=[

```



```

        {"role": "user", "content": "How large is the context window of claude-sonnet-4-5?"},
        {"role": "assistant", "content": response.content}],
    ],
    [{"role": "user",
      "content": [
        {
          "type": "tool_result",
          "tool_use_id": tool_use_id,
          "content": json.dumps(result)
        }
      ]
    }
  ]
)

print("\nFinal answer:")
print(final.content[0].text)

```

OpenAI tool calling

```

from openai import OpenAI
import os, json
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_model_info",
            "description": "Returns context window and pricing"

```

```

g for a given LLM.",
    "parameters": {
        "type": "object",
        "properties": {
            "model_name": {"type": "string", "description": "Model identifier."}
        },
        "required": ["model_name"]
    }
}
]

def get_model_info(model_name: str) -> dict:
    db = {
        "gpt-4o": {"context_k": 128, "cost_input": 2.50},
        "claude-sonnet-4-5": {"context_k": 200, "cost_input": 3.00},
    }
    return db.get(model_name, {"error": "unknown model"})

messages = [{"role": "user", "content": "What is gpt-4o's context window?"}]

response = client.chat.completions.create(
    model="gpt-4o",
    tools=tools,
    messages=messages
)

if response.choices[0].finish_reason == "tool_calls":
    tool_call = response.choices[0].message.tool_calls[0]
    name = tool_call.function.name
    args = json.loads(tool_call.function.arguments)
    result = get_model_info(**args)

```

```

# Append assistant message and tool result
messages.append(response.choices[0].message)
messages.append({
    "role": "tool",
    "tool_call_id": tool_call.id,
    "content": json.dumps(result)
})

final = client.chat.completions.create(model="gpt-4o", messages=messages)
print(final.choices[0].message.content)

```

6.5 Vision - Images as Input

Both APIs accept images alongside text in the same message.

```

import anthropic, os, base64
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

# Option A: URL (fastest)
def describe_image_url(url: str) -> str:
    response = client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=512,
        messages=[{
            "role": "user",
            "content": [
                {"type": "image", "source": {"type": "url",
"url": url}},

```

```

        {"type": "text", "text": "Describe what you
see in this image."}
    ]
}]]
)
return response.content[0].text

# Option B: base64 (for local files)
def describe_image_file(path: str) -> str:
    data = Path(path).read_bytes()
    b64 = base64.standard_b64encode(data).decode()
    ext = Path(path).suffix.lstrip(".").lower()
    media_type = f"image/{ext}" # image/png, image/jpeg, im
age/webp, image/gif

    response = client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=512,
        messages=[{
            "role": "user",
            "content": [
                {
                    "type": "image",
                    "source": {"type": "base64", "media_typ
e": media_type, "data": b64}
                },
                {"type": "text", "text": "What is in this ima
ge?"}
            ]
        }]
    )
    return response.content[0].text

# Usage:
# text = describe_image_url("https://upload.wikimedia.org/wik
ipedia/commons/thumb/1/1e/Sunrise_over_the_sea.jpg/1280px-Sun

```

```
rise_over_the_sea.jpg")
# print(text)
```

6.6 Token Management

Tokens are the billing unit. Understanding token counts prevents surprises and helps you design prompts that stay within context limits.

```
import anthropic, os
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

# Count tokens before sending (no API call made, no charge)
response = client.messages.count_tokens(
    model="claude-sonnet-4-5",
    system="You are a concise assistant.",
    messages=[{"role": "user", "content": "Explain the transformer architecture."}]
)
print(f"Estimated input tokens:{response.input_tokens}")

# Cost estimator
PRICING = {
    "claude-sonnet-4-5": {"input": 3.00, "output": 15.00},
    # per 1M tokens
    "claude-opus-4-5": {"input": 15.00, "output": 75.00},
    "gpt-4o": {"input": 2.50, "output": 10.00},
    "gpt-4o-mini": {"input": 0.15, "output": 0.60},
}

def estimate_cost(model: str, input_tokens: int, output_tokens: int) -> float:
```

```

    """Return estimated cost in USD."""
    if model not in PRICING:
        raise ValueError(f"Unknown model:{model}")
    p = PRICING[model]
    return (input_tokens * p["input"] + output_tokens * p["ou
tput"]) / 1_000_000

cost = estimate_cost("claude-sonnet-4-5", input_tokens=500, o
utput_tokens=300)
print(f"Estimated cost: ${cost:.6f}")

# Context window limits (always check before sending long doc
uments)
CONTEXT_LIMITS = {
    "claude-sonnet-4-5": 200_000,
    "claude-opus-4-5": 200_000,
    "gpt-4o": 128_000,
    "gpt-4o-mini": 128_000,
    "gemini-1.5-pro": 1_000_000,
}

def fits_in_context(model: str, token_count: int, reserve_for
_output: int = 2048) -> bool:
    limit = CONTEXT_LIMITS.get(model, 128_000)
    return token_count + reserve_for_output <= limit

```

6.7 Building a Provider-Agnostic Client

When you want to swap between Anthropic and OpenAI without rewriting your pipeline, use an abstraction layer:

```

from abc import ABC, abstractmethod
from dataclasses import dataclass
import anthropic, os
from openai import OpenAI

```

```

from dotenv import load_dotenv

load_dotenv()

@dataclass
class ChatMessage:
    role: str      # "user" or "assistant"
    content: str

@dataclass
class ChatResponse:
    text: str
    input_tokens: int
    output_tokens: int
    model: str

class BaseLLMClient(ABC):
    @abstractmethod
    def chat(
        self,
        messages: list[ChatMessage],
        system: str = "",
        max_tokens: int = 1024,
        temperature: float = 0.7,
    ) -> ChatResponse: ...

class AnthropicClient(BaseLLMClient):
    def __init__(self, model: str = "claude-sonnet-4-5"):
        self.model = model
        self._client = anthropic.Anthropic(api_key=os.environ[
            "ANTHROPIC_API_KEY"])

    def chat(self, messages, system="", max_tokens=1024, temperature=0.7) -> ChatResponse:
        resp = self._client.messages.create(
            model=self.model,

```

```

        max_tokens=max_tokens,
        system=system,
        messages=[{"role": m.role, "content": m.content}
for m in messages],
    )
    return ChatResponse(
        text=resp.content[0].text,
        input_tokens=resp.usage.input_tokens,
        output_tokens=resp.usage.output_tokens,
        model=self.model,
    )

class OpenAIClient(BaseLLMClient):
    def __init__(self, model: str = "gpt-4o"):
        self.model = model
        self._client = OpenAI(api_key=os.environ["OPENAI_API_
KEY"])

    def chat(self, messages, system="", max_tokens=1024, temp
erature=0.7) -> ChatResponse:
        api_messages = []
        if system:
            api_messages.append({"role": "system", "content":
system})
        api_messages += [{"role": m.role, "content": m.conten
t} for m in messages]

        resp = self._client.chat.completions.create(
            model=self.model,
            max_tokens=max_tokens,
            temperature=temperature,
            messages=api_messages,
        )
        return ChatResponse(
            text=resp.choices[0].message.content,
            input_tokens=resp.usage.prompt_tokens,

```



```

        output_tokens=resp.usage.completion_tokens,
        model=self.model,
    )

# Usage - same code, different backend:
# client: BaseLLMClient = AnthropicClient()
# client: BaseLLMClient = OpenAIClient()
#
# msgs = [ChatMessage(role="user", content="What is a vector
# database?")]
# result = client.chat(msgs, system="Be concise.")
# print(result.text)
# print(f"Cost estimate: {result.input_tokens} in, {result.output_tokens} out")

```

6.8 Module 06 Exercises

1. Write a function `retry_on_rate_limit(client, messages, max_retries=5)` that catches `anthropic.RateLimitError` (or `openai.RateLimitError`) and retries with exponential backoff. Test it by lowering your rate limit manually in a mock.
2. Build a `TokenBudgetManager` class that tracks cumulative token usage across multiple calls and raises a `BudgetExceeded` exception when a configured limit is reached.
3. Implement a function `compare_models(prompt: str, models: list[str]) -> pd.DataFrame` that calls the same prompt on multiple models concurrently (using `asyncio.gather`) and returns a DataFrame with columns: model, response_text, input_tokens, output_tokens, latency_ms.
4. Write a `stream_to_file(prompt: str, output_path: str)` function using the Anthropic streaming API that writes tokens to a file in real time as they arrive.

Module 07 - Prompt Engineering

System prompts · Few-shot · Chain-of-thought · Structured output · JSON mode

7.1 Why Prompt Engineering Matters

The same model can produce wildly different outputs depending on how you phrase the prompt. Prompt engineering is the practice of designing inputs that reliably produce useful, accurate, and correctly formatted outputs - without any model training.

The core principles are:

- **Be explicit** - state the format, tone, length, and constraints you want
- **Give examples** - a few well-chosen examples beat long instructions
- **Think step by step** - ask the model to reason before concluding
- **Separate concerns** - put instructions in the system prompt, data in the user message

7.2 System Prompts

The system prompt sets the model's persona, constraints, and output format for an entire conversation. Write it like an employment brief: role, responsibilities, rules, format.

```
import anthropic, os
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

# Poor system prompt - vague, no constraints
WEAK_SYSTEM = "You are an AI assistant."

# Strong system prompt - explicit role, rules, format
STRONG_SYSTEM = """You are a senior Python engineer reviewing
code for a production AI pipeline.

Your job:
- Identify bugs, security issues, and performance problems
```

- Suggest concrete improvements with code examples
- Explain WHY each issue matters

Rules:

- Be direct. Do not pad with compliments.
- If code is correct, say so briefly and move on.
- Always include the corrected code when suggesting a fix.

Format:

Return your review as a numbered list. Each item: Issue → Impact → Fix."""

```
messages = [
    {"role": "user", "content": """Review this function:

def get_user(user_id):
    key = os.getenv('DB_KEY')
    result = requests.get(f'http://db/{user_id}?key={key}')
    return result.json()"""}
]

response = client.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=1024,
    system=STRONG_SYSTEM,
    messages=messages,
)
print(response.content[0].text)
```

7.3 Few-Shot Prompting

Few-shot prompting provides examples of the desired input → output mapping inside the prompt. It is the fastest way to teach a model an unusual format or task without fine-tuning.

```

import anthropic, os
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

FEW_SHOT_SYSTEM = """You are a data extractor. Given a raw AI
benchmark result string,
extract: model name, task, and score as a JSON object.

Examples:

Input: "GPT-4o scored 87.3% on the MMLU science subset"
Output: {"model": "gpt-4o", "task": "MMLU science", "score":
87.3}

Input: "Claude Sonnet 4.5 achieved 92.1 on HumanEval"
Output: {"model": "claude-sonnet-4-5", "task": "HumanEval",
"score": 92.1}

Input: "Gemini 1.5 Pro: 78.9% accuracy on GSM8K math"
Output: {"model": "gemini-1.5-pro", "task": "GSM8K math", "score": 78.9}

Return ONLY the JSON object. No explanation."""

test_inputs = [
    "GPT-4o-mini reached 82.0% on MMLU",
    "Llama 3.1 70B: 86.4 on TruthfulQA",
    "Claude Opus 4.5 scored 96.7% on SWE-bench Verified",
]

for text in test_inputs:
    resp = client.messages.create(

```

```

model="claude-sonnet-4-5",
max_tokens=128,
system=FEW_SHOT_SYSTEM,
messages=[{"role": "user", "content": text}],
)
print(f"Input:{text}")
print(f"Output:{resp.content[0].text}\n")

```

7.4 Chain-of-Thought Prompting

Chain-of-thought (CoT) prompting asks the model to reason step by step before giving a final answer. It consistently improves accuracy on tasks requiring multi-step reasoning - math, logic, planning.

```

import anthropic, os
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

# Without CoT - model jumps to answer, more likely to be wrong
DIRECT_PROMPT = "If a model costs $3.00 per million input tokens and $15.00 per million output tokens, and a request uses 2,400 input tokens and 800 output tokens, what is the total cost in USD?"

# With CoT - model reasons through each step
COT_PROMPT = """If a model costs $3.00 per million input tokens and $15.00 per million output tokens, and a request uses 2,400 input tokens and 800 output tokens, what is the total cost in USD?

Think through this step by step before giving the final answer

```

```

r."""

# Zero-shot CoT: just adding "think step by step"
ZERO_SHOT_COT = """Solve this problem. Think step by step, showing each calculation.
Finally, state: ANSWER: $X.XXXXXX

Problem: A pipeline makes 50 API calls per hour. Each call uses an average of 1,200 input tokens
and 400 output tokens. The model costs $3.00/M input and $15.00/M output.
What is the daily cost?"""

for label, prompt in [("Direct", DIRECT_PROMPT), ("CoT", COT_PROMPT), ("Zero-shot CoT", ZERO_SHOT_COT)]:
    resp = client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=512,
        messages=[{"role": "user", "content": prompt}],
    )
    print(f"==={label} ===")
    print(resp.content[0].text[:300])
    print()

```

Structured CoT with XML tags

Claude responds well to XML-tagged reasoning sections, which makes it easy to parse the final answer programmatically:

```

import anthropic, os, re
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

```

```
SYSTEM = """Solve problems using this exact format:
```

```
<thinking>
```

```
Step-by-step reasoning here.
```

```
</thinking>
```

```
<answer>
```

```
The final answer only, no reasoning.
```

```
</answer>"""
```

```
resp = client.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=512,
    system=SYSTEM,
    messages=[{
        "role": "user",
        "content": "A RAG pipeline retrieves 5 documents, each 400 tokens. The query is 50 tokens. The model has a 4096 token limit for context. How many tokens remain for the response?"
    }],
)
```

```
text = resp.content[0].text
```

```
# Extract sections
```

```
thinking = re.search(r"<thinking>(.*?)</thinking>", text, re.DOTALL)
```

```
answer = re.search(r"<answer>(.*?)</answer>", text, re.DOTALL)
```

```
print("Reasoning:", thinking.group(1).strip() if thinking else "not found")
```

```
print("Answer: ", answer.group(1).strip() if answer else "not found")
```

7.5 Structured Output (JSON Mode)

Getting models to return valid, parseable JSON is one of the most common production requirements. There are two approaches.

Approach 1: Prompt-enforced JSON

Works with any model. The key is to give an exact schema example and instruct the model to return **only** JSON:

```
import anthropic, os, json
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

SYSTEM = """You are a data extractor. Extract information and
return ONLY a JSON object.
No markdown, no explanation, no code fences. Raw JSON only.

Schema:
{
  "company": string,
  "founded": integer or null,
  "products": [string],
  "headquarters": string or null,
  "is_public": boolean
}"""

texts = [
    "Anthropic was founded in 2021 by Dario Amodei and others. It makes Claude AI models and is headquartered in San Francisco. It is a private company.",
    "OpenAI, founded in 2015, created ChatGPT and GPT-4. Based in San Francisco, it remains private despite a major Micros
```



```

oft investment.",
]

def extract_company_info(text: str) -> dict:
    resp = client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=256,
        system=SYSTEM,
        messages=[{"role": "user", "content": text}],
    )
    raw = resp.content[0].text.strip()
    # Strip any accidental markdown fences
    raw = raw.removeprefix("`json").removeprefix("`").rem
ovesuffix("`").strip()
    return json.loads(raw)

for text in texts:
    info = extract_company_info(text)
    print(json.dumps(info, indent=2))
    print()

```

Approach 2: OpenAI JSON mode

OpenAI's API has a built-in `response_format` parameter that enforces JSON output at the API level:

```

from openai import OpenAI
import os, json
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

response = client.chat.completions.create(
    model="gpt-4o",
    response_format={"type": "json_object"}, # enforces val

```

```

id JSON
    messages=[
        {
            "role": "system",
            "content": ""Extract entities. Return JSON with
this schema:
{"people": [string], "organizations": [string], "locations":
[string]}""
        },
        {
            "role": "user",
            "content": "Elon Musk founded SpaceX in Hawthorn
e, California. He also leads Tesla."
        }
    ]
)

result = json.loads(response.choices[0].message.content)
print(result)
# {'people': ['Elon Musk'], 'organizations': ['SpaceX', 'Tesl
a'], 'locations': ['Hawthorne, California']}

```

7.6 Prompt Templates

Hard-coded prompts scattered across files are a maintenance nightmare. Centralise them as versioned templates:

```

from dataclasses import dataclass, field
from string import Formatter
from typing import Any

@dataclass
class PromptTemplate:
    """A reusable, versioned prompt template."""
    name: str

```

```

system: str
user: str
version: str = "1.0"
required_vars: list[str] = field(default_factory=list)

def __post_init__(self):
    # Auto-detect required variables from both templates
    formatter = Formatter()
    combined = self.system + self.user
    self.required_vars = [
        fname for _, fname, _, _ in formatter.parse(combined)

        if fname is not None
    ]

    def render(self, **kwargs: Any) -> tuple[str, str]:
        """Return (rendered_system, rendered_user). Raises if
        vars are missing."""
        missing = set(self.required_vars) - set(kwargs)
        if missing:
            raise ValueError(f"Missing template variables: {missing}")
        return self.system.format(**kwargs), self.user.format(**kwargs)

# Define templates as constants - easy to version and test
QA_TEMPLATE = PromptTemplate(
    name="question_answering",
    version="1.2",
    system="""You are a{domain} expert. Answer questions accurately and concisely.
    Cite sources when possible. If you are unsure, say so.""",
    user="Question:{question}\n\nContext:\n{context}",
)

```

```

SUMMARY_TEMPLATE = PromptTemplate(
    name="document_summary",
    version="1.0",
    system="You are a technical writer. Summarise documents clearly for a{audience} audience.",
    user="Summarise the following in{max_sentences} sentences or fewer:\n\n{document}",
)

# Usage
system, user = QA_TEMPLATE.render(
    domain="machine learning",
    question="What is the vanishing gradient problem?",
    context="Gradients in deep networks are computed via back propagation...",
)
print("System:", system)
print("User:", user)
print("Required vars:", QA_TEMPLATE.required_vars)

```

7.7 Prompt Evaluation

A prompt that works once is not a prompt - it is luck. Systematically evaluate prompts across a test set before using them in production:

```

import anthropic, os, json
from dataclasses import dataclass
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

@dataclass
class EvalCase:

```

```

    input_text: str
    expected_keywords: list[str]    # at least one must appear
in response
    must_be_json: bool = False

def evaluate_prompt(system: str, cases: list[EvalCase]) -> dict:
    """Run a prompt against test cases and return pass rate +
    details."""
    results = []

    for case in cases:
        resp = client.messages.create(
            model="claude-sonnet-4-5",
            max_tokens=256,
            system=system,
            messages=[{"role": "user", "content": case.input_
text}],
        )
        text = resp.content[0].text.strip()

        # Check keyword hit
        keyword_hit = any(kw.lower() in text.lower() for kw i
n case.expected_keywords)

        # Check JSON validity if required
        json_valid = True
        if case.must_be_json:
            try:
                json.loads(text)
            except json.JSONDecodeError:
                json_valid = False

        passed = keyword_hit and json_valid
        results.append({
            "input": case.input_text[:60],

```

```

        "passed": passed,
        "response_preview": text[:80],
    })

    pass_rate = sum(r["passed"] for r in results) / len(results)
    return {"pass_rate": pass_rate, "results": results}

# Test a classification prompt
CLASSIFY_SYSTEM = """Classify the AI task as one of: CLASSIFICATION, GENERATION, RETRIEVAL, EMBEDDING.
Return ONLY the category word."""

test_cases = [
    EvalCase("Predict whether an email is spam.", ["CLASSIFICATION"]),
    EvalCase("Write a product description for headphones.", ["GENERATION"]),
    EvalCase("Find the most relevant documents for a query.", ["RETRIEVAL"]),
    EvalCase("Convert this sentence to a vector.", ["EMBEDDING"]),
    EvalCase("Label customer reviews as positive or negative.", ["CLASSIFICATION"]),
]

report = evaluate_prompt(CLASSIFY_SYSTEM, test_cases)
print(f"Pass rate:{report['pass_rate']:.0%}")
for r in report["results"]:
    status = "PASS" if r["passed"] else "FAIL"
    print(f" [{status}]{r['input']!r} →{r['response_preview']!r}")

```

7.8 Module 07 Exercises

1. Write three versions of a system prompt for a "code review assistant" - basic, intermediate, and expert-level. Evaluate all three on the same 5 code snippets and compare output quality.
2. Build a `PromptLibrary` class that stores named `PromptTemplate` instances, supports saving/loading to JSON, and tracks which version of each template produced the last evaluation run.
3. Implement automatic JSON repair: a function `safe_json_parse(text: str) -> dict` that first tries `json.loads`, then strips markdown fences, then asks the model to fix the JSON if it is still invalid.
4. Design a CoT prompt for the following task: given a list of 10 LLM evaluation scores across 3 tasks, rank the models and write a 2-sentence recommendation. Verify it works correctly on at least 3 different inputs.

Module 08 - Embeddings & Semantic Search

Text embeddings · Cosine similarity · Nearest-neighbour search · Building a vector store

8.1 What Are Embeddings?

An embedding is a fixed-length vector of floating-point numbers that represents a piece of text. Semantically similar texts produce vectors that are geometrically close. This property powers semantic search, clustering, deduplication, and the retrieval step in RAG.

```
"What is RAG?"           → [0.021, -0.143, 0.892, ...] ← 1536 numbers
"Retrieval Augmented Generation" → [0.019, -0.139, 0.881, ...] ← geometrically close
"Who won the cricket match?" → [-0.312, 0.401, -0.203, ...] ← geometrically far
```

Key properties:

- **Dimensionality:** 768 to 3072 dimensions depending on the model
 - **Magnitude:** vectors are usually L2-normalised (length = 1)
 - **Comparison:** cosine similarity is the standard metric (dot product of normalised vectors)
-

8.2 Generating Embeddings

OpenAI embeddings

```
from openai import OpenAI
import os, numpy as np
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

def embed(texts: list[str], model: str = "text-embedding-3-small") -> np.ndarray:
    """Embed a list of texts. Returns array of shape (n, dim)."""
    # API accepts up to 2048 texts per call
    response = client.embeddings.create(input=texts, model=model)
    # Sort by index to guarantee order matches input
    vectors = sorted(response.data, key=lambda e: e.index)
    return np.array([v.embedding for v in vectors], dtype=np.float32)

texts = [
    "Retrieval-Augmented Generation combines search with LLMs.",
    "RAG retrieves documents then generates an answer from them.",
    "The Eiffel Tower is in Paris.",
    "Python is a popular programming language.",
    "Fine-tuning trains a model on new data.",
]

embeddings = embed(texts)
print(f"Shape:{embeddings.shape}")      # (5, 1536)
```



```
print(f"Norm of first vector:{np.linalg.norm(embeddings[0]):.4f}") # ≈ 1.0
```

Anthropic embeddings (via Voyage AI)

Anthropic recommends Voyage AI for embeddings when using Claude. Install with

```
pip install voyageai:
```

```
# pip install voyageai
import voyageai, os
import numpy as np
from dotenv import load_dotenv

load_dotenv()

vo = voyageai.Client(api_key=os.environ["VOYAGE_API_KEY"])

result = vo.embed(
    ["What is RAG?", "Explain vector databases."],
    model="voyage-3",          # current recommended model
    input_type="document",     # "document" for corpus, "query" for search queries
)

embeddings = np.array(result.embeddings, dtype=np.float32)
print(f"Shape:{embeddings.shape}") # (2, 1024)
print(f"Token usage:{result.total_tokens}")
```

Both OpenAI `text-embedding-3-small` and Voyage `voyage-3` are excellent general-purpose choices. Use `text-embedding-3-small` if you are already on OpenAI; use `voyage-3` for Anthropic/Claude stacks.

8.3 Cosine Similarity

```

import numpy as np

def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    """Cosine similarity between two 1-D vectors."""
    norm_a = np.linalg.norm(a)
    norm_b = np.linalg.norm(b)
    if norm_a == 0 or norm_b == 0:
        return 0.0
    return float(np.dot(a, b) / (norm_a * norm_b))

def pairwise_similarity(matrix: np.ndarray) -> np.ndarray:
    """
    Compute pairwise cosine similarity for all rows in a matrix.
    Returns an (n, n) matrix. All rows must be non-zero.
    """
    # Normalise all rows to unit length first
    norms = np.linalg.norm(matrix, axis=1, keepdims=True)
    norms = np.where(norms == 0, 1, norms)
    normed = matrix / norms
    # Dot product of normalised vectors = cosine similarity
    return (normed @ normed.T).astype(np.float32)

# Example
rng = np.random.default_rng(42)
vecs = rng.standard_normal((4, 8)).astype(np.float32)
sim_matrix = pairwise_similarity(vecs)

print("Pairwise similarities:")
for i in range(4):
    for j in range(i+1, 4):
        print(f"  vec[{i}] vs vec[{j}]:{sim_matrix[i, j]:.4f}")

```

```
# Diagonal is always 1.0 (a vector is identical to itself)
print(f"Diagonal:{np.diag(sim_matrix)}")
```

8.4 Semantic Search - Full Pipeline

This is the core of a RAG retrieval step. Given a query, find the k most semantically relevant documents from a corpus.

```
import numpy as np
from dataclasses import dataclass, field
from typing import Optional
from openai import OpenAI
import os
from dotenv import load_dotenv

load_dotenv()
openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

# — Data structures —


---



@dataclass
class Document:
    id: str
    text: str
    metadata: dict = field(default_factory=dict)
    embedding: Optional[np.ndarray] = field(default=None, repr=False)

@dataclass
class SearchResult:
    document: Document
    score: float
    rank: int
```

— Embedding helper —————

```
def embed_batch(texts: list[str], model: str = "text-embedding-3-small") -> np.ndarray:
    """Embed texts in a single API call. Returns (n, dim) float32 array."""
    response = openai_client.embeddings.create(input=texts, model=model)
    vectors = sorted(response.data, key=lambda e: e.index)
    return np.array([v.embedding for v in vectors], dtype=np.float32)
```

— Simple in-memory vector store —————

```
class VectorStore:
    """
    In-memory vector store for semantic search.
    Suitable for corpora up to ~100k documents.
    For larger collections use ChromaDB, Pinecone, or pgvector (covered in Phase 3).
    """

    def __init__(self, embed_model: str = "text-embedding-3-small"):
        self.embed_model = embed_model
        self._documents: list[Document] = []
        self._matrix: Optional[np.ndarray] = None  # (n, dim) normalised

    def add_documents(self, documents: list[Document]) -> None:
        """Embed and index a list of documents."""
        texts = [d.text for d in documents]
        vectors = embed_batch(texts, model=self.embed_model)
```

```

        # Normalise for fast cosine similarity via dot product
        norms = np.linalg.norm(vectors, axis=1, keepdims=True)
        norms = np.where(norms == 0, 1, norms)
        normed = (vectors / norms).astype(np.float32)

        for doc, vec in zip(documents, normed):
            doc.embedding = vec
            self._documents.append(doc)

        # Rebuild the full matrix
        self._matrix = np.array(
            [d.embedding for d in self._documents], dtype=np.
float32
        )
        print(f"Index now contains{len(self._documents)} documents.")

    def search(self, query: str, k: int = 5) -> list[SearchResult]:
        """Return the k most similar documents for a query string."""
        if self._matrix is None or len(self._documents) == 0:
            raise RuntimeError("No documents indexed yet.")

        # Embed and normalise query
        q_vec = embed_batch([query], model=self.embed_model)

        q_norm = np.linalg.norm(q_vec)
        if q_norm == 0:
            return []
        q_vec = (q_vec / q_norm).astype(np.float32)

        # Cosine similarities: matrix @ vector -> shape (n,)

```

```

scores = self._matrix @ q_vec

# Top-k indices (descending)
k = min(k, len(self._documents))
top_idx = np.argsort(scores)[::-1][:k]

return [
    SearchResult(
        document=self._documents[int(i)],
        score=float(scores[i]),
        rank=rank + 1,
    )
    for rank, i in enumerate(top_idx)
]

@property
def size(self) -> int:
    return len(self._documents)

```

— Demo —

```

CORPUS = [
    Document("d01", "Retrieval-Augmented Generation (RAG) combines information retrieval with language model generation to answer questions using external knowledge."),
    Document("d02", "Vector databases store high-dimensional embeddings and enable fast approximate nearest-neighbour search using algorithms like HNSW and IVF."),
    Document("d03", "Fine-tuning adapts a pre-trained language model to a specific task by continuing training on a curated dataset with task-specific examples."),
    Document("d04", "Prompt engineering involves designing and optimising input prompts to guide language models toward producing the desired output."),

```

```

    Document("d05", "LangChain is a Python framework that provides abstractions for building applications with large language models, including chains, agents, and memory."),
    Document("d06", "Cosine similarity measures the angle between two vectors and is the standard metric for comparing text embeddings in semantic search."),
    Document("d07", "RLHF (Reinforcement Learning from Human Feedback) aligns language models with human preferences by training a reward model on human rankings."),
    Document("d08", "Chunking strategies for RAG include fixed-size chunks, sentence-aware splits, and recursive character splitting with configurable overlap."),
    Document("d09", "The transformer architecture uses self-attention mechanisms to model relationships between all tokens in a sequence simultaneously."),
    Document("d10", "Agents use language models as a reasoning engine, enabling them to plan multi-step tasks, call tools, and take actions based on observations."),
]

store = VectorStore()
store.add_documents(CORPUS)

QUERIES = [
    "How does RAG work?",
    "What algorithms do vector databases use?",
    "How do I split documents for embedding?",
]

for query in QUERIES:
    print(f"\nQuery:{query!r}")
    results = store.search(query, k=3)
    for r in results:
        print(f"    [{r.rank}] score={r.score:.4f} | {r.document.text[:80]}...")

```

8.5 Chunking Text for Embedding

Real documents are too long to embed whole. Chunk them into pieces that fit within the embedding model's token limit (typically 512–8192 tokens depending on the model).

```
from dataclasses import dataclass
import re

@dataclass
class Chunk:
    doc_id: str
    chunk_index: int
    text: str
    char_start: int
    char_end: int

def chunk_by_sentences(
    text: str,
    doc_id: str,
    max_chars: int = 1000,
    overlap_chars: int = 100,
) -> list[Chunk]:
    """
    Split text into chunks that respect sentence boundaries.
    Adds overlap so context is not lost at chunk edges.
    """
    # Split on sentence endings
    sentences = re.split(r'(?<=[.!?])\s+', text.strip())
    chunks: list[Chunk] = []
    current = ""
    current_start = 0
    char_offset = 0
    chunk_idx = 0
```



```

for sentence in sentences:
    candidate = (current + " " + sentence).strip() if current else sentence

    if len(candidate) > max_chars and current:
        # Save current chunk
        end = char_offset + len(current)
        chunks.append(Chunk(doc_id, chunk_idx, current.strip(), char_offset, end))
        chunk_idx += 1

        # Start new chunk with overlap from end of previous
        overlap_start = max(0, len(current) - overlap_chars)

        overlap_text = current[overlap_start:]
        current = (overlap_text + " " + sentence).strip()

        char_offset = end - len(overlap_text)
    else:
        current = candidate

# Save final chunk
if current.strip():
    end = char_offset + len(current)
    chunks.append(Chunk(doc_id, chunk_idx, current.strip(), char_offset, end))

return chunks

```

Test with a sample document

document = """

Large language models (LLMs) are neural networks trained on vast amounts of text data.

They learn to predict the next token in a sequence, which gives them broad language understanding.

Models like GPT-4 and Claude are examples of LLMs used in production today.

Retrieval-Augmented Generation, or RAG, extends LLMs by connecting them to external knowledge bases.

Instead of relying solely on knowledge encoded during training, a RAG system retrieves relevant documents at inference time.

This allows the model to answer questions about recent events or private data it was never trained on.

The retrieval step in RAG typically uses embedding-based semantic search.

A query is embedded into a vector, and the nearest document vectors are retrieved from a database.

These documents are then injected into the LLM's context window alongside the query.

"""

```
chunks = chunk_by_sentences(document.strip(), doc_id="intro_to_llms", max_chars=300, overlap_chars=50)
for c in chunks:
    print(f"Chunk{c.chunk_index} ({c.char_start}-{c.char_end}):{c.text[:80]}...")
    print()
```

8.6 Evaluating Retrieval Quality

Before connecting retrieval to a generator, verify that the right documents are actually being retrieved.

```
import numpy as np
from dataclasses import dataclass

@dataclass
```

```

class RetrievalEvalCase:
    query: str
    relevant_doc_ids: list[str] # ground-truth relevant documents

def precision_at_k(retrieved_ids: list[str], relevant_ids: list[str], k: int) -> float:
    """Fraction of top-k results that are relevant."""
    top_k = retrieved_ids[:k]
    hits = sum(1 for doc_id in top_k if doc_id in relevant_ids)
    return hits / k

def recall_at_k(retrieved_ids: list[str], relevant_ids: list[str], k: int) -> float:
    """Fraction of all relevant docs found in top-k."""
    if not relevant_ids:
        return 0.0
    top_k = retrieved_ids[:k]
    hits = sum(1 for doc_id in top_k if doc_id in relevant_ids)
    return hits / len(relevant_ids)

def mean_reciprocal_rank(retrieved_ids: list[str], relevant_ids: list[str]) -> float:
    """MRR: reciprocal of the rank of the first relevant result."""
    for rank, doc_id in enumerate(retrieved_ids, start=1):
        if doc_id in relevant_ids:
            return 1.0 / rank
    return 0.0

def evaluate_retrieval(
    store, # VectorStore from section 8.4
    eval_cases: list[RetrievalEvalCase],
    k: int = 5,

```

```

) -> dict:
    """Run all eval cases and return aggregate metrics."""
    p_scores, r_scores, mrr_scores = [], [], []

    for case in eval_cases:
        results = store.search(case.query, k=k)
        retrieved_ids = [r.document.id for r in results]

        p_scores.append(precision_at_k(retrieved_ids, case.relevant_doc_ids, k))
        r_scores.append(recall_at_k(retrieved_ids, case.relevant_doc_ids, k))
        mrr_scores.append(mean_reciprocal_rank(retrieved_ids, case.relevant_doc_ids))

    return {
        f"precision@{k}": round(float(np.mean(p_scores))),
4),
        f"recall@{k}": round(float(np.mean(r_scores))),
4),
        "MRR": round(float(np.mean(mrr_scores))),
4),
    }

# Using the VectorStore and CORPUS from section 8.4
eval_cases = [
    RetrievalEvalCase("How does RAG work?",
["d01", "d08"]),
    RetrievalEvalCase("What are vector databases?",
["d02", "d06"]),
    RetrievalEvalCase("How do agents use language models?",
["d10"]),
    RetrievalEvalCase("What is fine-tuning?",
["d03"]),
    RetrievalEvalCase("How do transformers model token relationships?", ["d09"]),

```

```

]

# metrics = evaluate_retrieval(store, eval_cases, k=3)
# print(metrics)
# Example output: {'precision@3': 0.8, 'recall@3': 0.75, 'MR
R': 0.9167}

```

8.7 Metadata Filtering

Real vector stores combine semantic similarity with structured metadata filters - for example, retrieve only documents from a specific date range, author, or category.

```

from dataclasses import dataclass, field
from typing import Any, Callable, Optional
import numpy as np
from openai import OpenAI
import os
from dotenv import load_dotenv

load_dotenv()
openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

@dataclass
class FilteredDocument:
    id: str
    text: str
    metadata: dict[str, Any] = field(default_factory=dict)
    embedding: Optional[np.ndarray] = field(default=None, repr=False)

def embed_texts(texts: list[str]) -> np.ndarray:
    resp = openai_client.embeddings.create(input=texts, model="text-embedding-3-small")
    vecs = np.array([e.embedding for e in sorted(resp.data, k

```

```

ey=lambda x: x.index)], dtype=np.float32)
    norms = np.linalg.norm(vecs, axis=1, keepdims=True)
    return vecs / np.where(norms == 0, 1, norms)

class FilteredVectorStore:
    def __init__(self):
        self._docs: list[FilteredDocument] = []

    def add(self, docs: list[FilteredDocument]) -> None:
        embeddings = embed_texts([d.text for d in docs])
        for doc, emb in zip(docs, embeddings):
            doc.embedding = emb
            self._docs.append(doc)

    def search(
        self,
        query: str,
        k: int = 5,
        filter_fn: Optional[Callable[[FilteredDocument], bool]] = None,
    ) -> list[tuple[FilteredDocument, float]]:
        """Search with optional metadata filter applied before ranking."""

        # Apply pre-filter
        candidates = self._docs if filter_fn is None else [d
for d in self._docs if filter_fn(d)]
        if not candidates:
            return []

        # Embed query
        q_vec = embed_texts([query])[0]

        # Score
        matrix = np.array([d.embedding for d in candidates],
dtype=np.float32)

```

```

scores = matrix @ q_vec

# Top-k
k = min(k, len(candidates))
top_idx = np.argsort(scores)[::-1][:k]
return [(candidates[i], float(scores[i])) for i in to
p_idx]

# Demo - a mixed corpus with category metadata
docs = [
    FilteredDocument("a1", "GPT-4o supports vision and functi
on calling.", {"category": "openai", "year": 2024}),
    FilteredDocument("a2", "Claude 3.5 Sonnet excels at codin
g tasks.", {"category": "anthropic", "year": 2024}),
    FilteredDocument("a3", "GPT-4o-mini is a smaller, cheaper
model.", {"category": "openai", "year": 2024}),
    FilteredDocument("a4", "Claude Opus 4 is Anthropic's most
capable model.", {"category": "anthropic", "year": 2025}),
    FilteredDocument("a5", "GPT-4 Turbo has a 128K context wi
ndow.", {"category": "openai", "year": 2023}),
]

fstore = FilteredVectorStore()
fstore.add(docs)

# Search across all docs
print("=== All docs ===")
results = fstore.search("which model is good at coding?", k=
3)
for doc, score in results:
    print(f" [{score:.4f}]{doc.id}:{doc.text}")

# Search only Anthropic docs
print("\n=== Anthropic only ===")
results = fstore.search("which model is good at coding?", k=

```

```
3,
                                filter_fn=lambda d: d.metadata["category"] == "anthropic")
for doc, score in results:
    print(f" [{score:.4f}]{doc.id}:{doc.text}")
```

8.8 Module 08 Exercises

1. Implement a `DuplicateDetector` class that takes a list of documents, embeds them, and returns pairs with cosine similarity above a configurable threshold (e.g., 0.95). Use it to find near-duplicate entries in a 50-document corpus.
2. Build a `HybridSearch` class that combines semantic similarity (embedding cosine score) with a simple BM25-style keyword score. Implement a `alpha` parameter that blends the two scores: `final_score = alpha * semantic + (1 - alpha) * keyword`.
3. Extend the `VectorStore` from section 8.4 to support `delete(doc_id)` and `update(doc_id, new_text)` operations. Ensure the internal matrix stays consistent after each operation.
4. Implement `embed_with_cache`: a wrapper around the OpenAI embeddings API that stores results in a local SQLite database (keyed by SHA-256 of the text + model name) so repeated calls for the same text never hit the API twice.

Phase 2 Summary

What You Learned

Module	Core Skills
M06 - LLM APIs	Anthropic and OpenAI clients, multi-turn conversations, streaming, tool calling, vision inputs, token counting and cost estimation, provider-agnostic abstraction
M07 - Prompt Engineering	System prompts, few-shot examples, chain-of-thought, XML-tagged reasoning, structured JSON output, <code>PromptTemplate</code> with validation, systematic prompt evaluation

Module	Core Skills
M08 - Embeddings	Generating embeddings (OpenAI, Voyage AI), cosine similarity from scratch, in-memory vector store, sentence-aware chunking, Precision@k / Recall@k / MRR evaluation, metadata filtering

Key Patterns to Carry Forward

- The API is stateless - always send the full conversation history on every call
- Put instructions in the system prompt, dynamic data in the user message; keep them separated
- Always evaluate prompts on a test set - a prompt that works once is not a prompt
- Normalise embedding vectors to unit length before computing cosine similarity; then similarity = dot product
- Chunk documents before embedding - never embed a full multi-page document as one vector
- Build a retrieval eval set (Precision@k, Recall@k, MRR) before wiring retrieval into a RAG pipeline
- Abstract the LLM provider behind a `BaseLLMClient` - swapping providers becomes a one-line change

What Is Next - Phase 3

Phase 3 covers the major GenAI libraries one by one:

- **LangChain** - chains, LCEL, document loaders, output parsers, memory
- **LlamaIndex** - index types, query engines, retrieval pipelines, data connectors
- **Vector Databases** - ChromaDB, Pinecone, Weaviate - production-grade vector storage
- **Hugging Face Transformers** - local models, pipelines, tokenizers, PEFT/LoRA fine-tuning
- **LangGraph** - agentic loops, stateful graphs, multi-step tool use

- **CrewAI** - multi-agent crews with roles, tasks, and delegation

Install before Phase 3: `pip install langchain langchain-openai langchain-anthropic llama-index chromadb transformers`

Phase 3 - GenAI Libraries

Libraries: langchain 1.2 · llama-index 0.14 · chromadb 1.1 · transformers 5.6 ·
langgraph · crewai 1.14 **Install:** pip install langchain langchain-openai langchain-
anthropic langchain-community llama-index llama-index-llms-openai llama-index-embeddings-
openai chromadb transformers langgraph crewai

Module 09 - LangChain

Chains · LCEL · Prompts · Memory · Document Loaders · Output Parsers

9.1 What LangChain Is

LangChain is a framework for composing LLM-powered applications from reusable building blocks. Its core idea is the **chain** - a sequence of steps where the output of one step feeds into the next. The modern way to build chains is **LCEL** (LangChain Expression Language), a pipe-based syntax that makes composition explicit and readable.

LangChain does not replace the Anthropic or OpenAI SDKs - it wraps them with a uniform interface and adds utilities for prompts, memory, document processing, and retrieval.

When to use LangChain: RAG pipelines, conversational applications, document processing workflows, and any situation where you need to chain multiple LLM calls with structured prompts.

9.2 Chat Models

```
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
from langchain_core.messages import HumanMessage, SystemMessage
import os
from dotenv import load_dotenv
```

```

load_dotenv()

# OpenAI model
llm_oai = ChatOpenAI(
    model="gpt-4o",
    temperature=0.7,
    max_tokens=1024,
    api_key=os.environ["OPENAI_API_KEY"],
)

# Anthropic model - same interface
llm_ant = ChatAnthropic(
    model="claude-sonnet-4-5",
    temperature=0.7,
    max_tokens=1024,
    api_key=os.environ["ANTHROPIC_API_KEY"],
)

# Both accept the same message types
messages = [
    SystemMessage(content="You are a concise technical assistant."),
    HumanMessage(content="What is LangChain Expression Language?"),
]

# Invoke (blocking)
response = llm_ant.invoke(messages)
print(response.content)

# Stream
for chunk in llm_ant.stream(messages):
    print(chunk.content, end="", flush=True)
print()

# Batch - run multiple inputs in parallel

```

```

responses = llm_ant.batch([
    [HumanMessage(content="What is RAG?")],
    [HumanMessage(content="What is a vector database?")],
])
for r in responses:
    print(r.content[:80])

```

9.3 Prompt Templates

```

from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import HumanMessage

# Simple template
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a{role}. Answer in{language}.",),
    ("human", "{question}"),
])

# Format to messages (for inspection)
msgs = prompt.format_messages(
    role="Python tutor",
    language="English",
    question="What is a generator?",
)
print(msgs)

# Template with conversation history placeholder
chat_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant. Today's context: {context}"),
    MessagesPlaceholder(variable_name="history"),
    ("human", "{input}"),
])

```

```

formatted = chat_prompt.format_messages(
    context="User is learning about RAG",
    history=[
        HumanMessage(content="What is retrieval?"),
    ],
    input="How is it different from fine-tuning?",
)
print(f"Total messages in formatted prompt:{len(formatted)}")

```

9.4 LCEL - LangChain Expression Language

LCEL uses the `|` pipe operator to compose runnables. Every component - prompts, models, parsers, retrievers - implements the `Runnable` interface with `invoke`, `stream`, and `batch`.

```

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
import os
from dotenv import load_dotenv

load_dotenv()

llm = ChatOpenAI(model="gpt-4o", api_key=os.environ["OPENAI_API_KEY"])

# Basic chain: prompt | model | parser
prompt = ChatPromptTemplate.from_messages([
    ("system", "Summarise the following text in exactly{sente
nces} sentences."),
    ("human", "{text}"),
])

chain = prompt | llm | StrOutputParser()

```

```

result = chain.invoke({
    "sentences": 2,
    "text": """LangChain is a framework for developing applications powered by large language models.
    It provides abstractions for prompts, models, memory, and retrieval so developers can
    compose complex AI workflows from simple building blocks without writing boilerplate."""
})
print(result)

```

Branching with RunnablePassthrough

```

from langchain_core.runnables import RunnablePassthrough, RunnableLambda, RunnableParallel
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
import os
from dotenv import load_dotenv

load_dotenv()

llm = ChatOpenAI(model="gpt-4o", api_key=os.environ["OPENAI_API_KEY"])

# Run two chains in parallel and combine results
summarise_prompt = ChatPromptTemplate.from_messages([
    ("system", "Summarise this in one sentence."),
    ("human", "{text}"),
])

keywords_prompt = ChatPromptTemplate.from_messages([
    ("system", "Extract 3 keywords from this text. Return the m comma-separated."),

```

```

        ("human", "{text}"),
    ])

    parser = StrOutputParser()

    parallel_chain = RunnableParallel(
        summary = summarise_prompt | llm | parser,
        keywords = keywords_prompt | llm | parser,
    )

    result = parallel_chain.invoke({
        "text": "Retrieval-Augmented Generation (RAG) improves LL
M answers by fetching relevant documents at inference time."
    })

    print("Summary: ", result["summary"])
    print("Keywords:", result["keywords"])

```

Conditional routing

```

from langchain_core.runnables import RunnableLambda, Runnable
Passthrough
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
import os
from dotenv import load_dotenv

load_dotenv()

llm = ChatOpenAI(model="gpt-4o", api_key=os.environ["OPENAI_A
PI_KEY"])
parser = StrOutputParser()

code_prompt = ChatPromptTemplate.from_messages([

```



```

    ("system", "You are a Python expert. Provide a complete,
working code example."),
    ("human", "{input}"),
])

prose_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a clear technical writer. Explain con
ceptually."),
    ("human", "{input}"),
])

code_chain = code_prompt | llm | parser
prose_chain = prose_prompt | llm | parser

def route(input_dict: dict) -> str:
    """Route to code or prose chain based on the question."""
    q = input_dict["input"].lower()
    if any(w in q for w in ["write", "code", "implement", "ex
ample", "function"]):
        return "code"
    return "prose"

def run_routed(input_dict: dict) -> str:
    choice = route(input_dict)
    if choice == "code":
        return code_chain.invoke(input_dict)
    return prose_chain.invoke(input_dict)

router_chain = RunnableLambda(run_routed)

print(router_chain.invoke({"input": "Write a function to chun
k text with overlap."}))
print(router_chain.invoke({"input": "What is attention in tra
nsformers?"}))

```

9.5 Output Parsers

```
from langchain_core.output_parsers import StrOutputParser, Js
onOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field
from typing import List
import os
from dotenv import load_dotenv

load_dotenv()

llm = ChatOpenAI(model="gpt-4o", api_key=os.environ["OPENAI_A
PI_KEY"])

# Pydantic schema for structured output
class ModelComparison(BaseModel):
    models: List[str] = Field(description="Model
names compared")
    best_for_cost: str = Field(description="Most
cost-effective model")
    best_for_quality: str = Field(description="Highe
st quality model")
    recommendation: str = Field(description="Overa
ll recommendation")

# Use with_structured_output for schema-enforced responses (O
penAI)
structured_llm = llm.with_structured_output(ModelComparison)

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an AI model comparison expert."),
    ("human", "Compare these models for a startup on a budge
t:{models}"),
])
```

```

chain = prompt | structured_llm

result: ModelComparison = chain.invoke({
    "models": "gpt-4o, gpt-4o-mini, claude-sonnet-4-5"
})

print(f"Best for cost:{result.best_for_cost}")
print(f"Best for quality:{result.best_for_quality}")
print(f"Recommendation:{result.recommendation}")

```

9.6 Document Loaders and Text Splitters

```

from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Simulate loading documents (in practice use DirectoryLoader, PyPDFLoader, etc.)
raw_docs = [
    Document(
        page_content="""LangChain provides a standard interface for interacting with
        large language models and related tools. It simplifies the development of
        context-aware reasoning applications by combining prompts, models, memory,
        and retrieval in composable pipelines. LCEL makes these pipelines
        declarative and easy to debug.""",
        metadata={"source": "langchain_docs.txt", "page": 1}
    ),
    Document(
        page_content="""Retrieval-Augmented Generation (RAG) is a pattern that combines

```

a retriever with a generator. The retriever finds relevant documents from a

knowledge base. Those documents are injected into the generator's context window.

This allows models to answer questions about information they were never trained on.""",

```
        metadata={"source": "rag_guide.txt", "page": 1}
    ),
]

# Split into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=200,
    chunk_overlap=40,
    length_function=len,
    separators=["\n\n", "\n", ". ", " ", ""],
)

chunks = splitter.split_documents(raw_docs)
print(f"Original docs:{len(raw_docs)}, Chunks:{len(chunks)}")
for i, chunk in enumerate(chunks):
    print(f"  Chunk{i}:{len(chunk.page_content)} chars | source:{chunk.metadata['source']}")
```

9.7 Conversational Memory

LangChain provides memory abstractions that maintain conversation history across turns in a structured way.

```
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
```

```

from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
import os
from dotenv import load_dotenv

load_dotenv()

llm = ChatOpenAI(model="gpt-4o", api_key=os.environ["OPEN
AI_API_KEY"])
parser = StrOutputParser()

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful AI tutor. Be concise."),
    MessagesPlaceholder(variable_name="history"),
    ("human", "{input}"),
])

chain = prompt | llm | parser

# Session store - keyed by session_id
store: dict[str, InMemoryChatMessageHistory] = {}

def get_session_history(session_id: str) -> InMemoryChatMessa
geHistory:
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

# Wrap chain with history management
chain_with_history = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key="input",
    history_messages_key="history",
)

```

```

config = {"configurable": {"session_id": "user-123"}}

# Turn 1
r1 = chain_with_history.invoke({"input": "What is RAG?"}, config=config)
print("Turn 1:", r1[:100])

# Turn 2 - model remembers turn 1
r2 = chain_with_history.invoke({"input": "What are the main components I just asked about?"}, config=config)
print("Turn 2:", r2[:100])

# Inspect stored history
history = get_session_history("user-123")
print(f"Messages in history:{len(history.messages)}")

```

9.8 Building a Full RAG Chain with LangChain

```

from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough
from langchain_core.documents import Document
from langchain_community.vectorstores import Chroma
from langchain_text_splitters import RecursiveCharacterTextSplitter
import os
from dotenv import load_dotenv

load_dotenv()

# 1. Documents
docs = [
    Document(page_content="RAG stands for Retrieval-Augmented Generation. It connects LLMs to external knowledge bases."),

```

```

    Document(page_content="The retrieval step uses embeddings
and vector search to find relevant documents for a given quer
y."),
    Document(page_content="The generation step takes the retr
ieved context and the user question to produce a grounded ans
wer."),
    Document(page_content="Vector databases like ChromaDB, Pi
necone, and Weaviate store embeddings for fast similarity sea
rch."),
    Document(page_content="Fine-tuning bakes knowledge into m
odel weights. RAG retrieves knowledge at inference time. Both
have trade-offs."),
]

```

2. Split

```

splitter = RecursiveCharacterTextSplitter(chunk_size=200, chu
nk_overlap=30)
chunks   = splitter.split_documents(docs)

```

3. Embed and store in Chroma (in-memory)

```

embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small",
    api_key=os.environ["OPENAI_API_KEY"],
)
vectorstore = Chroma.from_documents(chunks, embeddings)
retriever   = vectorstore.as_retriever(search_kwargs={"k":
3})

```

4. RAG prompt

```

rag_prompt = ChatPromptTemplate.from_messages([
    ("system", """"You are a helpful assistant. Answer the que
stion using ONLY the provided context.
If the context does not contain the answer, say "I don't have
that information."

```

Context:

```

{context}"""),
    ("human", "{question}"),
])

# 5. Chain
llm = ChatOpenAI(model="gpt-4o", api_key=os.environ["OPENAI_API_KEY"])
parser = StrOutputParser()

def format_docs(docs: list[Document]) -> str:
    return "\n\n".join(d.page_content for d in docs)

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | rag_prompt
    | llm
    | parser
)

# 6. Ask questions
questions = [
    "What does RAG stand for?",
    "How is RAG different from fine-tuning?",
    "What is the capital of France?", # Not in context - should say so
]

for q in questions:
    print(f"Q:{q}")
    print(f"A:{rag_chain.invoke(q)}\n")

```

9.9 Module 09 Exercises

1. Build an LCEL chain that: (a) takes a raw document text, (b) extracts 3 key facts using one LLM call, (c) generates a quiz question for each fact using a second call, and (d) returns the questions as a Python list.
 2. Implement a `MultiQueryRetriever` manually: for a given query, generate 3 paraphrased versions of it, retrieve documents for each, deduplicate by content hash, and return the union.
 3. Build a summarisation chain that handles documents longer than the model's context window by splitting into chunks, summarising each chunk, then summarising the summaries (map-reduce pattern).
 4. Add a response-quality filter to the RAG chain from section 9.8: after the answer is generated, run a second LLM call that scores the answer's groundedness (0–10) against the retrieved context. Return both the answer and the score.
-

Module 10 - LlamaIndex

Index Types · Query Engines · Retrievers · Data Connectors · Pipelines

10.1 What LlamaIndex Is

LlamaIndex (formerly GPT Index) specialises in **data ingestion and indexing** for LLM applications. Where LangChain focuses on general chaining, LlamaIndex focuses on how documents are structured, indexed, and queried. It provides higher-level abstractions: you describe your data and query pattern, and LlamaIndex handles the plumbing.

Core concepts:

- **Node** - a chunk of text with metadata, the atomic unit
 - **Index** - a data structure built from nodes (VectorStoreIndex, SummaryIndex, KeywordIndex)
 - **Query Engine** - takes a question, retrieves nodes, synthesises an answer
 - **Retriever** - the retrieval component, usable independently from the query engine
-

10.2 Setup and Settings

```
from llama_index.core import Settings
from llama_index.llms.openai import OpenAI
from llama_index.embeddings.openai import OpenAIEmbedding
import os
from dotenv import load_dotenv

load_dotenv()

# Configure global settings - applied to all indexes and query engines
Settings.llm = OpenAI(
    model="gpt-4o",
    api_key=os.environ["OPENAI_API_KEY"],
    temperature=0.1,
    max_tokens=1024,
)

Settings.embed_model = OpenAIEmbedding(
    model="text-embedding-3-small",
    api_key=os.environ["OPENAI_API_KEY"],
)

Settings.chunk_size = 512      # tokens per chunk
Settings.chunk_overlap = 64    # token overlap between chunks
```

10.3 Documents and Nodes

```
from llama_index.core import Document
from llama_index.core.node_parser import SentenceSplitter
from llama_index.core.schema import TextNode

# Create documents from text
```

```

documents = [
    Document(
        text="""LlamaIndex is a data framework for LLM applic
ations.
        It provides tools for ingesting, structuring, and que
rying data
        using large language models. The core abstraction is
the Index,
        which organises documents into a structure optimised
for retrieval.""",
        metadata={"source": "llamaindex_overview", "categor
y": "framework"},
    ),
    Document(
        text="""Vector store indexes store document embedding
s in a vector database.
        They support fast semantic search using approximate n
earest-neighbour algorithms.
        LlamaIndex integrates with ChromaDB, Pinecone, Weavia
te, and many other backends.""",
        metadata={"source": "vector_store_guide", "category":
"retrieval"},
    ),
    Document(
        text="""The query engine is the highest-level abstrac
tion in LlamaIndex.
        It takes a natural language question, retrieves relev
ant nodes,
        and uses an LLM to synthesise a coherent answer from
the retrieved context.""",
        metadata={"source": "query_engine_docs", "category":
"querying"},
    ),
]

# Parse documents into nodes (chunks)

```

```

parser = SentenceSplitter(chunk_size=128, chunk_overlap=20)
nodes = parser.get_nodes_from_documents(documents)

print(f"Documents:{len(documents)}, Nodes:{len(nodes)}")
for n in nodes:
    print(f" Node{n.node_id[:8]}... |{len(n.text)} chars | s
    ource:{n.metadata.get('source')}")

```

10.4 VectorStoreIndex

The most commonly used index - builds a vector store from document nodes and supports semantic search.

```

from llama_index.core import VectorStoreIndex, Document, Settings
from llama_index.llms.openai import OpenAI
from llama_index.embeddings.openai import OpenAIEmbedding
import os
from dotenv import load_dotenv

load_dotenv()

Settings.llm = OpenAI(model="gpt-4o", api_key=os.environ["OPENAI_API_KEY"])
Settings.embed_model = OpenAIEmbedding(model="text-embedding-3-small", api_key=os.environ["OPENAI_API_KEY"])

documents = [
    Document(text="RAG combines retrieval with generation to answer questions from external knowledge."),
    Document(text="Fine-tuning trains a model on new data to improve performance on a specific task."),
    Document(text="Prompt engineering optimises the input to an LLM to get better outputs without any training."),
    Document(text="Vector databases store high-dimensional em

```

```

beddings and support fast approximate nearest-neighbour search.
h."),
    Document(text="LangChain provides abstractions for building chains, agents, and memory-enabled LLM applications."),
]

# Build index - embeds all documents automatically
index = VectorStoreIndex.from_documents(documents)

# Query engine - retrieves + synthesises answer
query_engine = index.as_query_engine(similarity_top_k=3)

response = query_engine.query("What is the difference between RAG and fine-tuning?")
print("Answer:", response)
print("\nSource nodes:")
for node in response.source_nodes:
    print(f"Score{node.score:.4f}:{node.text[:60]}...")

```

10.5 Retriever - Used Independently

Sometimes you want retrieval without synthesis - for building custom pipelines.

```

from llama_index.core import VectorStoreIndex, Document, Settings
from llama_index.core.retrievers import VectorIndexRetriever
from llama_index.llms.openai import OpenAI
from llama_index.embeddings.openai import OpenAIEmbedding
import os
from dotenv import load_dotenv

load_dotenv()

Settings.llm = OpenAI(model="gpt-4o", api_key=os.environ["OPENAI_API_KEY"])

```

```

Settings.embed_model = OpenAIEmbedding(model="text-embedding-3-small", api_key=os.environ["OPENAI_API_KEY"])

documents = [
    Document(text="ChromaDB is an open-source embedding database with a simple Python API.", metadata={"tag": "vector-db"}),
    Document(text="Pinecone is a managed vector database designed for production workloads.", metadata={"tag": "vector-db"}),
    Document(text="FAISS is a library by Meta for efficient similarity search on dense vectors.", metadata={"tag": "vector-search"}),
    Document(text="pgvector adds vector similarity search to PostgreSQL.", metadata={"tag": "vector-db"}),
    Document(text="Weaviate is an open-source vector database with a GraphQL API.", metadata={"tag": "vector-db"}),
]

index = VectorStoreIndex.from_documents(documents)
retriever = VectorIndexRetriever(index=index, similarity_top_k=3)

# Retrieve without synthesising an answer
nodes = retriever.retrieve("open source vector databases")

print("Retrieved nodes:")
for node in nodes:
    print(f" [{node.score:.4f}]{node.text}")
    print(f" tag:{node.metadata.get('tag')}")

```

10.6 Summary Index

A SummaryIndex is designed for question-answering over long documents where you need to synthesise across many chunks rather than retrieve the most similar

ones.

```
from llama_index.core import SummaryIndex, Document, Settings
from llama_index.llms.openai import OpenAI
from llama_index.embeddings.openai import OpenAIEmbedding
import os
from dotenv import load_dotenv

load_dotenv()

Settings.llm = OpenAI(model="gpt-4o", api_key=os.environ["OPENAI_API_KEY"])
Settings.embed_model = OpenAIEmbedding(model="text-embedding-3-small", api_key=os.environ["OPENAI_API_KEY"])

# Simulate a long document split into multiple sections
sections = [
    Document(text="Introduction: This report analyses the performance of five LLMs on coding, reasoning, and language tasks."),
    Document(text="Section 1 - Coding: Claude Sonnet 4.5 scored highest on HumanEval with 94.2%. GPT-4o followed at 90.1%."),
    Document(text="Section 2 - Reasoning: GPT-4o led on GSM8K with 97.1%. Claude Sonnet 4.5 achieved 96.8%."),
    Document(text="Section 3 - Language: Both models performed similarly on MMLU, within 1% of each other."),
    Document(text="Conclusion: For coding tasks, prefer Claude Sonnet 4.5. For balanced performance, GPT-4o is the safer choice."),
]

index = SummaryIndex.from_documents(sections)
query_engine = index.as_query_engine()

# SummaryIndex reads ALL nodes, not just top-k - good for doc
```

```
ument-wide questions
response = query_engine.query("Which model should I use for a
coding project?")
print(response)
```

10.7 Persistent Storage

```
from llama_index.core import VectorStoreIndex, Document, Stor
ageContext, load_index_from_storage, Settings
from llama_index.llms.openai import OpenAI
from llama_index.embeddings.openai import OpenAIEmbedding
import os
from dotenv import load_dotenv

load_dotenv()

Settings.llm = OpenAI(model="gpt-4o", api_key=os.environ["OPE
NAI_API_KEY"])
Settings.embed_model = OpenAIEmbedding(model="text-embedding-
3-small", api_key=os.environ["OPENAI_API_KEY"])

PERSIST_DIR = "/tmp/llama_index_store"

# Build and save index
documents = [
    Document(text="LlamaIndex persists indexes to disk as JSO
N files."),
    Document(text="Loading a persisted index avoids re-embedd
ing on every run."),
]

index = VectorStoreIndex.from_documents(documents)
index.storage_context.persist(persist_dir=PERSIST_DIR)
print(f"Index saved to{PERSIST_DIR}")
```



```
# Load index back - no re-embedding needed
storage_context = StorageContext.from_defaults(persist_dir=PE
RSIST_DIR)
loaded_index = load_index_from_storage(storage_context)

query_engine = loaded_index.as_query_engine()
response = query_engine.query("How does LlamaIndex handle per
sistence?")
print("After reload:", response)
```

10.8 Custom Query Pipeline

For advanced use cases, build a query pipeline explicitly:

```
from llama_index.core import VectorStoreIndex, Document, Sett
ings
from llama_index.core.query_engine import RetrieverQueryEngin
e
from llama_index.core.retrievers import VectorIndexRetriever
from llama_index.core.response_synthesizers import get_respon
se_synthesizer, ResponseMode
from llama_index.llms.openai import OpenAI
from llama_index.embeddings.openai import OpenAIEmbedding
import os
from dotenv import load_dotenv

load_dotenv()

Settings.llm = OpenAI(model="gpt-4o", api_key=os.environ["OPE
NAI_API_KEY"])
Settings.embed_model = OpenAIEmbedding(model="text-embedding-
3-small", api_key=os.environ["OPENAI_API_KEY"])

documents = [
    Document(text="HNSW (Hierarchical Navigable Small World)
```

```

is the graph algorithm powering most modern vector search."),
    Document(text="IVF (Inverted File Index) partitions vectors into clusters for fast approximate search."),
    Document(text="PQ (Product Quantisation) compresses vectors to reduce memory usage at the cost of recall."),
    Document(text="Exact nearest-neighbour search guarantees correctness but is O(n) - unusable at scale."),
]

index = VectorStoreIndex.from_documents(documents)
retriever = VectorIndexRetriever(index=index, similarity_top_k=2)

# Different response modes
for mode in [ResponseMode.COMPACT, ResponseMode.TREE_SUMMARIZE]:
    synthesiser = get_response_synthesizer(response_mode=mode)
    engine = RetrieverQueryEngine(retriever=retriever, response_synthesizer=synthesiser)
    resp = engine.query("What algorithms are used in vector search?")
    print(f"[{mode}]{str(resp)[:120]}\n")

```

10.9 Module 10 Exercises

1. Build a document Q&A system that reads from a folder of `.txt` files, indexes them with `LlamaIndex`, and exposes a `ask(question: str) -> dict` function returning both the answer and the source filenames.
2. Compare `VectorStoreIndex` and `SummaryIndex` on the same document set of 10 paragraphs. Run 5 questions on each and compare answer quality for specific fact retrieval vs document-wide synthesis.
3. Implement a metadata-filtered retriever: given a corpus tagged with `{"year": 2024, "topic": "embeddings"}`, filter retrieval to only return nodes matching both a

topic and year from the query metadata.

4. Build a two-stage pipeline: first retrieve top-10 nodes with a `VectorIndexRetriever`, then re-rank them using a second LLM call that scores each node's relevance to the question (0–10). Return only the top-3 by LLM score.

Module 11 - Vector Databases

ChromaDB · Pinecone · Weaviate · Production vector storage

11.1 Why a Dedicated Vector Database

The in-memory `VectorStore` from Phase 2 works for development, but production systems need:

- **Persistence** - data survives restarts
- **Scale** - millions of vectors without loading everything into RAM
- **Filtering** - combine vector search with metadata conditions
- **Updates** - add, delete, and update documents efficiently
- **Multi-tenancy** - isolate collections per user or tenant

The three most widely used options are ChromaDB (simple, open-source), Pinecone (managed, production), and Weaviate (open-source, hybrid search).

11.2 ChromaDB

ChromaDB is the easiest vector database to get started with. It runs in-process or as a server, and persists data to disk.

```
import chromadb
from chromadb.config import Settings
import numpy as np
import os
from openai import OpenAI
from dotenv import load_dotenv
```

```

load_dotenv()
openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

# --- Embedding helper ---
def embed(texts: list[str]) -> list[list[float]]:
    resp = openai_client.embeddings.create(model="text-embedding-3-small", input=texts)
    return [e.embedding for e in sorted(resp.data, key=lambda x: x.index)]

# --- Persistent client (saves to disk) ---
client = chromadb.PersistentClient(path="/tmp/chroma_db")

# Get or create a collection
collection = client.get_or_create_collection(
    name="ai_knowledge_base",
    metadata={"hnsw:space": "cosine"}, # cosine similarity
)

# --- Add documents with pre-computed embeddings ---
documents = [
    "RAG connects LLMs to external knowledge bases at inference time.",
    "Fine-tuning adapts model weights using task-specific training data.",
    "Prompt engineering guides LLM outputs without any model training.",
    "Vector databases index embeddings for fast similarity search.",
    "Agents use LLMs to plan and execute multi-step tasks with tools.",
]
ids = [f"doc_{i}" for i in range(len(documents))]
metadatas = [{"category": "technique", "index": i} for i in range(len(documents))]

```

```

embeddings = embed(documents)

collection.add(
    ids=ids,
    documents=documents,
    embeddings=embeddings,
    metadatas=metadatas,
)
print(f"Collection size:{collection.count()}")

# --- Query ---
query = "how do I add knowledge to an LLM without retraini
ng?"
q_embed = embed([query])[0]

results = collection.query(
    query_embeddings=[q_embed],
    n_results=3,
    include=["documents", "distances", "metadatas"],
)

print("\nTop results:")
for doc, dist, meta in zip(
    results["documents"][0],
    results["distances"][0],
    results["metadatas"][0],
):
    print(f"    [{dist:.4f}]{doc}")
    print(f"        category:{meta['category']}")

```

Metadata filtering

```

# Filter by metadata before similarity ranking
results_filtered = collection.query(
    query_embeddings=[q_embed],

```

```

    n_results=3,
    where={"category": {"$eq": "technique"}},    # pre-filter
    include=["documents", "distances"],
)

# Update a document
collection.update(
    ids=["doc_0"],
    documents=["RAG retrieves external documents and injects
them into the LLM context."],
    embeddings=embed(["RAG retrieves external documents and i
njects them into the LLM context."]),
)

# Delete a document
collection.delete(ids=["doc_4"])
print(f"After delete, count:{collection.count()}")

```

11.3 Pinecone

Pinecone is a fully managed vector database - no infrastructure to run. It scales to billions of vectors and is the most common choice for production RAG systems.

```

# pip install pinecone
from pinecone import Pinecone, ServerlessSpec
import os
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()
# Requires: PINECONE_API_KEY in .env

openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

def embed(texts: list[str]) -> list[list[float]]:

```

```

    resp = openai_client.embeddings.create(model="text-embedding-3-small", input=texts)
    return [e.embedding for e in sorted(resp.data, key=lambda x: x.index)]

# --- Connect to Pinecone ---
pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])

INDEX_NAME = "ai-knowledge-base"

# Create index if it doesn't exist
if INDEX_NAME not in pc.list_indexes().names():
    pc.create_index(
        name=INDEX_NAME,
        dimension=1536,  # text-embedding-3-small dimension
        metric="cosine",
        spec=ServerlessSpec(cloud="aws", region="us-east-1"),
    )

index = pc.Index(INDEX_NAME)

# --- Upsert vectors ---
documents = [
    {"id": "d1", "text": "RAG connects LLMs to external knowledge bases."},
    {"id": "d2", "text": "Fine-tuning updates model weights on task-specific data."},
    {"id": "d3", "text": "ChromaDB is an open-source local vector database."},
]

vectors = [
    {
        "id": doc["id"],
        "values": embed([doc["text"]])[0],
    }
]

```

```

        "metadata": {"text": doc["text"], "source": "guide"},
    }
    for doc in documents
]

index.upsert(vectors=vectors)
print(index.describe_index_stats())

# --- Query ---
q_vector = embed(["how to add external knowledge to an LLM?"])[0]
results = index.query(
    vector=q_vector,
    top_k=3,
    include_metadata=True,
)

for match in results["matches"]:
    print(f"[{match['score']:.4f}]{match['metadata']['text']}")

# --- Metadata filter ---
filtered = index.query(
    vector=q_vector,
    top_k=3,
    filter={"source": {"$eq": "guide"}},
    include_metadata=True,
)

```

11.4 Weaviate

Weaviate is an open-source vector database with built-in hybrid search (vector + BM25 keyword), a GraphQL API, and a Python client.


```

# pip install weaviate-client
import weaviate
import weaviate.classes as wvc
from openai import OpenAI
import os
from dotenv import load_dotenv

load_dotenv()

openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

def embed(texts: list[str]) -> list[list[float]]:
    resp = openai_client.embeddings.create(model="text-embedding-3-small", input=texts)
    return [e.embedding for e in sorted(resp.data, key=lambda x: x.index)]

# --- Connect (local Docker or Weaviate Cloud) ---
# Local: docker run -d -p 8080:8080 -p 50051:50051 cr.weaviate.io/semitechnologies/weaviate:latest
# client = weaviate.connect_to_local()
# Cloud:
# client = weaviate.connect_to_weaviate_cloud(
#     cluster_url=os.environ["WEAVIATE_URL"],
#     auth_credentials=weaviate.auth.AuthApiKey(os.environ["WEAVIATE_API_KEY"]),
# )

# --- Create collection (schema) ---
# client.collections.create(
#     name="AIDocument",
#     properties=[
#         wvc.config.Property(name="text", data_type=wvc.config.DataType.TEXT),
#         wvc.config.Property(name="source", data_type=wvc.config.DataType.TEXT),
#     ]
# )

```

```

config.DataType.TEXT),
#         wvc.config.Property(name="category", data_type=wvc.
config.DataType.TEXT),
#     ],
#     vectorizer_config=wvc.config.Configure.Vectorizer.none
(), # supply own embeddings
# )

# --- Insert objects with vectors ---
# collection = client.collections.get("AIDocument")
# with collection.batch.dynamic() as batch:
#     for doc in documents:
#         batch.add_object(
#             properties={"text": doc["text"], "source": "gui
de", "category": "ai"},
#             vector=embed([doc["text"]])[0],
#         )

# --- Vector search ---
# results = collection.query.near_vector(
#     near_vector=embed(["external knowledge for LLMs"])[0],
#     limit=3,
#     return_properties=["text", "source"],
# )

# --- Hybrid search (BM25 + vector) ---
# results = collection.query.hybrid(
#     query="external knowledge LLM", # BM25 key
word query
#     vector=embed(["external knowledge for LLMs"])[0], # ve
ctor query
#     alpha=0.5, # 0=pure B
M25, 1=pure vector
#     limit=3,
# )

```

```
# --- Metadata filter ---
# from weaviate.classes.query import Filter
# results = collection.query.near_vector(
#     near_vector=embed(["vector database"])[0],
#     limit=3,
#     filters=Filter.by_property("category").equal("ai"),
# )

print("Weaviate patterns shown above - requires a running Weaviate instance.")
print("Start locally: docker run -d -p 8080:8080 cr.weaviate.io/semitechnologies/weaviate:latest")
```

11.5 Choosing a Vector Database

Requirement	Best choice
Local development, simple setup	ChromaDB
Production, managed, no ops	Pinecone
Hybrid BM25 + vector search	Weaviate
Already using PostgreSQL	pgvector
Maximum scale, self-hosted	Qdrant or Weaviate

Decision rule: Start with ChromaDB in development. When you go to production, choose Pinecone if you want zero infrastructure work, or Weaviate/Qdrant if you need hybrid search or want to self-host.

11.6 Module 11 Exercises

1. Build a `VectorDBBenchmark` that inserts 1,000 mock documents into ChromaDB, then runs 100 queries and measures: average query latency, 95th percentile latency, and results-per-second throughput.
2. Implement a `MultiCollectionSearch` class for ChromaDB that queries across multiple named collections and merges results by score, returning a unified ranked list.

3. Design and implement a tenant-isolated vector store using ChromaDB: each tenant gets their own collection, and a `TenantVectorStore` class routes add/search operations to the correct collection by tenant ID.
 4. Write a script that migrates a ChromaDB collection to a JSON export format (list of `{id, text, embedding, metadata}` dicts) and another script that imports it back - useful for backups and migration between providers.
-

Module 12 - Hugging Face Transformers

Pipelines · Tokenizers · Local Inference · PEFT/LoRA Fine-tuning

12.1 What Hugging Face Provides

The Hugging Face ecosystem is the central hub for open-source AI models. The `transformers` library provides:

- **Pipeline API** - high-level inference for 30+ tasks in 2-3 lines of code
- **AutoModel / AutoTokenizer** - load any model from the Hub with a single string
- **Trainer API** - fine-tune models with minimal boilerplate
- **PEFT** - parameter-efficient fine-tuning (LoRA, QLoRA) that trains a fraction of weights

When to use Hugging Face over hosted APIs:

- Data privacy - model runs locally, no data leaves your infrastructure
 - Cost at scale - after the inference hardware cost, no per-token fees
 - Customisation - fine-tune on your domain data
 - Offline - works without internet after model download
-

12.2 The Pipeline API

The `pipeline` function is the fastest path to inference. It handles tokenisation, inference, and decoding automatically.

```
from transformers import pipeline

# Text classification (sentiment analysis)
```

```

classifier = pipeline(
    task="text-classification",
    model="distilbert-base-uncased-finetuned-sst-2-english",
    device=-1,    # -1 = CPU; 0 = first GPU
)

texts = [
    "This LLM API is incredibly fast and easy to use.",
    "The documentation is confusing and the latency is terrible.",
    "The model works fine for most tasks.",
]

results = classifier(texts)
for text, result in zip(texts, results):
    print(f"[{result['label']}] | {result['score']:.3f} | {text[:60]}")

```

```

from transformers import pipeline

# Named entity recognition
ner = pipeline(
    task="ner",
    model="dbmdz/bert-large-cased-finetuned-conll03-english",
    aggregation_strategy="simple",
    device=-1,
)

text = "Anthropic was founded in San Francisco by Dario Amodei and Daniela Amodei."
entities = ner(text)
for ent in entities:
    print(f"{ent['word']!r:25s} → {ent['entity_group']} ({ent['score']:.3f})")

```

```

from transformers import pipeline

# Text generation (small local model)
generator = pipeline(
    task="text-generation",
    model="distilgpt2",          # small model - runs on CPU
    device=-1,
)

output = generator(
    "The future of AI agents is",
    max_new_tokens=50,
    num_return_sequences=2,
    temperature=0.8,
    do_sample=True,
    pad_token_id=50256,          # suppress warning for GPT-
2
)

for i, seq in enumerate(output):
    print(f"Sequence{i+1}:{seq['generated_text']}")

```

```

from transformers import pipeline

# Summarisation
summariser = pipeline(
    task="summarization",
    model="facebook/bart-large-cnn",
    device=-1,
)

article = """
Large language models have fundamentally changed how software
is built.

```

Developers now reach for AI APIs as naturally as they reach for database clients.

The shift from rule-based systems to neural language models has accelerated

since the release of GPT-3 in 2020, and the pace has only increased with

instruction-tuned models like ChatGPT, Claude, and Gemini. These models can

write code, explain concepts, extract information, and reason through complex

problems - tasks that previously required specialised rule-based systems or

significant human effort.

"""

```
summary = summariser(article, max_length=60, min_length=20, do_sample=False)
```

```
print(summary[0]["summary_text"])
```

12.3 Tokenizers

Understanding tokenizers is essential for managing context windows and debugging model behaviour.

```
from transformers import AutoTokenizer
```

```
# Load tokenizer for any model
```

```
tokenizer = AutoTokenizer.from_pretrained("gpt2")
```

```
text = "Retrieval-Augmented Generation (RAG) combines search with LLMs."
```

```
# Encode to token IDs
```

```
tokens = tokenizer.encode(text)
```

```
print(f"Token count:{len(tokens)}")
```

```

print(f"Token IDs:{tokens}")

# Decode back to text
decoded = tokenizer.decode(tokens)
print(f"Decoded:{decoded}")

# Tokenize to see individual tokens
token_strings = tokenizer.convert_ids_to_tokens(tokens)
print(f"Tokens:{token_strings}")

# Batch encoding with padding and truncation (for model input)
batch = ["What is RAG?", "Explain vector databases in detail."]
encoded = tokenizer(
    batch,
    padding=True,
    truncation=True,
    max_length=512,
    return_tensors="pt",  # PyTorch tensors; use "tf" for TensorFlow
)
print(f"\nBatch input_ids shape:{encoded['input_ids'].shape}")
print(f"Attention mask shape:{encoded['attention_mask'].shape}")

# Count tokens for a prompt before sending to any model
def count_tokens(text: str, model: str = "gpt2") -> int:
    tok = AutoTokenizer.from_pretrained(model)
    return len(tok.encode(text))

prompt = "Summarise the following document: " + "word " * 500
print(f"\nPrompt token count:{count_tokens(prompt)}")

```


12.4 AutoModel - Loading Models Directly

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

# Load a small model for local inference (CPU-friendly)
model_name = "distilgpt2"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float32,  # float16 for GPU, float32 for CPU
)
model.eval()

prompt = "The key advantage of local LLMs is"
inputs = tokenizer(prompt, return_tensors="pt")

with torch.no_grad():
    output_ids = model.generate(
        inputs["input_ids"],
        max_new_tokens=40,
        do_sample=True,
        temperature=0.8,
        top_p=0.9,
        pad_token_id=tokenizer.eos_token_id,
    )

generated = tokenizer.decode(output_ids[0], skip_special_tokens=True)
print(generated)
```

12.5 PEFT / LoRA Fine-tuning

LoRA (Low-Rank Adaptation) fine-tunes only a small number of added parameters - typically 1–5% of the original model size. This makes fine-tuning feasible on consumer GPUs and dramatically reduces storage requirements.

```
# pip install peft transformers torch datasets accelerate
from transformers import AutoTokenizer, AutoModelForCausalLM,
TrainingArguments, Trainer
from peft import LoraConfig, TaskType, get_peft_model, PeftMo
del
from datasets import Dataset
import torch

# — 1. Load base model —————
model_name = "distilgpt2"          # swap for "meta-llama/Llama-
3.1-8B" on GPU

tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(model_name)

# — 2. Configure LoRA —————
lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=8,                          # rank - higher = more paramete
rs, more capacity
    lora_alpha=32,                 # scaling factor (usually 2x to
4x r)
    target_modules=["c_attn"],    # which layers to adapt (mode
l-specific)
    lora_dropout=0.1,
    bias="none",
)
```

```
peft_model = get_peft_model(model, lora_config)
peft_model.print_trainable_parameters()
# Output: trainable params: 294,912 || all params: 82,206,720
|| trainable%: 0.3588
```

— 3. Prepare dataset

```
training_examples = [
    "Question: What is RAG? Answer: RAG stands for Retrieval-
    Augmented Generation.",
    "Question: What is fine-tuning? Answer: Fine-tuning train
    s a pre-trained model on new data.",
    "Question: What is an embedding? Answer: An embedding is
    a dense vector representing text meaning.",
    "Question: What is a vector database? Answer: A vector da
    tabase stores embeddings for similarity search.",
    "Question: What is LangChain? Answer: LangChain is a fram
    ework for building LLM applications.",
]
```

```
def tokenize(examples):
    return tokenizer(
        examples["text"],
        truncation=True,
        max_length=128,
        padding="max_length",
    )
```

```
dataset = Dataset.from_dict({"text": training_examples})
tokenized = dataset.map(tokenize, batched=True)
tokenized = tokenized.map(lambda x: {"labels": x["input_id
s"]})
```

— 4. Training arguments

```
training_args = TrainingArguments(
```

```

    output_dir="/tmp/lora_output",
    num_train_epochs=3,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    fp16=False,                # True on GPU with float16 support
    logging_steps=5,
    save_strategy="epoch",
    report_to="none",          # disable wandb/tensorboard for this example
)

# — 5. Train —
trainer = Trainer(
    model=peft_model,
    args=training_args,
    train_dataset=tokenized,
)

trainer.train()

# — 6. Save adapter only (small file, not the full model) —
peft_model.save_pretrained("/tmp/lora_adapter")
tokenizer.save_pretrained("/tmp/lora_adapter")
print("LoRA adapter saved.")

# — 7. Load adapter for inference —
base = AutoModelForCausalLM.from_pretrained(model_name)
merged = PeftModel.from_pretrained(base, "/tmp/lora_adapter")
merged.eval()

inputs = tokenizer("Question: What is RAG? Answer:", return_t

```

```
ensors="pt")
with torch.no_grad():
    out = merged.generate(**inputs, max_new_tokens=30, pad_to_
ken_id=tokenizer.eos_token_id)
print(tokenizer.decode(out[0], skip_special_tokens=True))
```

Practical LoRA targets by model family:

- GPT-2 family: ["c_attn"]
- LLaMA / Mistral: ["q_proj", "v_proj"]
- BERT / RoBERTa: ["query", "value"]

12.6 Module 12 Exercises

1. Build a `ModelBenchmark` class that runs the same 10 prompts through two different HuggingFace text-generation pipelines, records latency and output length, and prints a comparison table.
2. Write a `TokenBudgetChecker` that, given a list of texts and a model name, filters out any text that would exceed a configurable token limit and returns the remaining texts with their token counts.
3. Use the NER pipeline to build a function `extract_entities_from_corpus(texts: list[str]) -> dict` that returns a dict mapping entity types (PERSON, ORG, LOC) to lists of unique extracted values across all texts.
4. Implement the full LoRA fine-tuning loop from section 12.5 on a custom dataset of 20 Q&A pairs about a topic of your choice. Verify the fine-tuned model answers those questions correctly at inference time.

Module 13 - LangGraph & Agents

Agentic loops · Tool use · ReAct · Stateful graphs · Multi-step tasks

13.1 What LangGraph Is

LangGraph is a library for building **stateful, multi-step LLM applications** modelled as directed graphs. Each node is a function that reads and writes to a shared state dict. Edges define control flow - including conditional branches and loops.

LangGraph is the right tool when:

- The number of LLM calls is not fixed in advance
 - The agent must decide which tool to call next based on results
 - You need persistent state between steps (memory, intermediate results)
 - You want human-in-the-loop checkpoints
-

13.2 Core Concepts

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Annotated
import operator

# State is a TypedDict - shared across all nodes
# Annotated with a reducer function that says how to merge up
# dates
class AgentState(TypedDict):
    messages: Annotated[list[str], operator.add] # append new messages
    step_count: int
    final_answer: str

# Nodes are plain Python functions
def think_node(state: AgentState) -> dict:
    thought = f"Step{state['step_count'] + 1}: analysing the problem..."
    return {
        "messages": [thought],
        "step_count": state["step_count"] + 1,
    }

def answer_node(state: AgentState) -> dict:
    return {"final_answer": "Based on my analysis, the answer is X."}

# Conditional edge - decides where to go next
```

```

def should_continue(state: AgentState) -> str:
    if state["step_count"] < 2:
        return "think"
    return "answer"

# Build the graph
graph = StateGraph(AgentState)
graph.add_node("think", think_node)
graph.add_node("answer", answer_node)

graph.add_edge(START, "think")
graph.add_conditional_edges("think", should_continue, {"think": "think", "answer": "answer"})
graph.add_edge("answer", END)

app = graph.compile()

# Run
result = app.invoke({"messages": [], "step_count": 0, "final_answer": ""})
print("Messages:", result["messages"])
print("Answer: ", result["final_answer"])

```

13.3 ReAct Agent - Reason + Act

The ReAct pattern (Reason + Act) is the most common agent architecture. The model reasons about what to do, calls a tool, observes the result, and repeats until it has a final answer.

```

from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, AIMessage, ToolMessage, SystemMessage
from typing import TypedDict, Annotated
import operator, json, os, math

```

```

from dotenv import load_dotenv

load_dotenv()

# — Tools —————

def calculator(expression: str) -> str:
    """Evaluate a safe mathematical expression."""
    try:
        # Allow only safe math operations
        allowed = {k: v for k, v in math.__dict__.items() if
not k.startswith("_")}
        allowed.update({"abs": abs, "round": round})
        result = eval(expression, {"__builtins__": {}}, allowed)

        return str(result)
    except Exception as e:
        return f"Error:{e}"

def token_cost_lookup(model: str) -> str:
    """Return cost per 1M tokens for a given model."""
    costs = {
        "gpt-4o": {"input": 2.50, "output": 10.0
0},
        "gpt-4o-mini": {"input": 0.15, "output": 0.6
0},
        "claude-sonnet-4-5": {"input": 3.00, "output": 15.0
0},
        "claude-haiku-4-5": {"input": 0.80, "output": 4.0
0},
    }
    if model not in costs:
        return f"Unknown model:{model}. Known:{list(costs.keys())}"
    c = costs[model]
    return json.dumps({"model": model, "input_per_1M": c["inp

```



```

ut"], "output_per_1M": c["output"]})

TOOLS = {
    "calculator": calculator,
    "token_cost_lookup": token_cost_lookup,
}

TOOL_SCHEMAS = [
    {
        "type": "function",
        "function": {
            "name": "calculator",
            "description": "Evaluate a mathematical expression. Use Python math syntax.",
            "parameters": {
                "type": "object",
                "properties": {"expression": {"type": "string", "description": "Math expression to evaluate"}},
                "required": ["expression"],
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "token_cost_lookup",
            "description": "Look up the cost per 1M tokens for a given LLM model.",
            "parameters": {
                "type": "object",
                "properties": {"model": {"type": "string", "description": "Model identifier"}},
                "required": ["model"],
            },
        },
    },
]

```

```

]

# — State —————
class ReActState(TypedDict):
    messages: Annotated[list, operator.add]

# — LLM —————

llm = ChatOpenAI(
    model="gpt-4o",
    api_key=os.environ["OPENAI_API_KEY"],
).bind_tools(TOOL_SCHEMAS)

SYSTEM = SystemMessage(content="""You are a helpful AI assistant with access to two tools:
- calculator: for math calculations
- token_cost_lookup: for LLM pricing information

Always use tools when calculations or pricing lookups are needed. Think step by step.""")

# — Nodes —————

def agent_node(state: ReActState) -> dict:
    """Call the LLM - it may request tool use or give a final answer."""
    response = llm.invoke([SYSTEM] + state["messages"])
    return {"messages": [response]}

def tool_node(state: ReActState) -> dict:
    """Execute all requested tool calls and return results."""
    last_msg = state["messages"][-1]
    results = []

```

```

    for tool_call in last_msg.tool_calls:
        fn = TOOLS[tool_call["name"]]
        output = fn(**tool_call["args"])
        results.append(
            ToolMessage(content=output, tool_call_id=tool_call["id"])
        )
    return {"messages": results}

def should_use_tools(state: ReActState) -> str:
    last = state["messages"][-1]
    if hasattr(last, "tool_calls") and last.tool_calls:
        return "tools"
    return END

# — Graph —————
graph = StateGraph(ReActState)
graph.add_node("agent", agent_node)
graph.add_node("tools", tool_node)

graph.add_edge(START, "agent")
graph.add_conditional_edges("agent", should_use_tools, {"tools": "tools", END: END})
graph.add_edge("tools", "agent") # after tools, back to agent

agent = graph.compile()

# — Run —————

question = """I'm running a pipeline that processes 10,000 requests per day.
Each request uses 800 input tokens and 300 output tokens with gpt-4o.
What is my daily cost in USD?"""

```

```

result = agent.invoke({
    "messages": [HumanMessage(content=question)]
})

# Print the final answer
for msg in result["messages"]:
    if isinstance(msg, AIMessage) and not getattr(msg, "tool_
calls", None):
        print("Final answer:\n", msg.content)

```

13.4 Human-in-the-Loop with Checkpoints

LangGraph supports pausing the graph mid-execution to get human approval or input.

```

from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from typing import TypedDict, Annotated
import operator

class ReviewState(TypedDict):
    plan: str
    approved: bool
    result: str
    messages: Annotated[list[str], operator.add]

def plan_node(state: ReviewState) -> dict:
    plan = "Step 1: Query vector DB. Step 2: Summarise results. Step 3: Format response."
    return {"plan": plan, "messages": [f"Plan created: {plan}"]}

def execute_node(state: ReviewState) -> dict:
    if not state["approved"]:

```

```

        return {"result": "Execution skipped - plan not approved.", "messages": ["Skipped."]}
        return {"result": "Pipeline executed successfully.", "messages": ["Executed."]}

def review_gate(state: ReviewState) -> str:
    """Interrupt here for human review."""
    return "execute"

# MemorySaver enables checkpointing (in-memory for dev; use SqliteSaver for prod)
checkpointer = MemorySaver()

graph = StateGraph(ReviewState)
graph.add_node("plan", plan_node)
graph.add_node("execute", execute_node)

graph.add_edge(START, "plan")
graph.add_edge("plan", "execute")
graph.add_edge("execute", END)

# interrupt_before pauses the graph before the named node
app = graph.compile(checkpointer=checkpointer, interrupt_before=["execute"])

config = {"configurable": {"thread_id": "review-session-1"}}

# Run until interrupt
state = app.invoke(
    {"plan": "", "approved": False, "result": "", "messages": []},
    config=config
)
print("Plan:", state["plan"])
print("Waiting for human approval...")

```

```
# Human approves and resumes
state["approved"] = True
final = app.invoke(state, config=config)
print("Result:", final["result"])
```

13.5 Module 13 Exercises

1. Build a research agent using LangGraph with three tools: `web_search` (mock it to return fixed results), `summarise(text)` (calls an LLM), and `save_note(content)` (appends to a list). The agent should research a topic, summarise findings, and save a structured note.
2. Add a maximum step limit to the ReAct agent from section 13.3: if the agent takes more than 8 tool calls without reaching a final answer, it should stop and return "Reached step limit."
3. Implement a `PlanAndExecute` agent with two nodes: `planner` (LLM generates a numbered list of steps) and `executor` (iterates through steps, calling tools for each). State should track the current step index.
4. Build a multi-turn agent that maintains memory across separate invocations using `MemorySaver` and a fixed `thread_id`. Demonstrate that the agent remembers context from a previous conversation in a new one.

Module 14 - CrewAI

Multi-agent crews · Roles · Tasks · Delegation · Orchestration

14.1 What CrewAI Is

CrewAI is a framework for orchestrating **multiple AI agents** that collaborate on a shared goal. Each agent has a role, backstory, and set of tools. Tasks are assigned to agents, and a Crew coordinates execution.

The key mental model:

- **Agent** - a specialist (researcher, writer, reviewer) with a defined expertise and tools

- **Task** - a concrete piece of work with an expected output
- **Crew** - a team of agents that executes tasks, either sequentially or hierarchically

When to use CrewAI: Content pipelines, multi-step research workflows, code review loops, and any process where distinct specialised roles add clarity and quality.

14.2 Agents

```
from crewai import Agent
from langchain_openai import ChatOpenAI
import os
from dotenv import load_dotenv

load_dotenv()

llm = ChatOpenAI(model="gpt-4o", api_key=os.environ["OPENAI_API_KEY"])

# Research specialist
researcher = Agent(
    role="AI Research Analyst",
    goal="Find accurate and up-to-date information on AI topics.",
    backstory="""You are a senior AI research analyst with 10 years of experience reading and synthesising academic papers, technical blogs, and release notes. You are known for being precise, citing sources, and never making things up.""",
    llm=llm,
    verbose=True,
    allow_delegation=False, # this agent does not delegate to others
    max_iter=5, # max reasoning iterations before stopping
```

```

)

# Writing specialist
writer = Agent(
    role="Technical Content Writer",
    goal="Write clear, accurate, and engaging technical content for developers.",
    backstory="""You are a technical writer with a background in software engineering.
    You excel at turning complex AI concepts into practical, well-structured guides.
    You always write in active voice, use concrete examples, and avoid jargon."""
    llm=llm,
    verbose=True,
    allow_delegation=False,
)

# Quality reviewer
reviewer = Agent(
    role="Content Quality Reviewer",
    goal="Ensure content is accurate, complete, and publication-ready.",
    backstory="""You are a meticulous editor who reviews technical content for
    accuracy, clarity, and completeness. You check facts, flag unsupported claims,
    and ensure the writing matches the target audience's level."""
    llm=llm,
    verbose=True,
    allow_delegation=True, # reviewer can delegate back to
    writer for revisions
)

```


14.3 Tools for Agents

```
from crewai.tools import BaseTool
from pydantic import BaseModel, Field
from typing import Type
import json

# Define a custom tool by subclassing BaseTool
class TokenCostCalculatorInput(BaseModel):
    model: str = Field(description="LLM model name, e.g. 'gpt-4o'")
    input_tokens: int = Field(description="Number of input tokens")
    output_tokens: int = Field(description="Number of output tokens")

class TokenCostCalculator(BaseTool):
    name: str = "Token Cost Calculator"
    description: str = "Calculate the USD cost of an LLM API call given model name and token counts."
    args_schema: Type[BaseModel] = TokenCostCalculatorInput

    def _run(self, model: str, input_tokens: int, output_tokens: int) -> str:
        pricing = {
            "gpt-4o": {"input": 2.50, "output": 10.00},
            "gpt-4o-mini": {"input": 0.15, "output": 0.60},
            "claude-sonnet-4-5": {"input": 3.00, "output": 15.00},
        }
        if model not in pricing:
            return f"Unknown model:{model}"
        p = pricing[model]
        cost = (input_tokens * p["input"] + output_tokens * p
```

```

["output"])) / 1_000_000
    return json.dumps({
        "model": model,
        "input_tokens": input_tokens,
        "output_tokens": output_tokens,
        "cost_usd": round(cost, 6),
    })

class ModelContextLookupInput(BaseModel):
    model: str = Field(description="LLM model name")

class ModelContextLookup(BaseTool):
    name: str = "Model Context Window Lookup"
    description: str = "Look up the context window size (in tokens) of an LLM."
    args_schema: Type[BaseModel] = ModelContextLookupInput

    def _run(self, model: str) -> str:
        windows = {
            "gpt-4o": 128_000,
            "gpt-4o-mini": 128_000,
            "claude-sonnet-4-5": 200_000,
            "claude-opus-4-5": 200_000,
            "gemini-1.5-pro": 1_000_000,
        }
        tokens = windows.get(model, "Unknown")
        return json.dumps({"model": model, "context_window_tokens": tokens})

cost_tool = TokenCostCalculator()
context_tool = ModelContextLookup()

```

14.4 Tasks

```

from crewai import Task
from crewai import Agent # assume agents from 14.2 are defined

# Each task has: description, expected_output, agent
def make_tasks(researcher, writer, reviewer):

    research_task = Task(
        description="""Research the current state of RAG (Retrieval-Augmented Generation) in 2024-2025.
        Cover:
        1. What RAG is and how it works
        2. The main variants (naive RAG, advanced RAG, modular RAG)
        3. Common production challenges and how practitioners solve them
        4. Three real-world use cases with specific results

        Be specific and factual. Avoid vague generalities.""",
        expected_output="""A structured research report with:
        - A clear definition and architecture overview
        - Descriptions of the three RAG variants with key differences
        - Three specific production challenges with concrete solutions
        - Three real-world use cases with measurable outcomes
        Format: Markdown with headers.""",
        agent=researcher,
    )

    writing_task = Task(
        description="""Using the research report provided, write a developer guide titled
        'Building Production RAG Systems'."""

```

```

    The guide should:
    - Be written for senior developers with Python experience
    - Include a conceptual overview, implementation considerations, and common pitfalls
    - Use active voice and concrete examples
    - Be 600-800 words"""
    expected_output="A complete, publication-ready developer guide in Markdown format.",
    agent=writer,
    context=[research_task],    # this task depends on research_task output
)

review_task = Task(
    description="""Review the developer guide for:
    1. Technical accuracy - flag any claims that are incorrect or unsupported
    2. Completeness - identify important topics that are missing
    3. Clarity - suggest any sections that need rewriting
    4. Audience fit - check that the level is appropriate for senior developers

    Provide a structured review with specific, actionable feedback."""
    expected_output="""A quality review report containing:
    - Overall assessment (Ready / Needs Revision)
    - Specific accuracy issues (if any) with corrections
    - Missing content recommendations
    - Clarity improvement suggestions
    - Final verdict""",
    agent=reviewer,
    context=[writing_task],

```

```
)

return research_task, writing_task, review_task
```

14.5 Crew - Putting It Together

```
from crewai import Crew, Process
from langchain_openai import ChatOpenAI
import os
from dotenv import load_dotenv

load_dotenv()

llm = ChatOpenAI(model="gpt-4o", api_key=os.environ["OPENAI_API_KEY"])

# Recreate agents with tools
researcher = Agent(
    role="AI Research Analyst",
    goal="Find accurate information on AI topics.",
    backstory="Senior AI research analyst with expertise in LLM systems.",
    llm=llm,
    tools=[context_tool], # can look up context windows
    verbose=False,
    max_iter=5,
)

writer = Agent(
    role="Technical Content Writer",
    goal="Write clear technical content for developers.",
    backstory="Technical writer with a software engineering background.",
    llm=llm,
    tools=[cost_tool], # can look up costs for examples
```

```

        verbose=False,
        max_iter=5,
    )

    reviewer = Agent(
        role="Content Quality Reviewer",
        goal="Ensure content is accurate and publication-ready.",
        backstory="Meticulous editor who reviews technical AI content.",
        llm=llm,
        verbose=False,
        max_iter=5,
    )

    research_task, writing_task, review_task = make_tasks(researcher, writer, reviewer)

    crew = Crew(
        agents=[researcher, writer, reviewer],
        tasks=[research_task, writing_task, review_task],
        process=Process.sequential, # tasks run in order; use Process.hierarchical for manager-led
        verbose=False,
        memory=False, # set True to enable cross-task memory (requires embeddings)
    )

    # Kick off the crew
    result = crew.kickoff()
    print("=== CREW OUTPUT ===")
    print(result)

    # Access individual task outputs
    print("\n=== TASK OUTPUTS ===")
    for task in crew.tasks:
        print(f"\n---{task.agent.role} ---")

```

```
if task.output:
    print(str(task.output)[:200], "...")
```

14.6 Hierarchical Process with a Manager

In a hierarchical process, a manager agent orchestrates which worker agent handles each task.

```
from crewai import Crew, Process, Agent
from langchain_openai import ChatOpenAI
import os
from dotenv import load_dotenv

load_dotenv()

llm = ChatOpenAI(model="gpt-4o", api_key=os.environ["OPENAI_API_KEY"])

# Manager - coordinates other agents
manager = Agent(
    role="AI Content Production Manager",
    goal="Coordinate a team to produce high-quality AI content efficiently.",
    backstory="""You are an experienced content production manager who excels at breaking down projects, assigning work to the right specialist, and ensuring quality standards are met before publication.""",
    llm=llm,
    allow_delegation=True,
    verbose=False,
)

# Workers
researcher = Agent(
```

```

        role="AI Research Analyst",
        goal="Research AI topics thoroughly.",
        backstory="Senior researcher specialising in LLM systems
and RAG.",
        llm=llm,
        verbose=False,
    )

writer = Agent(
    role="Technical Writer",
    goal="Write developer-focused technical content.",
    backstory="Technical writer with Python and ML experienc
e.",
    llm=llm,
    verbose=False,
)

from crewai import Task

main_task = Task(
    description="""Produce a complete technical brief on 'Vec
tor Databases for RAG in Production'.
    Include: what they are, how to choose one, and three impl
ementation best practices.""",
    expected_output="A structured technical brief in Markdow
n, 400-500 words.",
    agent=manager,    # manager is responsible but may delegat
e
)

crew = Crew(
    agents=[manager, researcher, writer],
    tasks=[main_task],
    process=Process.hierarchical,
    manager_agent=manager,
    verbose=False,

```



```
)

result = crew.kickoff()
print(result)
```

14.7 Module 14 Exercises

1. Build a three-agent content pipeline for technical blog posts: Researcher (finds key points), Outline Writer (structures the post), Full Writer (expands each section). Run it on "The practical guide to fine-tuning vs RAG" and print the final post.
2. Create a code review crew: Developer (writes code), Security Reviewer (checks for vulnerabilities), Performance Reviewer (checks for inefficiency), and Lead (merges feedback into a final report). Test with a provided Python snippet.
3. Implement a custom `WebSearchTool` CrewAI tool that wraps a mock search function returning predefined results. Wire it to a Researcher agent and verify it uses the tool when asked a question requiring research.
4. Compare `Process.sequential` vs `Process.hierarchical` on the same 3-task pipeline. Measure total execution time and compare the quality of outputs. Document when each process type is preferable.

Phase 3 Summary

What You Learned

Module	Core Skills
M09 - LangChain	LCEL pipe composition, ChatPromptTemplate, StrOutputParser, structured output with Pydantic, parallel chains, conditional routing, document splitting, conversational memory, full RAG chain
M10 - LlamaIndex	Document and Node model, VectorStoreIndex, SummaryIndex, standalone retrievers, persistent storage, custom query pipelines, response synthesisers

Module	Core Skills
M11 - Vector Databases	ChromaDB (local + persistent), Pinecone (managed), Weaviate (hybrid search), metadata filtering, choosing the right database for your use case
M12 - Hugging Face	Pipeline API for 30+ tasks, AutoTokenizer, AutoModelForCausalLM, counting tokens, LoRA/PEFT fine-tuning, saving and loading adapters
M13 - LangGraph	StateGraph with TypedDict state, conditional edges, ReAct agent with real tools, human-in-the-loop via <code>interrupt_before</code> , MemorySaver checkpointing
M14 - CrewAI	Agent roles and backstories, custom BaseTool, Task with context dependencies, sequential and hierarchical Process, manager-led delegation

Patterns to Carry Forward

- LCEL's `|` pipe operator makes data flow explicit - prefer it over manually chaining function calls
- LlamaIndex is better than LangChain for complex indexing; LangChain is better for flexible chaining - use them together via LangChain's LlamaIndex retriever integration
- Always supply embeddings explicitly to ChromaDB in production - the default auto-embedding downloads an ONNX model on first use
- LoRA adapters are small files (a few MB); the base model stays unchanged - always save the adapter, not the merged model, for storage efficiency
- LangGraph's `interrupt_before` is the correct way to implement human approval gates - not a simple `input()` call inside a node
- In CrewAI, use `context=[previous_task]` to wire task outputs as inputs; the framework injects the previous output into the agent's context automatically

What Is Next - Phase 4

Phase 4 covers **RAG and Memory Systems** in depth - going beyond basic retrieval into the patterns that make production RAG reliable:

- **M15** - RAG fundamentals: chunking strategies, retrieval pipelines, context injection, reranking

- **M16** - Advanced RAG patterns: HyDE, self-query, multi-hop reasoning, RAG evaluation with RAGAS
- **M17** - Memory and state: short and long-term memory, entity tracking, cross-session persistence

Install before Phase 4: `pip install ragas rank-bm25 sentence-transformers`

Phase 4 RAG & Memory Systems

Libraries: `langchain` · `langchain-openai` · `chromadb` · `rank-bm25` · `openai` ·

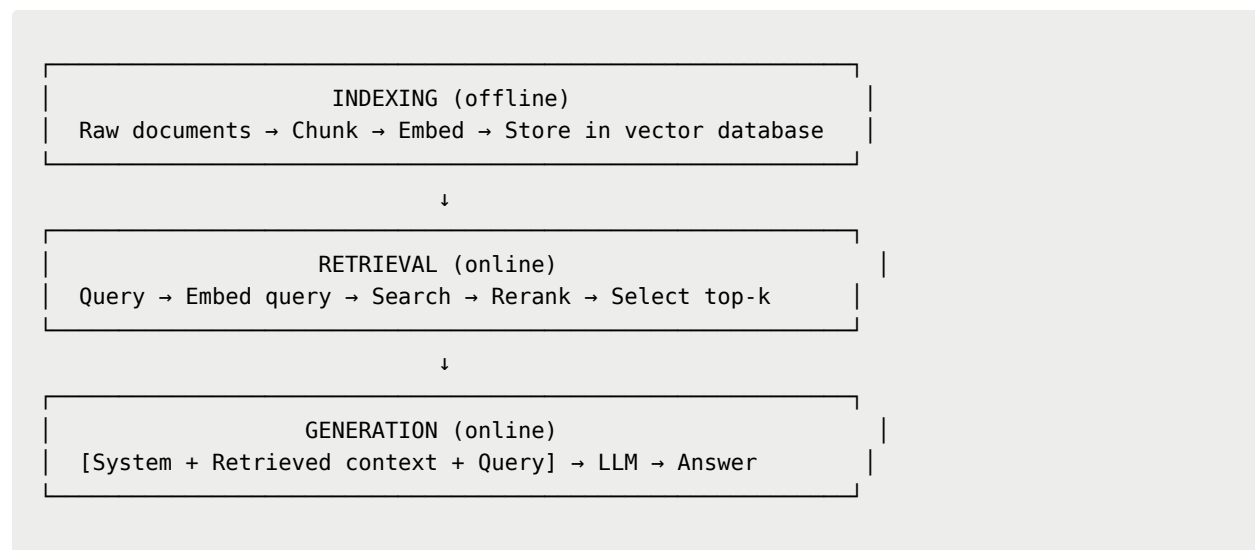
Install: `pip install langchain langchain-openai langchain-community chromadb rank-bm25 openai anthropic python-dotenv numpy`

Module 15 - RAG Fundamentals

Chunking strategies · Retrieval pipelines · Context injection · Reranking

15.1 The Full RAG Architecture

RAG has three distinct stages, and problems in each stage degrade the final answer in different ways. Understanding each stage independently is the key to debugging and improving a RAG system.



The most common failure modes are:

- **Retrieval failure** - the right document is not returned (wrong chunk size, poor embedding, no metadata filter)

- **Context too long** - too many retrieved chunks overflow the context window or dilute the signal
 - **Hallucination** - the LLM ignores the retrieved context and answers from training data
 - **Stale index** - the vector store has not been updated with new documents
-

15.2 Chunking Strategies

Chunking is the most impactful decision in RAG. The wrong chunk size causes poor retrieval regardless of how good your embedding model or vector database is.

Rules of thumb:

- Chunks should be semantically self-contained - a reader should understand the chunk without surrounding context
- Chunk size: 256–512 tokens works well for dense technical text; 128–256 for Q&A
- Overlap of 10–15% of chunk size preserves context at boundaries

Strategy 1: Fixed-size character chunking

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class Chunk:
```

```
    text: str
```

```
    doc_id: str
```

```
    chunk_index: int
```

```
    char_start: int
```

```
    char_end: int
```

```
def fixed_char_chunk(
```

```
    text: str,
```

```
    doc_id: str,
```

```
    chunk_size: int = 512,
```

```
    overlap: int = 64,
```

```

) -> list[Chunk]:
    """
    Split text into fixed-size character chunks with overlap.
    Fast and predictable, but may cut mid-sentence.
    """
    chunks = []
    start = 0
    idx = 0

    while start < len(text):
        end = min(start + chunk_size, len(text))
        chunks.append(Chunk(
            text=text[start:end],
            doc_id=doc_id,
            chunk_index=idx,
            char_start=start,
            char_end=end,
        ))
        idx += 1
        start += chunk_size - overlap

    return chunks

sample = "Large language models have changed AI. " * 20
chunks = fixed_char_chunk(sample, "doc_001", chunk_size=200,
overlap=30)
print(f"Fixed chunks:{len(chunks)}")
print(f"  First:{chunks[0].text[:60]}...")
print(f"  Last:{chunks[-1].text[:60]}...")

```

Strategy 2: Recursive character splitting

```

def recursive_split(
    text: str,
    doc_id: str,

```

```

    chunk_size: int = 512,
    overlap: int = 64,
    separators: list[str] | None = None,
) -> list[Chunk]:
    """
    Split on natural boundaries (paragraphs → sentences → words).
    Preserves semantic units better than fixed-size splitting.
    This is the same logic as LangChain's RecursiveCharacterTextSplitter.
    """
    if separators is None:
        separators = ["\n\n", "\n", ". ", "! ", "? ", " ",
"" ]

    def _split(text: str, seps: list[str]) -> list[str]:
        if not seps or len(text) <= chunk_size:
            return [text]

        sep = seps[0]
        parts = text.split(sep) if sep else list(text)

        result: list[str] = []
        current = ""

        for part in parts:
            candidate = (current + sep + part) if current else
e part
            if len(candidate) > chunk_size and current:
                result.append(current)
                current = part
            else:
                current = candidate

        if current:

```

```

        result.append(current)

    # Recursively split any chunk still over the limit
    final: list[str] = []
    for chunk in result:
        if len(chunk) > chunk_size:
            final.extend(_split(chunk, seps[1:]))
        else:
            final.append(chunk)
    return final

raw_chunks = _split(text, separators)

# Add overlap from previous chunk
chunks_with_overlap: list[str] = []
for i, chunk in enumerate(raw_chunks):
    if i > 0 and overlap > 0:
        prev = raw_chunks[i - 1]
        suffix = prev[-overlap:] if len(prev) > overlap else prev
        chunk = suffix + chunk
    chunks_with_overlap.append(chunk)

return [
    Chunk(text=c, doc_id=doc_id, chunk_index=i, char_start=0, char_end=len(c))
    for i, c in enumerate(chunks_with_overlap)
]

text = """
RAG stands for Retrieval-Augmented Generation.

```

It combines a retriever and a generator. The retriever finds relevant documents from a knowledge base. The generator uses those documents

to produce a grounded answer.

There are three main variants of RAG: naive RAG, advanced RAG, and modular RAG. Each offers different trade-offs in complexity and quality.

"""

```
chunks = recursive_split(text.strip(), "rag_doc", chunk_size=
150, overlap=20)
for c in chunks:
    print(f" Chunk{c.chunk_index} ({len(c.text)} chars):{c.text[:60]}...")
```

Strategy 3: Sentence-aware chunking

```
import re

def sentence_chunk(
    text: str,
    doc_id: str,
    sentences_per_chunk: int = 5,
    overlap_sentences: int = 1,
) -> list[Chunk]:
    """
    Group a fixed number of sentences per chunk.
    Best for prose and narrative documents.
    """
    # Split on sentence endings - handles abbreviations imperfectly
    # but well enough for most technical text
    sentences = re.split(r'(?<=[.!?])\s+', text.strip())
    sentences = [s.strip() for s in sentences if s.strip()]

    chunks = []
```

```

step = max(1, sentences_per_chunk - overlap_sentences)

for i in range(0, len(sentences), step):
    batch = sentences[i : i + sentences_per_chunk]
    text_chunk = " ".join(batch)
    chunks.append(Chunk(
        text=text_chunk,
        doc_id=doc_id,
        chunk_index=len(chunks),
        char_start=0,
        char_end=len(text_chunk),
    ))

return chunks

```

doc = """RAG is a powerful technique for grounding LLM answers. It retrieves documents at inference time. Those documents are injected into the context window. The model generates an answer from the context. This avoids hallucination on factual questions. It also allows the model to answer about recent events or private data it was not trained on."""

```

chunks = sentence_chunk(doc, "rag_intro", sentences_per_chunk=3, overlap_sentences=1)
for c in chunks:
    print(f"  Chunk{c.chunk_index}:{c.text[:80]}...")

```

Choosing a strategy

Document type	Recommended strategy
Technical docs, code, structured text	Recursive split, chunk_size 400–600

Document type	Recommended strategy
Prose, articles, narratives	Sentence-aware, 4–6 sentences
Short Q&A pairs	Fixed-size, chunk_size 128–256
PDFs with headers	Header-aware splitting (split on <code>\n#</code>)
Code files	Function/class-aware splitting

15.3 Embedding and Indexing Pipeline

```

import os, json
import numpy as np
import chromadb
from openai import OpenAI
from pathlib import Path
from dataclasses import dataclass, field
from dotenv import load_dotenv

load_dotenv()

openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

# — Embedding —

def embed_texts(texts: list[str], model: str = "text-embedding-3-small") -> list[list[float]]:
    """Embed a list of texts. Handles batching for large inputs."""
    BATCH_SIZE = 100 # OpenAI accepts up to 2048 per call; use 100 for safety
    all_embeddings: list[list[float]] = []

    for i in range(0, len(texts), BATCH_SIZE):
        batch = texts[i : i + BATCH_SIZE]
        response = openai_client.embeddings.create(model=model, input=batch)
        all_embeddings.extend([e.embedding for e in response.data])

```

```

l, input=batch)
    batch_embeddings = [e.embedding for e in sorted(response.data, key=lambda x: x.index)]
    all_embeddings.extend(batch_embeddings)

    return all_embeddings

# — Indexing pipeline —————

class RAGIndexer:
    """
    Builds and manages a ChromaDB collection for RAG retrieval.

    Accepts pre-chunked Chunk objects, embeds them, and stores with metadata.
    """

    def __init__(self, collection_name: str, persist_dir: str = "/tmp/rag_index"):
        self._client = chromadb.PersistentClient(path=persist_dir)
        self._collection = self._client.get_or_create_collection(
            name=collection_name,
            metadata={"hnsw:space": "cosine"},
        )

    def index_chunks(self, chunks: list[Chunk], extra_metadata: dict | None = None) -> None:
        """Embed and store chunks. Skips chunks already present by ID."""
        new_chunks = []
        for chunk in chunks:
            chunk_id = f"{chunk.doc_id}_{chunk.index}"

```

```

        existing = self._collection.get(ids=[chunk_id])
        if not existing["ids"]:
            new_chunks.append((chunk_id, chunk))

    if not new_chunks:
        print("All chunks already indexed.")
        return

    texts      = [c.text for _, c in new_chunks]
    embeddings = embed_texts(texts)
    ids        = [cid for cid, _ in new_chunks]
    metadatas  = [
        {
            "doc_id":      c.doc_id,
            "chunk_index": c.chunk_index,
            "char_start":  c.char_start,
            "char_end":    c.char_end,
            **({extra_metadata or {}}),
        }
        for _, c in new_chunks
    ]

    self._collection.add(
        ids=ids,
        documents=texts,
        embeddings=embeddings,
        metadatas=metadatas,
    )
    print(f"Indexed{len(new_chunks)} new chunks. Total:{self._collection.count()}")

    def delete_document(self, doc_id: str) -> None:
        """Remove all chunks belonging to a document."""
        results = self._collection.get(where={"doc_id": {"$eq": doc_id}})
        if results["ids"]:

```

```

        self._collection.delete(ids=results["ids"])
        print(f"Deleted{len(results['ids'])} chunks for d
oc '{doc_id}'")

    @property
    def count(self) -> int:
        return self._collection.count()

```

15.4 The Retrieval Pipeline

```

from dataclasses import dataclass

@dataclass
class RetrievedChunk:
    text: str
    doc_id: str
    chunk_index: int
    score: float
    metadata: dict

class RAGRetriever:
    """
    Retrieves relevant chunks from a ChromaDB collection.
    Supports semantic search and metadata pre-filtering.
    """

    def __init__(self, collection):
        self._collection = collection

    def retrieve(
        self,
        query: str,
        k: int = 5,
        metadata_filter: dict | None = None,
    ) -> list[RetrievedChunk]:

```

```

        """Embed the query and return the top-k most relevant
        chunks."""
        q_embedding = embed_texts([query])[0]

        query_kwargs: dict = {
            "query_embeddings": [q_embedding],
            "n_results": min(k, self._collection.count
            ()),
            "include": ["documents", "distances", "m
            etadatas"],
        }
        if metadata_filter:
            query_kwargs["where"] = metadata_filter

        results = self._collection.query(**query_kwargs)

        chunks = []
        for text, dist, meta in zip(
            results["documents"][0],
            results["distances"][0],
            results["metadatas"][0],
        ):
            # ChromaDB cosine distance: 0 = identical, 2 = op
            posite

            # Convert to similarity score in [0, 1]
            similarity = 1 - (dist / 2)
            chunks.append(RetrievedChunk(
                text=text,
                doc_id=meta.get("doc_id", ""),
                chunk_index=meta.get("chunk_index", 0),
                score=round(similarity, 4),
                metadata=meta,
            ))

```

```
        return sorted(chunks, key=lambda c: c.score, reverse=True)
```

15.5 Context Injection and Prompt Construction

How you format retrieved context into the prompt significantly affects answer quality.

```
from dataclasses import dataclass

@dataclass
class RAGPromptBuilder:
    """
    Assembles the final prompt from a query and retrieved chunks.
    Handles context window budgeting and citation markers.
    """

    system_template: str = """You are a helpful assistant. Answer the user's question using ONLY the provided context. If the context does not contain enough information to answer, say "I don't have enough information to answer that."

    Do not make up facts. Do not use knowledge outside the provided context."""

    max_context_chars: int = 8000  # rough proxy for token budget

    def build(
        self,
        query: str,
        chunks: list[RetrievedChunk],
```



```

        min_score: float = 0.3,
    ) -> tuple[str, str]:
        """
        Returns (system_prompt, user_message).
        Filters low-score chunks and respects context budget.
        """
        # Filter by minimum relevance score
        relevant = [c for c in chunks if c.score >= min_score]

    e]

    if not relevant:
        context = "No relevant context found."
    else:
        # Build context with source labels, respect char
        budget

        context_parts = []
        total_chars = 0

        for i, chunk in enumerate(relevant):
            source_label = f"[Source{i+1}]:{chunk.doc_id},
            chunk{chunk.chunk_index}]"
            block = f"{source_label}\n{chunk.text}"

            if total_chars + len(block) > self.max_context_chars:
                break

            context_parts.append(block)
            total_chars += len(block)

        context = "\n\n---\n\n".join(context_parts)

        user_message = f"""Context:
{context}

```

```

---

Question:{query}"""

    return self.system_template, user_message

# Usage
builder = RAGPromptBuilder()
chunks = [
    RetrievedChunk("RAG retrieves documents at inference time.", "guide", 0, 0.92, {}),
    RetrievedChunk("Chunking splits documents into embeddable pieces.", "guide", 1, 0.78, {}),
]
system, user = builder.build("How does RAG handle long documents?", chunks)
print("System:", system[:80], "...")
print("\nUser:", user[:200], "...")

```

15.6 Reranking

The first retrieval pass (embedding similarity) is fast but imprecise. Reranking uses a more expensive cross-encoder model or LLM-as-judge to score each candidate more accurately.

```

import os
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

def llm_rerank(
    query: str,
    chunks: list[RetrievedChunk],

```

```

    top_k: int = 3,
) -> list[RetrievedChunk]:
    """
    Use an LLM to score each candidate chunk's relevance to the query.
    More accurate than embedding similarity but ~5-10x more expensive.
    Use for top-k=20 → rerank → return top-3.
    """
    if not chunks:
        return []

    # Format candidates for scoring
    candidate_texts = "\n\n".join(
        f"[Candidate{i+1}]\n{chunk.text}"
        for i, chunk in enumerate(chunks)
    )

    prompt = f"""Score each candidate's relevance to the query on a scale of 0-10.
    Return ONLY a JSON array of integers, one per candidate, in order.

    Query:{query}

    Candidates:
    {candidate_texts}

    Return format: [score1, score2, ...]"""

    response = client.chat.completions.create(
        model="gpt-4o-mini", # cheap model fine for scoring
        max_tokens=256,
        temperature=0,
        response_format={"type": "json_object"},
        messages=[

```

```

        {"role": "system", "content": "Return only valid
JSON."},
        {"role": "user", "content": prompt},
    ],
)

import json
try:
    raw = json.loads(response.choices[0].message.conte
nt)
    # Handle both {"scores": [...]} and [...] formats
    scores = raw if isinstance(raw, list) else list(raw.v
alues())[0]
    scores = [float(s) for s in scores]
except (json.JSONDecodeError, KeyError, TypeError, ValueE
rror):
    # Fall back to original order if parsing fails
    return chunks[:top_k]

# Apply LLM scores and sort
scored = [
    (chunk, llm_score)
    for chunk, llm_score in zip(chunks, scores)
]
scored.sort(key=lambda x: x[1], reverse=True)

# Return top_k chunks with updated scores
reranked = []
for chunk, llm_score in scored[:top_k]:
    chunk.score = llm_score / 10.0 # normalise to [0,
1]
    reranked.append(chunk)

return reranked

# Retrieval pipeline with reranking

```

```
def retrieve_and_rerank(
    query: str,
    retriever: "RAGRetriever",
    first_pass_k: int = 10,
    final_k: int = 3,
) -> list[RetrievedChunk]:
    """Two-stage retrieval: fetch more than needed, then rerank."""
    candidates = retriever.retrieve(query, k=first_pass_k)
    if len(candidates) <= final_k:
        return candidates
    return llm_rerank(query, candidates, top_k=final_k)
```

15.7 End-to-End RAG Pipeline

```
import os
from openai import OpenAI
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()

openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
anthropic_client = Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

class RAGPipeline:
    """
    Complete RAG pipeline: index → retrieve → rerank → generate.
    """

    def __init__(
        self,
```

```

        indexer: "RAGIndexer",
        retriever: "RAGRetriever",
        use_reranking: bool = False,
        first_pass_k: int = 10,
        final_k: int = 3,
    ):
        self.indexer = indexer
        self.retriever = retriever
        self.prompt_builder = RAGPromptBuilder()
        self.use_reranking = use_reranking
        self.first_pass_k = first_pass_k
        self.final_k = final_k

    def add_documents(
        self,
        texts: list[str],
        doc_ids: list[str],
        strategy: str = "recursive",
        **chunk_kwargs,
    ) -> None:
        """Chunk, embed, and index a list of documents."""
        all_chunks: list[Chunk] = []
        for text, doc_id in zip(texts, doc_ids):
            if strategy == "fixed":
                chunks = fixed_char_chunk(text, doc_id, **chunk_kwargs)
            elif strategy == "sentence":
                chunks = sentence_chunk(text, doc_id, **chunk_kwargs)
            else:
                chunks = recursive_split(text, doc_id, **chunk_kwargs)
            all_chunks.extend(chunks)

        self.indexer.index_chunks(all_chunks)

```

```

def query(
    self,
    question: str,
    metadata_filter: dict | None = None,
    model: str = "claude-sonnet-4-5",
) -> dict:
    """Full RAG query: retrieve → (rerank) → generate → r
    eturn."""

    # 1. Retrieve
    if self.use_reranking:
        chunks = retrieve_and_rerank(
            question, self.retriever,
            first_pass_k=self.first_pass_k,
            final_k=self.final_k,
        )
    else:
        chunks = self.retriever.retrieve(
            question, k=self.final_k, metadata_filter=met
adata_filter
        )

    # 2. Build prompt
    system, user = self.prompt_builder.build(question, ch
unks)

    # 3. Generate
    if "claude" in model:
        response = anthropic_client.messages.create(
            model=model,
            max_tokens=1024,
            system=system,
            messages=[{"role": "user", "content": user}],
        )
        answer = response.content[0].text
    else:

```

```

        response = openai_client.chat.completions.create(
            model=model,
            max_tokens=1024,
            messages=[
                {"role": "system", "content": system},
                {"role": "user", "content": user},
            ],
        )
        answer = response.choices[0].message.content

    return {
        "question": question,
        "answer": answer,
        "sources": [{"doc_id": c.doc_id, "score":
c.score} for c in chunks],
        "chunks_used": len(chunks),
    }

```

15.8 Module 15 Exercises

1. Build a document ingestion CLI: `python ingest.py --input-dir ./docs --chunk-strategy recursive --chunk-size 400 --overlap 50`. It should scan all `.txt` and `.md` files, chunk them, and index them into a named ChromaDB collection.
2. Implement a `ChunkQualityScorer` that rates chunks on: average sentence length, vocabulary richness (unique words / total words), and whether the chunk ends mid-sentence. Filter out chunks scoring below a threshold.
3. Compare three chunk sizes (256, 512, 1024 characters) on the same 5-document corpus by measuring retrieval Precision@3 against a set of 10 hand-labelled query-answer pairs.
4. Add query expansion to the retrieval pipeline: before embedding the query, use an LLM to generate 3 semantically different paraphrases of the query, embed all 4 versions, and merge results using reciprocal rank fusion.

Module 16 - Advanced RAG Patterns

HyDE · Self-query · Multi-hop · Reciprocal Rank Fusion · RAGAS evaluation

16.1 HyDE - Hypothetical Document Embeddings

Embedding a short query often does not match the embedding space of longer, information-dense documents. HyDE solves this by asking the LLM to generate a hypothetical answer document, then embedding that document for retrieval. The hypothesis is usually much closer to the actual document in embedding space.

```
import os
from openai import OpenAI
from anthropic import Anthropic
import numpy as np
from dotenv import load_dotenv

load_dotenv()

openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
anthropic_client = Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

def generate_hypothetical_document(query: str) -> str:
    """
    Ask the LLM to write a short passage that WOULD answer the
    query.
    This passage is then used for retrieval instead of the query itself.
    """
    response = anthropic_client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=300,
        system="""Write a short, factual passage (3-5 sentences) that would directly answer"""
```

the given question. Write as if you are the ideal document that should be retrieved.

Be specific and informative. Do not say 'this passage' - just write the content."""

```
        messages=[{"role": "user", "content": f"Question:{query}"}],
    )
    return response.content[0].text
```

```
def hyde_retrieve(
    query: str,
    retriever: "RAGRetriever",
    k: int = 5,
    use_both: bool = True,
) -> list["RetrievedChunk"]:
    """
    HyDE retrieval:
    1. Generate a hypothetical answer document
    2. Embed the hypothesis
    3. Retrieve using the hypothesis embedding
```

If use_both=True, also retrieve with the original query and merge results.

```
    """
    # Generate hypothesis
    hypothesis = generate_hypothetical_document(query)
    print(f"Hypothesis:{hypothesis[:100]}...")

    # Embed hypothesis for retrieval
    h_embedding = embed_texts([hypothesis])[0]
    q_embedding = embed_texts([query])[0]

    # Retrieve with hypothesis embedding
    h_results = retriever._collection.query(
        query_embeddings=[h_embedding], n_results=k,
        include=["documents", "distances", "metadatas"]
```

```

)

chunks: list[RetrievedChunk] = []
seen_ids: set[str] = set()

for text, dist, meta in zip(
    h_results["documents"][0],
    h_results["distances"][0],
    h_results["metadatas"][0],
):
    chunk_id = f"{meta.get('doc_id')}__{text[:30]}"
    if chunk_id not in seen_ids:
        seen_ids.add(chunk_id)
        chunks.append(RetrievedChunk(
            text=text, doc_id=meta.get("doc_id", ""),
            chunk_index=meta.get("chunk_index", 0),
            score=round(1 - dist / 2, 4), metadata=meta,
        ))

if use_both:
    # Also retrieve with original query and add unique re
sults
    q_results = retriever._collection.query(
        query_embeddings=[q_embedding], n_results=k,
        include=["documents", "distances", "metadatas"]
    )
    for text, dist, meta in zip(
        q_results["documents"][0],
        q_results["distances"][0],
        q_results["metadatas"][0],
    ):
        chunk_id = f"{meta.get('doc_id')}__{text[:30]}"
        if chunk_id not in seen_ids:
            seen_ids.add(chunk_id)
            chunks.append(RetrievedChunk(
                text=text, doc_id=meta.get("doc_id", ""),

```

```

        chunk_index=meta.get("chunk_index", 0),
        score=round(1 - dist / 2, 4), metadata=meta,
    ))

    return sorted(chunks, key=lambda c: c.score, reverse=True)[:k]

```

16.2 Self-Query Retrieval

Self-query uses an LLM to parse the natural language query into a structured query with both a semantic search component and metadata filters. This lets users say “Find Python tutorials from 2024” and have the system automatically filter by `language=python` and `year=2024`.

```

import os, json
from openai import OpenAI
from pydantic import BaseModel, Field
from typing import Any
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

class MetadataFilter(BaseModel):
    field: str = Field(description="Metadata field name to filter on")
    operator: str = Field(description="Comparison operator: eq, ne, gt, gte, lt, lte, in")
    value: Any = Field(description="Value to compare against")

class ParsedQuery(BaseModel):
    semantic_query: str = Field(description="The core semantic search query, stripped of filter conditions")

```

```

ns")
    filters: list[MetadataFilter] = Field(description
="Metadata filters to apply before semantic search")
    explanation: str = Field(description
="Brief explanation of what was extracted")

def self_query_parse(
    natural_language_query: str,
    available_metadata_fields: dict[str, str],
) -> ParsedQuery:
    """
    Parse a natural language query into a semantic query + me
tadata filters.

    Args:
        natural_language_query: The user's raw query
        available_metadata_fields: Dict of {field_name: descr
iption}
    """
    fields_desc = "\n".join(
        f" -{name}:{desc}" for name, desc in available_metad
ata_fields.items()
    )

    response = client.chat.completions.create(
        model="gpt-4o",
        max_tokens=512,
        temperature=0,
        response_format={"type": "json_object"},
        messages=[
            {
                "role": "system",
                "content": f"""You parse natural language que
ries into structured search queries.
Extract the core semantic intent and any metadata filters.

```

Available metadata fields:

```
{fields_desc}
```

Return JSON matching this schema:

```
{{
  "semantic_query": "string - the core topic to search for",
  "filters": [
    {"field": "field_name", "operator": "eq|ne|gt|gte|lt|lte|i
n", "value": <value>}}
  ],
  "explanation": "string - what you extracted"
}}""",
    },
    {"role": "user", "content": natural_language_quer
y},
  ],
)
```

```
data = json.loads(response.choices[0].message.content)
return ParsedQuery(**data)
```

```
def apply_filters_to_chroma(filters: list[MetadataFilter]) ->
dict | None:
    """Convert ParsedQuery filters to ChromaDB where claus
e."""
```

```
    if not filters:
        return None
```

```
    op_map = {
        "eq": "$eq", "ne": "$ne",
        "gt": "$gt", "gte": "$gte",
        "lt": "$lt", "lte": "$lte",
        "in": "$in",
    }
```

```
    if len(filters) == 1:
```

```

        f = filters[0]
        return {f.field: {op_map[f.operator]: f.value}}

    return {
        "$and": [
            {f.field: {op_map[f.operator]: f.value}}
            for f in filters
        ]
    }

# Example usage
METADATA_FIELDS = {
    "category": "Document category: tutorial, reference, case
_study, benchmark",
    "year": "Publication year as integer (e.g. 2024)",
    "language": "Programming language: python, javascript, ru
st, etc.",
    "difficulty": "Content difficulty: beginner, intermediat
e, advanced",
}

queries = [
    "Find Python tutorials about RAG from 2024",
    "Advanced benchmarks comparing LLM retrieval methods",
    "Beginner-friendly introduction to vector databases",
]

for query in queries:
    parsed = self_query_parse(query, METADATA_FIELDS)
    chroma_filter = apply_filters_to_chroma(parsed.filters)
    print(f"Query:{query}")
    print(f"Semantic:{parsed.semantic_query}")
    print(f"Chroma filter:{chroma_filter}")
    print(f"Explanation:{parsed.explanation}")
    print()

```

16.3 Reciprocal Rank Fusion

When you retrieve from multiple sources (different collections, BM25 + vector, HyDE + direct), Reciprocal Rank Fusion (RRF) merges ranked lists without requiring score normalisation.

```
from dataclasses import dataclass
from collections import defaultdict

@dataclass
class RankedResult:
    id: str
    text: str
    score: float

def reciprocal_rank_fusion(
    ranked_lists: list[list[RankedResult]],
    k: int = 60,          # RRF constant - higher k reduces
    impact of top ranks
    top_n: int = 5,
) -> list[RankedResult]:
    """
    Merge multiple ranked result lists using RRF.

    RRF score = sum(1 / (k + rank)) across all lists.
    Works well even when individual list scores are on different
    scales.

    Args:
        ranked_lists: Each inner list is a ranked list of results
        k: Smoothing constant (60 is the standard default)
        top_n: Number of results to return
    """
```



```

rrf_scores: dict[str, float] = defaultdict(float)
result_lookup: dict[str, RankedResult] = {}

for ranked_list in ranked_lists:
    for rank, result in enumerate(ranked_list, start=1):
        rrf_scores[result.id] += 1.0 / (k + rank)
        result_lookup[result.id] = result

# Sort by RRF score descending
sorted_ids = sorted(rrf_scores, key=lambda rid: rrf_scores[rid], reverse=True)

merged = []
for rid in sorted_ids[:top_n]:
    result = result_lookup[rid]
    merged.append(RankedResult(
        id=rid, text=result.text, score=round(rrf_scores[rid], 6)
    ))

return merged

# Example: merge BM25 results and vector search results
bm25_results = [
    RankedResult("d3", "BM25 is a keyword retrieval algorithm", 0.91),
    RankedResult("d1", "RAG combines retrieval and generation", 0.78),
    RankedResult("d5", "Hybrid search blends BM25 and vector search", 0.65),
    RankedResult("d2", "Vector databases store embeddings", 0.40),
]

vector_results = [
    RankedResult("d1", "RAG combines retrieval and generation", 0.78),

```

```

n",
    0.95),
    RankedResult("d5", "Hybrid search blends BM25 and vector
search", 0.88),
    RankedResult("d4", "Chunking splits documents into small
r pieces", 0.72),
    RankedResult("d2", "Vector databases store embeddings",
0.61),
]

merged = reciprocal_rank_fusion([bm25_results, vector_result
s], k=60, top_n=4)
print("RRF merged results:")
for r in merged:
    print(f" [{r.score:.6f}]{r.text}")

```

16.4 Hybrid Search - BM25 + Dense

```

import re, numpy as np
from rank_bm25 import BM25Okapi
from dataclasses import dataclass, field

@dataclass
class HybridDocument:
    id: str
    text: str
    embedding: list[float] = field(default_factory=list)
    metadata: dict = field(default_factory=dict)

def tokenize(text: str) -> list[str]:
    return re.findall(r'\w+', text.lower())

class HybridSearchEngine:
    """
    Combines BM25 sparse retrieval with dense embedding retrie
eval.

```

```

    alpha controls the blend: 1.0 = pure dense, 0.0 = pure BM
25.
    """

    def __init__(self, alpha: float = 0.5):
        self.alpha = alpha
        self._docs: list[HybridDocument] = []
        self._bm25: BM25kapi | None = None
        self._matrix: np.ndarray | None = None # (n, dim) no
        rmalised embeddings

    def add_documents(self, docs: list[HybridDocument]) -> No
    ne:
        self._docs = docs
        # Build BM25 index
        self._bm25 = BM25kapi([tokenize(d.text) for d in doc
        s])

        # Build dense matrix
        embs = np.array([d.embedding for d in docs], dtype=n
        p.float32)
        norms = np.linalg.norm(embs, axis=1, keepdims=True)
        self._matrix = embs / np.where(norms == 0, 1, norms)

    def search(self, query: str, query_embedding: list[floa
    t], k: int = 5) -> list[tuple[HybridDocument, float]]:
        if not self._docs or self._bm25 is None:
            raise RuntimeError("No documents indexed.")

        # BM25 scores - normalise to [0, 1]
        bm25_raw = np.array(self._bm25.get_scores(tokenize(qu
        ery)), dtype=np.float32)
        bm25_max = bm25_raw.max()
        bm25_scores = bm25_raw / bm25_max if bm25_max > 0 els
        e bm25_raw

        # Dense scores - cosine similarity (dot product of no

```

```

rmalised vectors)
    q_vec = np.array(query_embedding, dtype=np.float32)
    q_norm = np.linalg.norm(q_vec)
    q_vec = q_vec / q_norm if q_norm > 0 else q_vec
    dense_scores = self._matrix @ q_vec
    dense_scores = (dense_scores + 1.0) / 2.0 # shift [-
1,1] → [0,1]

    # Weighted combination
    final_scores = self.alpha * dense_scores + (1 - self.
alpha) * bm25_scores

    top_idx = np.argsort(final_scores)[::-1][:k]
    return [(self._docs[int(i)], float(final_scores[i]))
for i in top_idx]

# Usage
rng = np.random.default_rng(42)
dim = 8 # small for illustration; use 1536 with real embed
dings

corpus_texts = [
    "RAG combines retrieval and generation for question answe
ring",
    "BM25 is a sparse retrieval algorithm based on term frequ
ency inverse document frequency",
    "Vector databases store dense embeddings for semantic sim
ilarity search",
    "Hybrid search blends BM25 keyword matching with dense em
bedding retrieval",
    "Chunking splits documents into pieces small enough to em
bed accurately",
]

docs = [
    HybridDocument(

```

```

        id=f"d{i}",
        text=text,
        embedding=rng.standard_normal(dim).tolist(),
    )
    for i, text in enumerate(corpus_texts)
]

engine = HybridSearchEngine(alpha=0.4)    # 40% dense, 60% BM25
engine.add_documents(docs)

query      = "how to blend keyword and semantic search"
q_embed    = rng.standard_normal(dim).tolist()
results    = engine.search(query, q_embed, k=3)

print("Hybrid search results:")
for doc, score in results:
    print(f" [{score:.4f}]{doc.text}")

```

16.5 Multi-Hop Reasoning

Some questions require multiple retrieval steps - the answer to step one informs what to retrieve in step two. This is called multi-hop retrieval.

```

import os, json
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

def extract_follow_up_queries(
    original_question: str,
    retrieved_context: str,
    answer_so_far: str,

```

```

) -> list[str]:
    """
    After a first retrieval round, ask the LLM if additional
    queries
    are needed to fully answer the question.
    """
    response = client.chat.completions.create(
        model="gpt-4o",
        max_tokens=256,
        temperature=0,
        response_format={"type": "json_object"},
        messages=[
            {
                "role": "system",
                "content": """Analyse whether the retrieved c
ontext fully answers the question.
If additional information is needed, return follow-up search
queries.

Return JSON:
{"needs_more": true|false, "follow_up_queries": ["query1", "q
uery2"]}""",
            },
            {
                "role": "user",
                "content": f"""Original question:{original_qu
estion}

Retrieved context:
{retrieved_context}

Partial answer:{answer_so_far}

Does the context fully answer the question? If not, what else
needs to be retrieved?""",
            },

```

```

    ],
)
return json.loads(response.choices[0].message.content)

def multi_hop_rag(
    question: str,
    retriever: "RAGRetriever",
    max_hops: int = 3,
    k_per_hop: int = 3,
) -> dict:
    """
    Multi-hop RAG: iteratively retrieve and check if more con
    text is needed.

    Hop 1: Retrieve for original question
    Hop 2+: Use LLM to decide if more retrieval is needed and
    what to search for
    """
    all_chunks: list[RetrievedChunk] = []
    seen_ids: set[str] = set()
    queries_used = [question]

    for hop in range(max_hops):
        current_query = queries_used[-1]
        new_chunks = retriever.retrieve(current_query, k=k
_per_hop)

        for chunk in new_chunks:
            chunk_id = f"{chunk.doc_id}_{chunk.chunk_index}"
            if chunk_id not in seen_ids:
                seen_ids.add(chunk_id)
                all_chunks.append(chunk)

    if hop == max_hops - 1:
        break # last hop - skip follow-up check

```

```

        # Check if we need another hop
        context_so_far = "\n\n".join(c.text for c in all_chunks)

        check = extract_follow_up_queries(
            original_question=question,
            retrieved_context=context_so_far[:2000],  # truncate for the check
            answer_so_far="[not yet generated]",
        )

        if not check.get("needs_more") or not check.get("follow_up_queries"):
            break

        next_query = check["follow_up_queries"][0]
        queries_used.append(next_query)
        print(f"    Hop{hop + 2}: searching for '{next_query}'")

    return {
        "chunks": all_chunks,
        "hops": len(queries_used),
        "queries_used": queries_used,
    }

```

16.6 RAG Evaluation Framework

Evaluating RAG without a proper framework leads to false confidence. The key metrics are:

- **Context Precision** - are all retrieved chunks actually relevant?
- **Context Recall** - does the retrieved context contain all information needed?
- **Faithfulness** - does the answer stay within the retrieved context (no hallucination)?
- **Answer Relevance** - does the answer actually address the question?


```

import os, json
from openai import OpenAI
from dataclasses import dataclass
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

@dataclass
class RAGEvalResult:
    question: str
    answer: str
    context: str
    ground_truth: str
    faithfulness: float # 0-1: answer supported by context
    answer_relevance: float # 0-1: answer addresses the question
    context_recall: float # 0-1: context contains ground truth info

def evaluate_faithfulness(answer: str, context: str) -> float:
    """
    Score how faithfully the answer is grounded in the context.

    1.0 = every claim in the answer is supported by context
    0.0 = answer contains claims not in context
    """
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        max_tokens=128,
        temperature=0,
        response_format={"type": "json_object"},
        messages=[

```

```

        {
            "role": "system",
            "content": ""Score how faithfully the answer
is grounded in the context.
Return JSON: {"score": <float 0.0-1.0>, "reason": "<one sente
nce>"}""",
        },
        {
            "role": "user",
            "content": f"Context:\n{context}\n\nAnswer:\n
{answer}",
        },
    ],
)
data = json.loads(response.choices[0].message.content)
return float(data.get("score", 0.0))

def evaluate_answer_relevance(question: str, answer: str) ->
float:
    ""Score how well the answer addresses the question.""
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        max_tokens=128,
        temperature=0,
        response_format={"type": "json_object"},
        messages=[
            {
                "role": "system",
                "content": 'Score how well the answer address
es the question. Return JSON: {"score": <float 0-1>, "reaso
n": "<one sentence>"}',
            },
            {
                "role": "user",
                "content": f"Question:{question}\n\nAnswer:{a
nswer}",
            },
        ],
    )
    data = json.loads(response.choices[0].message.content)
    return float(data.get("score", 0.0))

```

```

        },
    ],
)
data = json.loads(response.choices[0].message.content)
return float(data.get("score", 0.0))

def evaluate_context_recall(context: str, ground_truth: str)
-> float:
    """Score whether the context contains the key information
    from ground truth."""
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        max_tokens=128,
        temperature=0,
        response_format={"type": "json_object"},
        messages=[
            {
                "role": "system",
                "content": 'Score how much of the ground truth
h information is present in the context. Return JSON: {"score": <float 0-1>, "reason": "<one sentence>"',
            },
            {
                "role": "user",
                "content": f"Ground truth:{ground_truth}\n\nContext:{context}",
            },
        ],
    )
    data = json.loads(response.choices[0].message.content)
    return float(data.get("score", 0.0))

def run_rag_eval(test_cases: list[dict], pipeline: "RAGPipeline") -> list[RAGEvalResult]:
    """
    Run evaluation across a test set.

```

```

Each test case: {"question": str, "ground_truth": str}
"""
results = []
for case in test_cases:
    output = pipeline.query(case["question"])
    context = "\n\n".join(c["doc_id"] for c in output["sources"]) # simplified

    faithfulness = evaluate_faithfulness(output["answer"], context)
    answer_relevance = evaluate_answer_relevance(case["question"], output["answer"])
    context_recall = evaluate_context_recall(context, case["ground_truth"])

    results.append(RAGEvalResult(
        question=case["question"],
        answer=output["answer"],
        context=context,
        ground_truth=case["ground_truth"],
        faithfulness=faithfulness,
        answer_relevance=answer_relevance,
        context_recall=context_recall,
    ))

return results

def print_eval_report(results: list[RAGEvalResult]) -> None:
    print(f"{'Metric':<22}{'Mean':>8}{'Min':>8}{'Max':>8}")
    print("-" * 50)
    for metric in ["faithfulness", "answer_relevance", "context_recall"]:
        scores = [getattr(r, metric) for r in results]
        print(f"{metric:<22}{sum(scores)/len(scores):>8.3f}{m

```

```
in(scores) :>8.3f}{max(scores) :>8.3f}")
print(f"\nTotal evaluated:{len(results)} questions")
```

16.7 Module 16 Exercises

1. Implement HyDE on your RAG pipeline from Module 15. Compare retrieval Precision@3 with and without HyDE on 10 test queries. Which query types benefit most from HyDE?
2. Build a complete self-query system: define a schema for 5 metadata fields, generate 15 test queries that mix semantic search with filters, parse all 15 with your LLM, and verify the extracted filters are correct.
3. Implement RRF to merge results from: (a) BM25 retrieval, (b) dense retrieval with `text-embedding-3-small`, and (c) HyDE retrieval. Compare RRF against each individual method using Precision@3.
4. Build a `RAGEvalDashboard` that runs the evaluation framework from section 16.6 on 20 test cases, saves results to CSV, and prints a report table showing per-question scores and aggregate statistics.

Module 17 - Memory & State

Short/long-term memory · Entity memory · Summary compression · Cross-session persistence

17.1 Memory in LLM Applications

LLMs have no memory between API calls - every call starts blank. Memory is entirely your responsibility. The four main memory types each solve a different problem:

Type	What it stores	When to use
Window memory	Last N message turns	Most conversational apps
Summary memory	Compressed older context	Long conversations, cost reduction

Type	What it stores	When to use
Entity memory	Facts about named entities	Customer service, personal assistants
Vector memory	Semantically searchable history	Research assistants, large history

17.2 Window Memory

```

from dataclasses import dataclass, field
from typing import Optional
import time

@dataclass
class Message:
    role: str # "user" | "assistant" | "system"
    content: str
    timestamp: float = field(default_factory=time.time)

class WindowMemory:
    """
    Keeps the last `max_turns` user-assistant turn pairs.
    Oldest messages are silently dropped when the limit is exceeded.
    """

    def __init__(self, max_turns: int = 10):
        self.max_turns = max_turns
        self._messages: list[Message] = []

    def add(self, role: str, content: str) -> None:
        if role not in ("user", "assistant"):
            raise ValueError(f"role must be 'user' or 'assistant', got{role!r}")
        self._messages.append(Message(role=role, content=content))

```

```

        max_messages = self.max_turns * 2
        if len(self._messages) > max_messages:
            self._messages = self._messages[-max_messages:]

    def get_messages(self, system_prompt: Optional[str] = None) -> list[dict]:
        """Return messages ready for an LLM API call."""
        result = []
        if system_prompt:
            result.append({"role": "system", "content": system_prompt})
        result.extend({"role": m.role, "content": m.content}
            for m in self._messages)
        return result

    def clear(self) -> None:
        self._messages.clear()

    def token_estimate(self, chars_per_token: float = 4.0) -> int:
        total_chars = sum(len(m.content) for m in self._messages)
        return int(total_chars / chars_per_token)

    def __len__(self) -> int:
        return len(self._messages)

    def __repr__(self) -> str:
        return f"WindowMemory(turns={len(self._messages)//2},
            max={self.max_turns})"

# Usage
mem = WindowMemory(max_turns=3)
pairs = [
    ("user", "What is RAG?"),
    ("assistant", "RAG stands for Retrieval-Augmented Generat

```

```

ion."),
    ("user", "How does it differ from fine-tuning?"),
    ("assistant", "Fine-tuning updates model weights; RAG retrieves at inference time."),
    ("user", "Which should I use for customer support?"),
    ("assistant", "RAG is better for dynamic or frequently-updated knowledge bases."),
    ("user", "What about costs?"),
    ("assistant", "RAG has per-query retrieval costs; fine-tuning has upfront training costs."),
]

for role, content in pairs:
    mem.add(role, content)

print(repr(mem))
messages = mem.get_messages(system_prompt="You are a helpful AI assistant.")
print(f"API messages:{len(messages)}")
for m in messages:
    print(f" [{m['role']}] {m['content'][:60]}")

```

17.3 Summary Memory

When conversations grow long, window memory drops old context abruptly. Summary memory compresses older turns into a running summary, preserving the gist at lower cost.

```

import os
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

```



```

class SummaryMemory:
    """
    Maintains a sliding window of recent messages plus a compressed
    summary of older context. When the window overflows, the oldest
    turns are summarised and appended to the running summary.
    """

    def __init__(self, max_recent_turns: int = 6, summary_model: str = "claude-haiku-4-5"):
        self.max_recent = max_recent_turns
        self.summary_model = summary_model
        self._recent: list[Message] = []
        self._summary: str = ""

    def add(self, role: str, content: str) -> None:
        self._recent.append(Message(role=role, content=content))

        if len(self._recent) > self.max_recent * 2:
            # Compress the oldest half
            to_compress = self._recent[:self.max_recent]
            self._recent = self._recent[self.max_recent:]
            self._summary = self._compress(to_compress)

    def _compress(self, messages: list[Message]) -> str:
        """Summarise a list of messages into a single paragraph. """
        formatted = "\n".join(
            f"{m.role.upper()}:{m.content}" for m in messages
        )
        new_summary_prompt = f"""Summarise this conversation segment concisely (2-3 sentences). Preserve key facts, decisions, and user preferences. Omit ple

```

```
asantries.
```

```
{f"Previous summary:{self._summary}" if self._summary else  
""}
```

Conversation segment:

```
{formatted}"""
```

```
        response = client.messages.create(  
            model=self.summary_model,  
            max_tokens=256,  
            messages=[{"role": "user", "content": new_summary  
_prompt}],  
        )  
        return response.content[0].text.strip()
```

```
def get_messages(self, system_prompt: str = "") -> list[dict]:
```

```
    """Build API message list with summary prepended to system."""
```

```
    result = []  
    effective_system = system_prompt  
    if self._summary:  
        effective_system += f"\n\nConversation summary so far:\n{self._summary}"
```

```
    if effective_system:  
        result.append({"role": "system", "content": effective_system.strip()})
```

```
    result.extend(  
        {"role": m.role, "content": m.content} for m in self._recent  
    )  
    return result
```

```
def stats(self) -> dict:
    return {
        "recent_messages": len(self._recent),
        "summary_chars": len(self._summary),
        "has_summary": bool(self._summary),
    }
```

17.4 Entity Memory

Entity memory tracks structured facts about people, places, organisations, and concepts mentioned in the conversation.

```
import os, json, re
from openai import OpenAI
from dataclasses import dataclass, field
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

@dataclass
class Entity:
    name: str
    type: str # PERSON, ORG, PRODUCT, CONCEPT, etc.
    attributes: dict = field(default_factory=dict)

    def update(self, new_attributes: dict) -> None:
        self.attributes.update(new_attributes)

    def to_context_string(self) -> str:
        if not self.attributes:
            return f"{self.name} ({self.type})"
        attrs = "; ".join(f"{k}:{v}" for k, v in self.attributes.items())
```

```

        return f"{self.name} ({self.type}) -{attrs}"

class EntityMemory:
    """
    Extracts entities from conversation turns and maintains
    a structured fact store. Injects relevant entity facts
    into context before each LLM call.
    """

    def __init__(self):
        self._entities: dict[str, Entity] = {}

    def extract_and_update(self, text: str, role: str) -> list[str]:
        """Extract entities from a message and update the store."""
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            max_tokens=512,
            temperature=0,
            response_format={"type": "json_object"},
            messages=[
                {
                    "role": "system",
                    "content": """Extract entities and their
attributes from this text.
Return JSON:
{
  "entities": [
    {"name": "string", "type": "PERSON|ORG|PRODUCT|CONCEPT|PLACE", "attributes": {"key": "value"}}
  ]
}
Only include entities with at least one concrete attribute."""
                },
            ],
        )

```

```

        {"role": "user", "content": text},
    ],
)
try:
    data = json.loads(response.choices[0].message.con
tent)
except json.JSONDecodeError:
    return []

updated = []
for raw in data.get("entities", []):
    name = raw.get("name", "").strip()
    if not name:
        continue

    if name in self._entities:
        self._entities[name].update(raw.get("attribut
es", {}))
    else:
        self._entities[name] = Entity(
            name=name,
            type=raw.get("type", "UNKNOWN"),
            attributes=raw.get("attributes", {}),
        )
    updated.append(name)

return updated

def get_relevant_context(self, text: str) -> str:
    """Return entity facts for entities mentioned in the
text."""
    if not self._entities:
        return ""

    mentioned = [
        name for name in self._entities

```

```

        if name.lower() in text.lower():
    ]

    if not mentioned:
        return ""

    lines = ["Known context:"]
    for name in mentioned:
        lines.append(f" • {self._entities[name].to_context_string()}")

    return "\n".join(lines)

def get_all_entities(self) -> list[Entity]:
    return list(self._entities.values())

# Usage
em = EntityMemory()
messages = [
    "I'm working with Alice Chen, our ML engineer. She's been building a RAG system using LangChain and ChromaDB.",
    "Alice prefers to use Claude for generation and text-embedding-3-small for embeddings.",
    "Our company, DataFlow AI, is a 12-person startup focused on enterprise RAG solutions.",
]

for msg in messages:
    updated = em.extract_and_update(msg, role="user")
    print(f"Updated entities:{updated}")

query = "What does Alice use for embeddings at DataFlow AI?"
context = em.get_relevant_context(query)
print(f"\nEntity context for query:\n{context}")

```

17.5 Cross-Session Persistence with SQLite

In-memory memory objects disappear when your process ends. For persistent memory across sessions, use a database.

```
import sqlite3, json, time, os
from pathlib import Path
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class PersistentSession:
    """
    SQLite-backed memory store for cross-session persistence.
    Stores message history, entity facts, and session summaries.
    """

    db_path: str
    session_id: str
    max_window: int = 20

    def __post_init__(self):
        self._conn = sqlite3.connect(self.db_path, check_same_thread=False)
        self._init_schema()

    def _init_schema(self) -> None:
        self._conn.executescript("""
            CREATE TABLE IF NOT EXISTS messages (
                id            INTEGER PRIMARY KEY AUTOINCREMENT,
                session_id    TEXT      NOT NULL,
                role           TEXT      NOT NULL,
                content        TEXT      NOT NULL,
                timestamp      REAL      NOT NULL
            )
        """)
```

```

);

CREATE TABLE IF NOT EXISTS entities (
    session_id TEXT NOT NULL,
    name       TEXT NOT NULL,
    type       TEXT NOT NULL,
    attributes TEXT NOT NULL,
    updated_at REAL NOT NULL,
    PRIMARY KEY (session_id, name)
);

CREATE TABLE IF NOT EXISTS summaries (
    session_id TEXT PRIMARY KEY,
    content    TEXT NOT NULL,
    updated_at REAL NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_messages_session
ON messages (session_id, timestamp);
"""
self._conn.commit()

```

— Messages

```

def add_message(self, role: str, content: str) -> None:
    self._conn.execute(
        "INSERT INTO messages (session_id, role, content,
timestamp) VALUES (?, ?, ?, ?)",
        (self.session_id, role, content, time.time())
    )
    self._conn.commit()

def get_recent_messages(self, n: Optional[int] = None) ->
list[dict]:
    limit = n or self.max_window

```



```

rows = self._conn.execute(
    """SELECT role, content FROM messages
       WHERE session_id=?
       ORDER BY timestamp DESC LIMIT ?""",
    (self.session_id, limit)
).fetchall()
return [{"role": r[0], "content": r[1]} for r in reversed(rows)]

```

```

def message_count(self) -> int:
    return self._conn.execute(
        "SELECT COUNT(*) FROM messages WHERE session_id=?",
        (self.session_id,)
    ).fetchone()[0]

```

— Entities

```

def upsert_entity(self, name: str, entity_type: str, attributes: dict) -> None:
    existing = self._conn.execute(
        "SELECT attributes FROM entities WHERE session_id=? AND name=?",
        (self.session_id, name)
    ).fetchone()

    if existing:
        merged = json.loads(existing[0])
        merged.update(attributes)
    else:
        merged = attributes

    self._conn.execute(
        """INSERT OR REPLACE INTO entities
           (session_id, name, type, attributes, updated_a

```

```

t)
        VALUES (?, ?, ?, ?, ?)""" ,
        (self.session_id, name, entity_type, json.dumps(m
merged), time.time())
    )
    self._conn.commit()

def get_entities(self) -> list[dict]:
    rows = self._conn.execute(
        "SELECT name, type, attributes FROM entities WHER
E session_id=?",
        (self.session_id,)
    ).fetchall()
    return [
        {"name": r[0], "type": r[1], "attributes": json.l
oads(r[2])}
        for r in rows
    ]

# — Summary —————

def save_summary(self, summary: str) -> None:
    self._conn.execute(
        "INSERT OR REPLACE INTO summaries (session_id, co
ntent, updated_at) VALUES (?, ?, ?)",
        (self.session_id, summary, time.time())
    )
    self._conn.commit()

def load_summary(self) -> Optional[str]:
    row = self._conn.execute(
        "SELECT content FROM summaries WHERE session_id
=?",
        (self.session_id,)
    ).fetchone()

```

```

        return row[0] if row else None

    def close(self) -> None:
        self._conn.close()

# Usage - simulating two separate process runs
DB = "/tmp/session_memory.db"

# Session 1 - first conversation
s1 = PersistentSession(DB, session_id="user_alice_001")
s1.add_message("user", "I'm building a RAG pipeline with LangChain.")
s1.add_message("assistant", "Great! What vector database are you planning to use?")
s1.add_message("user", "Probably ChromaDB for local development and Pinecone for production.")
s1.upsert_entity("Alice", "PERSON", {"project": "RAG pipeline", "stack": "LangChain"})
s1.upsert_entity("ChromaDB", "PRODUCT", {"use_case": "local dev vector db"})
s1.save_summary("Alice is building a RAG pipeline using LangChain, ChromaDB locally, and Pinecone for production.")
s1.close()

# Session 2 - resumed later (simulates a new process)
s2 = PersistentSession(DB, session_id="user_alice_001")
print(f"Messages from previous session:{s2.message_count()}")
print(f"Summary:{s2.load_summary()}")
print(f"Entities:{s2.get_entities()}")
recent = s2.get_recent_messages(n=4)
print(f"Recent messages:{len(recent)}")
s2.close()

Path(DB).unlink()

```

17.6 Vector Memory - Searchable Long-Term History

For very long conversation histories, window memory discards old turns and summary memory loses detail. Vector memory embeds all turns and retrieves only the relevant ones.

```
import os, time, uuid
import numpy as np
import chromadb
from openai import OpenAI
from dataclasses import dataclass
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

def embed(texts: list[str]) -> list[list[float]]:
    resp = client.embeddings.create(model="text-embedding-3-small", input=texts)
    return [e.embedding for e in sorted(resp.data, key=lambda x: x.index)]

class VectorMemory:
    """
    Stores all conversation turns as embeddings.
    At query time, retrieves only the most semantically relevant past turns.
    Scales to thousands of turns without cost proportional to history length.
    """

    def __init__(self, session_id: str, persist_dir: str = "/tmp/vector_memory"):
        self.session_id = session_id
        self.chroma = chromadb.PersistentClient(path=persist_dir)
```

```

self._collection = chroma.get_or_create_collection(
    name=f"memory_{session_id.replace('-', '_')}",
    metadata={"hnsw:space": "cosine"},
)

def add_turn(self, role: str, content: str) -> None:
    """Embed and store a single message turn."""
    embedding = embed([content])[0]
    self._collection.add(
        ids=[str(uuid.uuid4())],
        documents=[content],
        embeddings=[embedding],
        metadatas=[{
            "role": role,
            "session": self.session_id,
            "timestamp": time.time(),
        }],
    )

def recall(
    self,
    query: str,
    k: int = 5,
    role_filter: Optional[str] = None,
) -> list[dict]:
    """Return the k most relevant past turns for a query.
    y."""
    n = min(k, self._collection.count())
    if n == 0:
        return []

    q_embedding = embed([query])[0]
    where = {"role": {"$eq": role_filter}} if role_filter else None

    kwargs: dict = {

```

```

        "query_embeddings": [q_embedding],
        "n_results":        n,
        "include":          ["documents", "metadatas", "d
instances"],
    }
    if where:
        kwargs["where"] = where

    results = self._collection.query(**kwargs)

    return [
        {
            "content":    doc,
            "role":       meta["role"],
            "timestamp":  meta["timestamp"],
            "relevance":  round(1 - dist / 2, 4),
        }
        for doc, meta, dist in zip(
            results["documents"][0],
            results["metadatas"][0],
            results["distances"][0],
        )
    ]

    def get_relevant_context(self, current_query: str, k: int
= 4) -> str:
        """Format recalled memories as a context string for t
he LLM."""
        memories = self.recall(current_query, k=k)
        if not memories:
            return ""

        # Sort by timestamp to restore chronological order
        memories.sort(key=lambda m: m["timestamp"])

        lines = ["Relevant conversation history:"]

```

```

        for m in memories:
            lines.append(f"{m['role'].upper()}: {m['content'][:120]}")

        return "\n".join(lines)

@property
def size(self) -> int:
    return self._collection.count()

# Usage
vm = VectorMemory("session_test_001", persist_dir="/tmp/vec_memory_test")

# Simulate a long conversation
turns = [
    ("user", "I'm building a RAG system. What vector database should I use?"),
    ("assistant", "For development, start with ChromaDB. For production, consider Pinecone or Weaviate."),
    ("user", "What embedding model do you recommend?"),
    ("assistant", "text-embedding-3-small from OpenAI is a great balance of quality and cost."),
    ("user", "How should I chunk my documents?"),
    ("assistant", "Use RecursiveCharacterTextSplitter with chunk_size=512 and overlap=64 for most cases."),
    ("user", "What LLM should I use for generation?"),
    ("assistant", "Claude Sonnet 4.5 or GPT-4o are both excellent for RAG generation tasks."),
    ("user", "I'm worried about hallucination."),
    ("assistant", "Add a faithfulness score check after generation using LLM-as-judge."),
]

for role, content in turns:
    vm.add_turn(role, content)

```

```

print(f"Stored{vm.size} turns\n")

new_query = "Remind me what you said about vector databases?"
context = vm.get_relevant_context(new_query, k=3)
print(context)

import shutil
shutil.rmtree("/tmp/vec_memory_test", ignore_errors=True)

```

17.7 Choosing the Right Memory Strategy

```

def recommend_memory_strategy(
    expected_turns_per_session: int,
    sessions_per_user: int,
    needs_entity_tracking: bool,
    privacy_sensitive: bool,
) -> dict:
    """
    Decision helper for memory architecture.
    Returns recommended primary and optional secondary strategy.
    """
    recommendations = []

    if expected_turns_per_session <= 20:
        primary = "WindowMemory"
        reason = "Short sessions fit entirely in window; simplest approach."
    elif expected_turns_per_session <= 100:
        primary = "SummaryMemory"
        reason = "Medium sessions benefit from compression of older context."
    else:
        primary = "VectorMemory"

```



```

        reason = "Long sessions need semantic retrieval to a
void context overflow."

    recommendations.append({"strategy": primary, "reason": re
ason})

    if needs_entity_tracking:
        recommendations.append({
            "strategy": "EntityMemory (add-on)",
            "reason": "Extract and track structured facts a
bout people, products, and concepts."
        })

    if sessions_per_user > 1:
        recommendations.append({
            "strategy": "PersistentSession (SQLite or Postgre
SQL)",
            "reason": "Cross-session persistence required f
or returning users."
        })

    if privacy_sensitive:
        recommendations.append({
            "note": "Use local storage only (SQLite, local Ch
romaDB). Avoid sending history to hosted vector DBs."
        })

    return {
        "primary": primary,
        "all_strategies": recommendations,
        "session_turns": expected_turns_per_session,
        "multi_session": sessions_per_user > 1,
    }

examples = [
    (8, 1, False, False), # simple chatbot

```

```

        (50, 5, True, False),    # customer support
        (500, 20, True, True),   # personal assistant, privacy-
sensitive
    ]

for turns, sessions, entities, private in examples:
    rec = recommend_memory_strategy(turns, sessions, entities, private)
    print(f"Turns={turns}, Sessions={sessions}, Entities={entities}, Private={private}")
    print(f"    → Primary:{rec['primary']}")
    for s in rec['all_strategies']:
        if 'strategy' in s:
            print(f"        +{s['strategy']}: {s['reason']}")
    print()

```

17.8 Module 17 Exercises

1. Build a `HybridMemory` class that combines `WindowMemory` (for recency) and `VectorMemory` (for long-term relevance). When building the context for an LLM call, merge the recent window with the top-3 semantically relevant past turns, deduplicating any overlap.
2. Implement automatic entity extraction in the conversation loop: after each assistant response, run `EntityMemory.extract_and_update` on the full exchange, then inject the updated entity context into the next system prompt.
3. Extend `PersistentSession` with a `search_history(query: str) -> list[dict]` method that embeds the query and finds the most relevant past messages using cosine similarity computed in Python (no additional vector DB needed).
4. Build a multi-user memory system: a `MemoryManager` class that routes to the correct `PersistentSession` by `user_id`, enforces a maximum history size per user (deleting oldest messages when exceeded), and supports `export_session(user_id) → JSON`.

Phase 4 Summary

What You Learned

Module	Core Skills
M15 - RAG Fundamentals	Fixed, recursive, and sentence-aware chunking; <code>RAGIndexer</code> with ChromaDB; <code>RAGRetriever</code> with metadata filtering; <code>RAGPromptBuilder</code> with context budgeting and source labels; LLM-as-judge reranking; end-to-end <code>RAGPipeline</code>
M16 - Advanced RAG	HyDE (hypothetical document embeddings); self-query with LLM filter extraction; reciprocal rank fusion across multiple retrieval lists; hybrid BM25 + dense search with <code>HybridSearchEngine</code> ; multi-hop iterative retrieval; RAG evaluation (faithfulness, answer relevance, context recall)
M17 - Memory & State	<code>WindowMemory</code> with turn-pair trimming; <code>SummaryMemory</code> with LLM compression; <code>EntityMemory</code> with structured fact extraction; <code>PersistentSession</code> with SQLite for cross-session storage; <code>VectorMemory</code> for semantically searchable long-term history; memory strategy selection framework

Patterns to Carry Forward

- Always evaluate retrieval (Precision@k, Recall@k) before evaluating generation - most RAG failures happen in retrieval, not generation
- Use HyDE when queries are short and documents are long; the query/document embedding space mismatch is where HyDE helps most
- RRF is score-agnostic - you can merge results from completely different retrieval systems without normalisation
- Chunk overlap prevents information loss at boundaries; 10–15% of chunk size is the standard
- SQLite is sufficient for single-server deployments; move to PostgreSQL when you need multi-process writes or horizontal scaling
- `SummaryMemory` reduces input token costs proportionally to how much history is compressed - profile the cost savings before adding the complexity

- LLM-as-judge reranking is expensive - apply it only on the top-20 candidates, not the full corpus
-

What Is Next - Phase 5

Phase 5 covers **Deployment and Production** - taking everything built in Phases 1-4 and making it production-ready:

- **M18** - Streamlit for AI app UIs: chat interfaces, file upload, session state, streaming
- **M19** - FastAPI for GenAI backends: REST endpoints, streaming SSE, async, auth middleware
- **M20** - Evaluation and Observability: RAGAS, LangSmith tracing, structured logging, hallucination detection

Install before Phase 5: `pip install streamlit fastapi uvicorn python-multipart`

Phase 5 - Deployment & Production

Libraries: `streamlit 1.56` · `fastapi 0.136` · `uvicorn 0.46` · `anthropic 0.97` · `openai 2.32`
Install: `pip install streamlit fastapi uvicorn python-multipart anthropic openai python-dotenv`

Module 18 - Building AI Apps with Streamlit

Chat UIs · File upload · Session state · Streaming responses · Deployment

18.1 Why Streamlit for AI Apps

Streamlit turns Python scripts into interactive web applications with no HTML, CSS, or JavaScript. For AI applications it offers:

- **Chat primitives** - `st.chat_message` and `st.chat_input` render a proper chat UI in minutes
- **Session state** - persists conversation history across reruns within a user session
- **Streaming** - `st.write_stream` displays LLM tokens as they arrive
- **File upload** - built-in widget for PDFs, images, CSVs
- **Caching** - `@st.cache_resource` caches expensive objects (model clients, vector stores) across sessions

Every Streamlit script re-runs top to bottom on every user interaction. Session state is the mechanism for preserving data between reruns.

Run any Streamlit app: `streamlit run app.py`

Open `http://localhost:8501` in your browser.

18.2 Minimal Chat App

Save as `chat_app.py` and run with `streamlit run chat_app.py`.

```
# chat_app.py
import streamlit as st
import anthropic
import os
from dotenv import load_dotenv

load_dotenv()

st.set_page_config(page_title="AI Chat", page_icon="🤖", layout="centered")
st.title("AI Chat")

# — Client (cached - created once per server process) —
@st.cache_resource
def get_client() -> anthropic.Anthropic:
    return anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

client = get_client()

# — Session state initialisation —
if "messages" not in st.session_state:
    st.session_state.messages = [] # list of {"role": str, "content": str}

# — Sidebar settings —
with st.sidebar:
    st.header("Settings")
    model = st.selectbox(
```

```

        "Model",
        ["claude-sonnet-4-5", "claude-haiku-4-5", "claude-opus-4-5"],
    )
    temperature = st.slider("Temperature", 0.0, 1.0, 0.7, step=0.05)
    max_tokens = st.number_input("Max tokens", min_value=64, max_value=4096, value=1024, step=64)
    system_prompt = st.text_area(
        "System prompt",
        value="You are a helpful and concise AI assistant.",
        height=100,
    )
    if st.button("Clear conversation"):
        st.session_state.messages = []
        st.rerun()

# — Render existing messages —————
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])

# — Handle new user input —————
if user_input := st.chat_input("Type a message..."):
    # Add and display user message
    st.session_state.messages.append({"role": "user", "content": user_input})
    with st.chat_message("user"):
        st.markdown(user_input)

    # Stream assistant response
    with st.chat_message("assistant"):
        with client.messages.stream(
            model=model,

```

```

        max_tokens=max_tokens,
        system=system_prompt,
        messages=st.session_state.messages,
    ) as stream:
        response_text = st.write_stream(stream.text_stream)

    st.session_state.messages.append({"role": "assistant", "content": response_text})

```

18.3 RAG Chat App with File Upload

```

# rag_chat_app.py
import streamlit as st
import anthropic
import chromadb
import numpy as np
import tempfile, os, re, json
from pathlib import Path
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()

st.set_page_config(page_title="RAG Chat", page_icon="📖", layout="wide")
st.title("📖 Chat with Your Documents")

# — Clients —


---


@st.cache_resource
def get_clients():
    return (
        anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"]),

```



```

        OpenAI(api_key=os.environ["OPENAI_API_KEY"]),
    )

@st.cache_resource
def get_vector_store():
    client = chromadb.Client()
    return client.get_or_create_collection(
        "uploaded_docs", metadata={"hnsw:space": "cosine"}
    )

ant_client, oai_client = get_clients()
collection = get_vector_store()

# — Session state —————
if "messages" not in st.session_state: st.session_state.messages = []
if "indexed_files" not in st.session_state: st.session_state.indexed_files = []
if "doc_counter" not in st.session_state: st.session_state.doc_counter = 0

# — Helpers —————
def chunk_text(text: str, size: int = 500, overlap: int = 50)
-> list[str]:
    chunks, start = [], 0
    while start < len(text):
        end = min(start + size, len(text))
        chunks.append(text[start:end])
        start += size - overlap
    return chunks

def embed(texts: list[str]) -> list[list[float]]:
    resp = oai_client.embeddings.create(model="text-embedding-3-small", input=texts)

```

```

    return [e.embedding for e in sorted(resp.data, key=lambda
x: x.index)]

def index_document(text: str, filename: str) -> int:
    chunks = chunk_text(text)
    if not chunks:
        return 0
    embeddings = embed(chunks)
    doc_id = f"doc_{st.session_state.doc_counter}"
    st.session_state.doc_counter += 1

    collection.add(
        ids = [f"{doc_id}_chunk_{i}" for i in range(len(chunks))],
        documents = chunks,
        embeddings = embeddings,
        metadatas = [{"filename": filename, "chunk": i} for
i in range(len(chunks))],
    )
    return len(chunks)

def retrieve(query: str, k: int = 4) -> list[str]:
    if collection.count() == 0:
        return []
    q_emb = embed([query])[0]
    results = collection.query(
        query_embeddings=[q_emb],
        n_results=min(k, collection.count()),
        include=["documents", "metadatas"],
    )
    return results["documents"][0]

# — Sidebar: file upload —————
with st.sidebar:
    st.header("Upload Documents")

```

```

uploaded = st.file_uploader(
    "Upload .txt or .md files",
    type=["txt", "md"],
    accept_multiple_files=True,
)

if uploaded:
    for file in uploaded:
        if file.name not in st.session_state.indexed_files:
s:
            with st.spinner(f"Indexing{file.name}..."):
                text = file.read().decode("utf-8", errors="ignore")

                n = index_document(text, file.name)
                st.session_state.indexed_files.append(file.name)

                st.success(f"{file.name}:{n} chunks indexed")

if st.session_state.indexed_files:
    st.subheader("Indexed files")
    for f in st.session_state.indexed_files:
        st.caption(f"✓{f}")

st.divider()
k = st.slider("Chunks to retrieve", 1, 8, 4)
show_src = st.checkbox("Show retrieved context", value=False)

if st.button("Clear all"):
    st.session_state.messages = []
    st.session_state.indexed_files = []
    st.rerun()

# — Main chat —

```

```

col_chat, col_context = st.columns([3, 1]) if show_src else
(st, None)

with col_chat:
    for msg in st.session_state.messages:
        with st.chat_message(msg["role"]):
            st.markdown(msg["content"])

    if user_input := st.chat_input("Ask about your document
s..."):
        st.session_state.messages.append({"role": "user", "co
ntent": user_input})
        with st.chat_message("user"):
            st.markdown(user_input)

        # Retrieve context
        chunks = retrieve(user_input, k=k)
        context = "\n\n---\n\n".join(chunks) if chunks else
"No documents indexed yet."

        if show_src and col_context:
            with col_context:
                st.subheader("Retrieved context")
                for i, chunk in enumerate(chunks, 1):
                    with st.expander(f"Chunk{i}"):
                        st.caption(chunk[:300])

            system = f"""Answer the user's question using ONLY th
e context below.
If the context does not contain the answer, say so clearly.

Context:
{context}"""

        with st.chat_message("assistant"):
            with ant_client.messages.stream(

```

```

        model="claude-sonnet-4-5",
        max_tokens=1024,
        system=system,
        messages=st.session_state.messages,
    ) as stream:
        reply = st.write_stream(stream.text_stream)

    st.session_state.messages.append({"role": "assistant", "content": reply})

```

18.4 Multi-Page App Structure

For larger AI applications, use Streamlit's multi-page structure:

```

my_ai_app/
  app.py                ← entry point (Home page)
  pages/
    1_Chat.py
    2_RAG.py
    3_Evaluation.py
  components/
    __init__.py
    sidebar.py          ← shared sidebar component
    chat.py             ← reusable chat renderer
  utils/
    __init__.py
    clients.py          ← cached API clients
    rag.py              ← retrieval helpers
  .streamlit/
    config.toml         ← theme and server settings
    secrets.toml        ← API keys (never commit this)

```

```

# .streamlit/config.toml
[theme]
primaryColor = "#4F46E5"
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#F8F7FF"
textColor = "#1F2937"
font = "sans serif"

```

```
[server]
maxUploadSize = 50      # MB
headless = true         # required for cloud deployment
```

```
# .streamlit/secrets.toml ← add to .gitignore
ANTHROPIC_API_KEY = "sk-ant-..."
OPENAI_API_KEY = "sk-..."
```

```
# utils/clients.py
import streamlit as st
import anthropic
from openai import OpenAI

@st.cache_resource
def get_anthropic_client() -> anthropic.Anthropic:
    return anthropic.Anthropic(api_key=st.secrets["ANTHROPIC_API_KEY"])

@st.cache_resource
def get_openai_client() -> OpenAI:
    return OpenAI(api_key=st.secrets["OPENAI_API_KEY"])
```

```
# components/chat.py
import streamlit as st

def render_messages(messages: list[dict]) -> None:
    """Render a message list in the chat UI."""
    for msg in messages:
        with st.chat_message(msg["role"]):
            st.markdown(msg["content"])

def init_session(key: str = "messages") -> None:
    """Initialise session state for a chat page."""
```

```
if key not in st.session_state:
    st.session_state[key] = []
```

18.5 Performance Patterns

```
# Correct use of @st.cache_resource and @st.cache_data
import streamlit as st
import chromadb
from openai import OpenAI

# @st.cache_resource: for objects that are NOT serialisable
# (API clients, DB connections, loaded models)
# Created once per server process, shared across all sessions
@st.cache_resource
def get_vector_store(persist_dir: str = "/tmp/chroma"):
    client = chromadb.PersistentClient(path=persist_dir)
    return client.get_or_create_collection("docs", metadata={
        "hnsw:space": "cosine"})

# @st.cache_data: for serialisable data (DataFrames, dicts, lists)
# Cached per unique set of arguments, per session
@st.cache_data(ttl=3600) # expire after 1 hour
def load_benchmark_results(filepath: str) -> list[dict]:
    import json
    with open(filepath) as f:
        return json.load(f)

# Progress indicators for long operations
def index_with_progress(texts: list[str], filenames: list[str]) -> None:
    progress = st.progress(0, text="Indexing documents...")
    status = st.empty()

    for i, (text, name) in enumerate(zip(texts, filenames)):
```

```

        status.caption(f"Processing{name}...")
        # ... indexing logic ...
        progress.progress((i + 1) / len(texts), text=f"Indexed{i + 1}/{len(texts)}")

    progress.empty()
    status.empty()
    st.success(f"Indexed{len(texts)} documents.")

```

18.6 Deploying to Streamlit Cloud

```

# 1. Push your app to a public GitHub repo

# 2. Create requirements.txt
pip freeze > requirements.txt
# OR manually specify pinned versions:
# streamlit==1.56.0
# anthropic==0.97.0
# openai==2.32.0
# chromadb==1.1.1
# python-dotenv==1.0.0

# 3. Add .streamlit/secrets.toml to .gitignore
echo ".streamlit/secrets.toml" >> .gitignore

# 4. Go to share.streamlit.io
#   → New app → select your repo/branch/file
#   → Advanced settings → add secrets (same format as secrets.toml)
#   → Deploy

# Runtime tips:
# - Streamlit Cloud gives you 1 GB RAM free tier
# - Use @st.cache_resource for ChromaDB - it persists between reruns

```



```
# - For persistent vector stores on Cloud, use Pinecone or Qdrant instead of local ChromaDB
# - Set PYTHONPATH if using a src/ layout: add to .streamlit/config.toml:
#   [runner]
#   pythonPath = "src"
```

18.7 Module 18 Exercises

1. Build a **Model Comparison** Streamlit app: the user types a prompt, the app calls two models simultaneously (using `asyncio.gather` in a thread), and displays responses side by side with latency and token counts.
2. Add **conversation export** to the basic chat app: a sidebar button that serialises `st.session_state.messages` to JSON and offers it as a download via `st.download_button`.
3. Build an **Image Q&A** app: user uploads a PNG/JPG, the app sends it to Claude with a text question using base64 encoding, and streams the response.
4. Add **usage analytics** to the RAG chat app: track total queries, total tokens used, and average retrieval latency per session. Display a live metrics dashboard in the sidebar using `st.metric`.

Module 19 - FastAPI for GenAI Backends

REST endpoints · Streaming SSE · Background tasks · Auth · Async

19.1 Why FastAPI for GenAI

FastAPI is the standard for building production Python APIs. For GenAI backends it provides:

- **Async-first** - LLM API calls are I/O-bound; async lets you serve many concurrent requests without threads
- **Pydantic validation** - request/response schemas are validated automatically
- **Server-Sent Events** - first-class streaming support for LLM token streams

- **Auto-generated docs** - OpenAPI spec at `/docs`, importable by frontend teams

Typical GenAI backend architecture:

```
Client (browser / mobile)
  ↓ HTTP POST /chat
FastAPI app (uvicorn)
  ↓ SDK call
Anthropic / OpenAI API
  ↓ stream
FastAPI SSE response
  ↓ token-by-token
Client
```

19.2 Application Structure

```
ai_backend/
  src/
    main.py          ← FastAPI app, middleware, lifespan
    routers/
      chat.py        ← /chat endpoints
      rag.py          ← /rag endpoints
      health.py       ← /health, /metrics
    models/
      requests.py     ← Pydantic request models
      responses.py    ← Pydantic response models
    services/
      llm.py          ← LLM client wrappers
      retrieval.py     ← RAG retrieval service
      memory.py       ← Session memory service
    middleware/
      auth.py         ← API key verification
      logging.py      ← Request logging
      rate_limit.py
    config.py         ← Settings from environment
  tests/
    test_chat.py
    test_rag.py
  pyproject.toml
```

19.3 Core Application

```

# src/main.py
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from contextlib import asynccontextmanager
import time, logging, os
from dotenv import load_dotenv

load_dotenv()
log = logging.getLogger("ai_backend")

# — Lifespan: startup and shutdown —
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup: initialise shared resources
    from anthropic import AsyncAnthropic
    from openai import AsyncOpenAI

    app.state.anthropic = AsyncAnthropic(api_key=os.environ[
        "ANTHROPIC_API_KEY"])
    app.state.openai = AsyncOpenAI(api_key=os.environ["OPE
        NAI_API_KEY"])
    app.state.start_time = time.time()

    log.info("Application started")
    yield

    # Shutdown: close connections, flush buffers
    await app.state.anthropic.close()
    await app.state.openai.close()
    log.info("Application shut down cleanly")

# — App —

```

```

app = FastAPI(
    title="GenAI Backend API",
    version="1.0.0",
    description="Production API for LLM-powered features",
    lifespan=lifespan,
    docs_url="/docs",
    redoc_url="/redoc",
)

# — CORS —————
app.add_middleware(
    CORSMiddleware,
    allow_origins=os.environ.get("ALLOWED_ORIGINS", "*").split(", "),
    allow_credentials=True,
    allow_methods=["GET", "POST", "DELETE"],
    allow_headers=["Authorization", "Content-Type"],
)

# — Request timing middleware —————
@app.middleware("http")
async def add_timing_header(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    elapsed = round((time.perf_counter() - start) * 1000, 2)
    response.headers["X-Process-Time-Ms"] = str(elapsed)
    return response

# — Global exception handler —————
@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    log.error("Unhandled exception", exc_info=exc, extra={"pa

```

```

th": request.url.path}))
    return JSONResponse(
        status_code=500,
        content={"error": "Internal server error", "detail":
str(exc)},
    )

# — Include routers —————
# from src.routers import chat, rag, health
# app.include_router(chat.router, prefix="/v1", tags=["cha
t"])
# app.include_router(rag.router, prefix="/v1", tags=["ra
g"])
# app.include_router(health.router, tags=["ops"])

```

19.4 Request/Response Models

```

# src/models/requests.py
from pydantic import BaseModel, Field, field_validator
from typing import Optional, Literal

class Message(BaseModel):
    role: Literal["user", "assistant"]
    content: str = Field(..., min_length=1, max_length=100_00
0)

class ChatRequest(BaseModel):
    messages: list[Message] = Field(..., min_length=1, m
ax_length=100)
    model: str = Field(default="claude-sonn
et-4-5")
    temperature: float = Field(default=0.7, ge=0.0,
le=2.0)
    max_tokens: int = Field(default=1024, ge=1,

```

```

le=8192)
    system_prompt: Optional[str] = Field(default=None, max_length=10_000)
    stream: bool = False
    session_id: Optional[str] = Field(default=None, max_length=128)

    @field_validator("model")
    @classmethod
    def validate_model(cls, v: str) -> str:
        allowed = {
            "claude-sonnet-4-5", "claude-haiku-4-5", "claude-opus-4-5",
            "gpt-4o", "gpt-4o-mini",
        }
        if v not in allowed:
            raise ValueError(f"Model must be one of{allowed}")
        return v

class RAGRequest(BaseModel):
    query: str = Field(..., min_length=1, max_length=5000)
    collection_name: str = Field(default="default")
    k: int = Field(default=5, ge=1, le=20)
    use_reranking: bool = False
    metadata_filter: Optional[dict] = None
    model: str = Field(default="claude-sonnet-4-5")

class EmbedRequest(BaseModel):
    texts: list[str] = Field(..., min_length=1, max_length=100)
    model: str = Field(default="text-embedding-3-small")

```

```
# src/models/responses.py
class ChatResponse(BaseModel):
    response: str
    session_id: Optional[str]
    model: str
    input_tokens: int
    output_tokens: int
    latency_ms: float

class RAGResponse(BaseModel):
    answer: str
    sources: list[dict]
    chunks_used: int
    latency_ms: float

class HealthResponse(BaseModel):
    status: str
    uptime_s: float
    version: str
```

19.5 Chat Endpoint with Streaming

```
# src/routers/chat.py
from fastapi import APIRouter, Request, HTTPException, Depends
from fastapi.responses import StreamingResponse
from pydantic import BaseModel, Field
from typing import Optional, AsyncGenerator, Literal
import json, time, asyncio

router = APIRouter()

# — Pydantic models (inline for clarity) —
```

```

class Message(BaseModel):
    role: Literal["user", "assistant"]
    content: str

class ChatRequest(BaseModel):
    messages: list[Message]
    model: str = "claude-sonnet-4-5"
    temperature: float = 0.7
    max_tokens: int = 1024
    system_prompt: Optional[str] = None
    stream: bool = False

class ChatResponse(BaseModel):
    response: str
    model: str
    input_tokens: int
    output_tokens: int
    latency_ms: float

# — Auth dependency —————
from fastapi.security import APIKeyHeader
from fastapi import Security

api_key_header = APIKeyHeader(name="X-API-Key", auto_error=True)

async def verify_api_key(api_key: str = Security(api_key_header)) -> str:
    import os
    valid_keys = set(os.environ.get("API_KEYS", "dev-key-123").split(","))
    if api_key not in valid_keys:
        raise HTTPException(status_code=401, detail="Invalid API key")
    return api_key

```

```

# — Non-streaming endpoint —————

@router.post("/chat", response_model=ChatResponse)
async def chat(
    request: ChatRequest,
    req: Request,
    api_key: str = Depends(verify_api_key),
) -> ChatResponse:
    start = time.perf_counter()
    client = req.app.state.anthropic

    api_messages = [{"role": m.role, "content": m.content} for m in request.messages]

    kwargs: dict = {
        "model": request.model,
        "max_tokens": request.max_tokens,
        "messages": api_messages,
    }
    if request.system_prompt:
        kwargs["system"] = request.system_prompt

    response = await client.messages.create(**kwargs)

    return ChatResponse(
        response = response.content[0].text,
        model = request.model,
        input_tokens = response.usage.input_tokens,
        output_tokens = response.usage.output_tokens,
        latency_ms = round((time.perf_counter() - start) *
1000, 2),
    )

# — Streaming endpoint (Server-Sent Events) —————

```

```

async def stream_chat_response(
    client,
    messages: list[dict],
    model: str,
    max_tokens: int,
    system_prompt: Optional[str],
) -> AsyncGenerator[str, None]:
    """
    Yield SSE-formatted events.
    Format: "data: {json}\n\n"
    Client reads: EventSource or fetch with ReadableStream
    """
    try:
        async with client.messages.stream(
            model=model,
            max_tokens=max_tokens,
            messages=messages,
            **({"system": system_prompt} if system_prompt else {}),
        ) as stream:
            async for text in stream.text_stream:
                event = json.dumps({"type": "token", "content": text})
                yield f"data:{event}\n\n"

            # Send final usage stats
            final = await stream.get_final_message()
            done_event = json.dumps({
                "type": "done",
                "input_tokens": final.usage.input_tokens,
                "output_tokens": final.usage.output_tokens,
            })
            yield f"data:{done_event}\n\n"

    except Exception as e:
        error_event = json.dumps({"type": "error", "detail":

```

```

str(e))
        yield f"data:{error_event}\n\n"

@router.post("/chat/stream")
async def chat_stream(
    request: ChatRequest,
    req: Request,
    api_key: str = Depends(verify_api_key),
) -> StreamingResponse:
    client = req.app.state.anthropic
    api_messages = [{"role": m.role, "content": m.content} for m in request.messages]

    return StreamingResponse(
        stream_chat_response(
            client = client,
            messages = api_messages,
            model = request.model,
            max_tokens = request.max_tokens,
            system_prompt = request.system_prompt,
        ),
        media_type = "text/event-stream",
        headers = {
            "Cache-Control": "no-cache",
            "X-Accel-Buffering": "no", # disable nginx buffering
            "Transfer-Encoding": "chunked",
        },
    )

```

19.6 RAG Endpoint

```

# src/routers/rag.py
from fastapi import APIRouter, Request, HTTPException, Depends

```

```

from pydantic import BaseModel, Field
from typing import Optional
import time

router = APIRouter()

class RAGRequest(BaseModel):
    query: str = Field(..., min_length=1)
    collection_name: str = Field(default="default")
    k: int = Field(default=5, ge=1, le=20)
    model: str = Field(default="claude-sonnet-4-5")
    metadata_filter: Optional[dict] = None

class RAGResponse(BaseModel):
    answer: str
    sources: list[dict]
    chunks_used: int
    latency_ms: float

@router.post("/rag/query", response_model=RAGResponse)
async def rag_query(request: RAGRequest, req: Request) -> RAGResponse:
    start = time.perf_counter()
    client = req.app.state.anthropic
    oai = req.app.state.openai

    # 1. Embed query
    emb_resp = await oai.embeddings.create(
        model="text-embedding-3-small",
        input=[request.query],
    )
    q_vector = emb_resp.data[0].embedding

    # 2. Retrieve from ChromaDB (stored in app state)

```

```

        collection = getattr(req.app.state, "chroma_collections",
        {}).get(
            request.collection_name
        )
        if collection is None:
            raise HTTPException(
                status_code=404,
                detail=f"Collection '{request.collection_name}' n
ot found.",
            )

        query_kwargs: dict = {
            "query_embeddings": [q_vector],
            "n_results": min(request.k, collection.count
        ()),
            "include": ["documents", "distances", "metad
atas"],
        }
        if request.metadata_filter:
            query_kwargs["where"] = request.metadata_filter

        results = collection.query(**query_kwargs)
        chunks = results["documents"][0]
        metas = results["metadatas"][0]
        dists = results["distances"][0]

        if not chunks:
            return RAGResponse(
                answer="No relevant documents found.",
                sources=[], chunks_used=0,
                latency_ms=round((time.perf_counter() - start) *
1000, 2),
            )

        # 3. Build context
        context = "\n\n---\n\n".join(

```

```

        f"[Source:{m.get('source', 'unknown')}]\\n{chunk}"
        for chunk, m in zip(chunks, metas)
    )

# 4. Generate
response = await client.messages.create(
    model      = request.model,
    max_tokens = 1024,
    system     = """Answer the question using ONLY the provided context.
If the context is insufficient, say so clearly. Do not hallucinate."""
    messages  = [{"role": "user", "content": f"Context:\\n{context}\\n\\nQuestion:{request.query}"}],
)

sources = [
    {"text": chunk[:100], "score": round(1 - dist / 2, 4), **meta}
    for chunk, dist, meta in zip(chunks, dists, metas)
]

return RAGResponse(
    answer      = response.content[0].text,
    sources     = sources,
    chunks_used = len(chunks),
    latency_ms  = round((time.perf_counter() - start) * 1000, 2),
)

@router.post("/rag/index")
async def index_documents(req: Request, body: dict) -> dict:
    """
    Index documents into a named collection.
    Body: {"collection_name": str, "documents": [{"text": str, "metadata": dict}]}
    """

```

```

"""
collection_name = body.get("collection_name", "default")
documents       = body.get("documents", [])

if not documents:
    raise HTTPException(status_code=400, detail="No documents provided.")

# Lazy-init ChromaDB collections in app state
if not hasattr(req.app.state, "chroma_collections"):
    import chromadb
    req.app.state.chroma_client = chromadb.Client()
    req.app.state.chroma_collections = {}

if collection_name not in req.app.state.chroma_collections:
    req.app.state.chroma_collections[collection_name] = (
        req.app.state.chroma_client.get_or_create_collection(
            collection_name, metadata={"hnsw:space": "cosine"}
        )
    )

col = req.app.state.chroma_collections[collection_name]
oai = req.app.state.openai
texts = [d["text"] for d in documents]
metas = [d.get("metadata", {}) for d in documents]

# Embed in batch
emb_resp = await oai.embeddings.create(model="text-embedding-3-small", input=texts)
embeddings = [e.embedding for e in sorted(emb_resp.data, key=lambda x: x.index)]

col.add(

```

```

        ids = [f"{collection_name}_{i}_{hash(t) % 10**8}" for i, t in enumerate(texts)],
        documents = texts,
        embeddings = embeddings,
        metadatas = metas,
    )

    return {"indexed": len(documents), "collection": collection_name, "total": col.count()}

```

19.7 Background Tasks and Rate Limiting

```

# Logging interactions in the background (non-blocking)
from fastapi import BackgroundTasks
import sqlite3, time, json

def log_to_db(session_id: str, query: str, response: str, tokens: int, latency_ms: float):
    """Write interaction log to SQLite. Runs in a thread pool, non-blocking."""
    conn = sqlite3.connect("/tmp/interactions.db")
    conn.execute("CREATE TABLE IF NOT EXISTS logs (session_id TEXT, query TEXT, response TEXT, tokens INT, latency_ms REAL, ts REAL)")
    conn.execute(
        "INSERT INTO logs VALUES (?, ?, ?, ?, ?, ?)",
        (session_id, query, response[:500], tokens, latency_ms, time.time())
    )
    conn.commit()
    conn.close()

# Usage in any endpoint:
# background_tasks.add_task(log_to_db, session_id, query, answer, tokens, latency)

```



```

# — Simple token-bucket rate limiter —————

from collections import defaultdict
from fastapi import HTTPException, Request
import asyncio, time

class RateLimiter:
    """
    Token-bucket rate limiter keyed by API key or IP.
    Thread-safe with asyncio.Lock.
    """

    def __init__(self, requests_per_minute: int = 60):
        self.rpm = requests_per_minute
        self._buckets: dict[str, list[float]] = defaultdict(
            list
        )
        self._lock = asyncio.Lock()

    async def check(self, key: str) -> None:
        async with self._lock:
            now = time.time()
            window = now - 60.0
            self._buckets[key] = [t for t in self._buckets[key] if t > window]

            if len(self._buckets[key]) >= self.rpm:
                oldest = self._buckets[key][0]
                retry_in = round(60.0 - (now - oldest), 1)
                raise HTTPException(
                    status_code=429,
                    detail=f"Rate limit exceeded. Retry after {retry_in}s.",
                    headers={"Retry-After": str(retry_in)},
                )

```

```

        self._buckets[key].append(now)

# Attach to app and use as dependency
rate_limiter = RateLimiter(requests_per_minute=60)

async def rate_limit_dep(req: Request, api_key: str = Depends(
    verify_api_key)):
    await rate_limiter.check(api_key)
    return api_key

```

19.8 Health and Metrics Endpoints

```

# src/routers/health.py
from fastapi import APIRouter, Request
from pydantic import BaseModel
import time, os

router = APIRouter()

class HealthResponse(BaseModel):
    status: str
    uptime_s: float
    version: str
    checks: dict

@router.get("/health", response_model=HealthResponse)
async def health(req: Request) -> HealthResponse:
    uptime = round(time.time() - req.app.state.start_time, 1)

    # Run dependency checks
    checks: dict = {}

    # Check Anthropic reachability
    try:

```

```

        await req.app.state.anthropic.models.list()
        checks["anthropic"] = "ok"
    except Exception as e:
        checks["anthropic"] = f"error:{e}"

    return HealthResponse(
        status = "ok" if all(v == "ok" for v in checks.values()) else "degraded",
        uptime_s = uptime,
        version = os.environ.get("APP_VERSION", "1.0.0"),
        checks = checks,
    )

@router.get("/metrics")
async def metrics(req: Request) -> dict:
    """
    Basic Prometheus-compatible metrics endpoint.
    For production use the prometheus-fastapi-instrumentator
    library.
    """
    return {
        "uptime_seconds": round(time.time() - req.app.state.start_time, 1),
        "requests_total": getattr(req.app.state, "requests_total", 0),
        "requests_errored": getattr(req.app.state, "requests_errored", 0),
        "tokens_total": getattr(req.app.state, "tokens_total", 0),
    }

```

19.9 Running and Testing

```

# Development - hot reload
uvicorn src.main:app --reload --port 8000

```

```
# Production - multiple workers
uvicorn src.main:app --host 0.0.0.0 --port 8000 --workers 4

# With gunicorn (more robust process management)
pip install gunicorn
gunicorn src.main:app -w 4 -k uvicorn.workers.UvicornWorker -
-bind 0.0.0.0:8000
```

```
# tests/test_chat.py
from fastapi.testclient import TestClient
from unittest.mock import AsyncMock, patch
from src.main import app

client = TestClient(app)

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] in ("ok", "degraded")

def test_chat_missing_auth():
    response = client.post("/v1/chat", json={"messages": [{"role": "user", "content": "hi"}]})
    assert response.status_code == 403 # missing X-API-Key header

def test_chat_invalid_model():
    response = client.post(
        "/v1/chat",
        headers={"X-API-Key": "dev-key-123"},
        json={"messages": [{"role": "user", "content": "hi"}], "model": "gpt-9000"},
    )
    assert response.status_code == 422 # pydantic validation
```

```
n error
```

```
# Run: pytest tests/ -v
```

```
# Test streaming with curl
```

```
curl -X POST http://localhost:8000/v1/chat/stream \
  -H "Content-Type: application/json" \
  -H "X-API-Key: dev-key-123" \
  -d '{"messages": [{"role": "user", "content": "Count to 5 s
lowly."}], "stream": true}' \
  --no-buffer
```

19.10 Module 19 Exercises

1. Add a `/v1/embed` endpoint that accepts a list of texts and returns their embeddings as a list of float arrays. Validate that the list has at most 100 items.
2. Implement a `ConversationStore` backed by SQLite that the RAG and chat endpoints use to persist session history. Expose `GET /v1/sessions/{session_id}` to retrieve a session's full message history.
3. Add Prometheus metrics using `prometheus-fastapi-instrumentator`: track `http_requests_total`, `http_request_duration_seconds`, and a custom `llm_tokens_total` counter. Expose at `/metrics`.
4. Write integration tests for the streaming endpoint using `httpx.AsyncClient` and verify that: (a) each chunk is valid SSE format, (b) a `done` event arrives at the end, and (c) the total token count in the `done` event is positive.

Module 20 - Evaluation & Observability

RAGAS · LangSmith tracing · Structured logging · Hallucination detection · Dashboards

20.1 Why Observability Matters

A production GenAI system without observability is a black box. You cannot improve what you cannot measure. The three pillars are:

- **Logs** - structured records of every event (requests, errors, LLM calls, retrieval results)
- **Traces** - end-to-end timing across all steps of a single request (retrieval → rerank → generate)
- **Metrics** - aggregated measurements over time (P95 latency, token cost per day, RAG precision)

Evaluation sits on top of these: it uses the logged inputs/outputs to compute quality scores that drive iteration.

20.2 Structured Logging

```
# src/middleware/logging.py
import logging, json, time, sys, uuid
from fastapi import Request, Response
from typing import Callable, Any

class JSONFormatter(logging.Formatter):
    """Emit log records as single-line JSON for log aggregators (Datadog, CloudWatch, etc.)."""

    def format(self, record: logging.LogRecord) -> str:
        base: dict[str, Any] = {
            "ts": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime(record.created)),
            "level": record.levelname,
            "logger": record.name,
            "msg": record.getMessage(),
        }
        if record.exc_info:
            base["exception"] = self.formatException(record.exc_info)
```

```

        # Merge any extra fields passed via extra={}
        skip = {
            "msg", "args", "levelname", "levelno", "pathname",
            "filename",
            "module", "exc_info", "exc_text", "stack_info",
            "lineno",
            "funcName", "created", "msecs", "relativeCreated",
            "thread",
            "threadName", "processName", "process", "name",
            "message", "taskName",
        }
        for k, v in record.__dict__.items():
            if k not in skip:
                base[k] = v

        return json.dumps(base)

def configure_logging(level: str = "INFO") -> None:
    handler = logging.StreamHandler(sys.stdout)
    handler.setFormatter(JSONFormatter())
    root = logging.getLogger()
    root.handlers = []
    root.addHandler(handler)
    root.setLevel(getattr(logging, level.upper(), logging.INFO))

log = logging.getLogger("ai_backend")

# Attach as middleware
async def log_requests(request: Request, call_next: Callable)
-> Response:
    request_id = str(uuid.uuid4())[:8]
    start = time.perf_counter()

    log.info(
        "Request started",

```

```

        extra={
            "request_id": request_id,
            "method":     request.method,
            "path":       request.url.path,
        },
    )

    response = await call_next(request)
    elapsed = round((time.perf_counter() - start) * 1000, 2)

    log.info(
        "Request completed",
        extra={
            "request_id": request_id,
            "status_code": response.status_code,
            "latency_ms": elapsed,
        },
    )
    return response

```

20.3 Request Tracing

```

import time, uuid, json, logging
from dataclasses import dataclass, field
from contextlib import contextmanager
from typing import Optional, Any

log = logging.getLogger("tracer")

@dataclass
class Span:
    name: str
    trace_id: str
    span_id: str = field(default_factory=lambda: str(uuid.uuid4())[:8])

```



```

    parent_id: Optional[str] = None
    start_time: float = field(default_factory=time.perf_counter)
    end_time: Optional[float] = None
    attributes: dict = field(default_factory=dict)
    events: list[dict] = field(default_factory=list)
    status: str = "ok" # "ok" | "error"

    @property
    def duration_ms(self) -> float:
        if self.end_time is None:
            return 0.0
        return round((self.end_time - self.start_time) * 1000, 2)

    def set(self, key: str, value: Any) -> None:
        self.attributes[key] = value

    def event(self, name: str, **attrs) -> None:
        self.events.append({"name": name, "ts": time.perf_counter(), **attrs})

    def to_dict(self) -> dict:
        return {
            "trace_id": self.trace_id,
            "span_id": self.span_id,
            "name": self.name,
            "duration_ms": self.duration_ms,
            "status": self.status,
            "attributes": self.attributes,
            "events": self.events,
        }

class Tracer:
    def __init__(self):
        self._spans: list[Span] = []

```

```

@contextmanager
def span(self, name: str, trace_id: Optional[str] = None,
parent_id: Optional[str] = None, **attrs):
    tid = trace_id or str(uuid.uuid4())[:8]
    span = Span(name=name, trace_id=tid, parent_id=parent
_id, attributes=dict(attrs))
    try:
        yield span
    except Exception as e:
        span.status = "error"
        span.set("error.message", str(e))
        raise
    finally:
        span.end_time = time.perf_counter()
        self._spans.append(span)
        log.info("Span completed", extra=span.to_dict())

def get_trace(self, trace_id: str) -> list[Span]:
    return [s for s in self._spans if s.trace_id == trace
_id]

tracer = Tracer() # module-level singleton

# Usage in a RAG request:
def traced_rag_query(question: str, retriever, generator) ->
dict:
    tid = str(uuid.uuid4())[:8]

    with tracer.span("rag_request", trace_id=tid, question=qu
estion[:80]) as root:

        with tracer.span("retrieve", trace_id=tid, parent_id=
root.span_id) as ret:
            chunks = retriever.retrieve(question, k=5)
            ret.set("chunks_returned", len(chunks))

```

```

        ret.set("top_score", chunks[0].score if chunks else 0)

        with tracer.span("generate", trace_id=tid, parent_id=root.span_id) as gen:
            answer = generator(question, chunks)
            gen.set("answer_chars", len(answer))

        root.set("total_chunks", len(chunks))
        return {"answer": answer, "trace_id": tid}

```

20.4 LLM-as-Judge Evaluation

LLM-as-judge uses a capable LLM to score responses on dimensions that are hard to measure with exact metrics.

```

import os, json
from openai import OpenAI
from dataclasses import dataclass
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

@dataclass
class EvalScore:
    score: float # 0.0 - 1.0
    reasoning: str
    passed: bool

JUDGE_MODEL = "gpt-4o" # use a strong judge, even if app uses a cheaper model

def judge(system: str, user: str) -> EvalScore:
    """Call the judge LLM and parse a structured score."""

```

```

resp = client.chat.completions.create(
    model = JUDGE_MODEL,
    max_tokens = 256,
    temperature = 0,
    response_format={"type": "json_object"},
    messages = [
        {"role": "system", "content": system},
        {"role": "user", "content": user},
    ],
)
data = json.loads(resp.choices[0].message.content)
score = float(data.get("score", 0.0))
return EvalScore(
    score=score,
    reasoning=data.get("reasoning", ""),
    passed=score >= 0.7,
)

def eval_faithfulness(answer: str, context: str) -> EvalScore:
    """Every claim in the answer must be supported by the context."""
    return judge(
        system="""Score whether the answer is fully grounded in the context.
1.0 = every claim is supported. 0.0 = answer contradicts or ignores context.
Return JSON: {"score": float, "reasoning": "one sentence"}""",
        user=f"Context:\n{context}\n\nAnswer:\n{answer}",
    )

def eval_answer_relevance(question: str, answer: str) -> EvalScore:
    """The answer should directly and completely address the question."""

```

```

    return judge(
        system="""Score how well the answer addresses the question.
1.0 = completely and directly answers. 0.0 = irrelevant or evasive.
Return JSON: {"score": float, "reasoning": "one sentence"}""",
        user=f"Question:{question}\n\nAnswer:{answer}",
    )

def eval_hallucination(answer: str, context: str) -> EvalScore:
    """Detect whether the answer introduces facts not present in the context."""
    return judge(
        system="""Check whether the answer introduces facts NOT present in the context.
1.0 = no hallucination detected. 0.0 = clear hallucination found.
Return JSON: {"score": float, "reasoning": "one sentence", "hallucinated_claims": ["..."]}""",
        user=f"Context:\n{context}\n\nAnswer:\n{answer}",
    )

def eval_conciseness(question: str, answer: str, ideal_words: int = 100) -> EvalScore:
    """Penalise unnecessary verbosity."""
    word_count = len(answer.split())
    ratio = min(ideal_words / max(word_count, 1), 1.0)
    # Blend length ratio with LLM score
    llm_score = judge(
        system=f"""Score the answer's conciseness for the question.
Ideal length is around{ideal_words} words.
1.0 = perfectly concise. 0.0 = extremely verbose or padded.
Return JSON:{{{"score": float, "reasoning": "one sentence

```

```

e"}}""",
        user=f"Question:{question}\n\nAnswer ({word_count} words):{answer}",
    )
    combined = 0.5 * llm_score.score + 0.5 * ratio
    return EvalScore(score=round(combined, 3), reasoning=llm_score.reasoning, passed=combined >= 0.7)

# — Run a full eval suite —————

@dataclass
class RAGEvalCase:
    question: str
    context: str
    answer: str
    ground_truth: str

def run_eval_suite(cases: list[RAGEvalCase]) -> dict:
    results = []
    for case in cases:
        result = {
            "question": case.question,
            "faithfulness": eval_faithfulness(case.answer, case.context).score,
            "answer_relevance": eval_answer_relevance(case.question, case.answer).score,
            "hallucination": eval_hallucination(case.answer, case.context).score,
        }
        results.append(result)

    metrics = ["faithfulness", "answer_relevance", "hallucination"]
    summary = {
        m: round(sum(r[m] for r in results) / len(results),
4)

```

```

        for m in metrics
    }
    summary["n"] = len(results)
    return {"summary": summary, "results": results}

```

20.5 Automated Regression Testing

```

import json, time
from pathlib import Path
from dataclasses import dataclass, asdict
from typing import Callable

@dataclass
class RegressionTestCase:
    id: str
    input: dict # request payload
    expected: dict # expected keys/values in response
    eval_fns: list[str] # names of eval functions to run

class RegressionSuite:
    """
    Run a fixed set of test cases against the current app.
    Compare scores to a baseline to detect regressions.
    """

    def __init__(self, baseline_path: str = "baselines/eval_baseline.json"):
        self.baseline_path = Path(baseline_path)
        self._baseline = self._load_baseline()

    def _load_baseline(self) -> dict:
        if self.baseline_path.exists():
            return json.loads(self.baseline_path.read_text())

```

```

        return {}

    def save_baseline(self, results: dict) -> None:
        self.baseline_path.parent.mkdir(parents=True, exist_o
k=True)
        self.baseline_path.write_text(json.dumps(results, ind
ent=2))
        print(f"Baseline saved to{self.baseline_path}")

    def compare_to_baseline(self, current: dict, threshold: f
loat = 0.05) -> list[dict]:
        """
        Flag any metric that dropped by more than `threshold`
        vs baseline.
        Returns list of regressions.
        """
        regressions = []
        for metric, current_score in current.get("summary",
{}).items():
            if metric == "n":
                continue
            baseline_score = self._baseline.get("summary",
{}).get(metric)
            if baseline_score is None:
                continue
            drop = baseline_score - current_score
            if drop > threshold:
                regressions.append({
                    "metric": metric,
                    "baseline": baseline_score,
                    "current": current_score,
                    "drop": round(drop, 4),
                })
        return regressions

def run_regression_check(

```



```

    pipeline_fn: Callable[[str], dict],
    test_cases: list[dict],
    suite: RegressionSuite,
) -> bool:
    """
    Returns True if no regressions detected.
    Each test_case: {"question": str, "context": str, "ground
_truth": str}
    """
    eval_cases = []
    for tc in test_cases:
        result = pipeline_fn(tc["question"])
        context = "\n\n".join(s["text"] for s in result.get
("sources", []))
        eval_cases.append(RAGEvalCase(
            question=tc["question"],
            context=context or tc.get("context", ""),
            answer=result.get("answer", ""),
            ground_truth=tc["ground_truth"],
        ))

    current_results = run_eval_suite(eval_cases)
    regressions = suite.compare_to_baseline(current_results)

    if regressions:
        print("REGRESSIONS DETECTED:")
        for r in regressions:
            print(f"{r['metric']}: {r['baseline']:.3f} → {r['current']:.3f} (drop: {r['drop']:.3f})")
        return False

    print(f"No regressions. Scores: {current_results['summary']}")
    return True

```

20.6 Metrics Dashboard

```
# metrics_dashboard.py - run with: streamlit run metrics_dash
board.py
import streamlit as st
import json, time
from pathlib import Path
from collections import defaultdict, deque
from dataclasses import dataclass, field

st.set_page_config(page_title="GenAI Metrics", page_icon
="📊", layout="wide")
st.title("📊 GenAI System Metrics")

# — Mock metrics store (replace with your real metrics backe
nd) —————
@dataclass
class MetricsStore:
    latencies: deque = field(default_factory=lambda: deque
(maxlen=500))
    token_costs: deque = field(default_factory=lambda: deque
(maxlen=500))
    eval_scores: dict = field(default_factory=lambda: defa
ultdict(list))
    request_count: int = 0
    error_count: int = 0

@st.cache_resource
def get_metrics() -> MetricsStore:
    import random, math
    store = MetricsStore()
    rng = random.Random(42)
    # Simulate historical data
    for i in range(200):
        store.latencies.append(rng.gauss(450, 80))
```

```

        store.token_costs.append(rng.gauss(0.004, 0.001))
        store.eval_scores["faithfulness"].append(min(1.0, rn
g.gauss(0.85, 0.08)))
        store.eval_scores["answer_relevance"].append(min(1.0,
rng.gauss(0.88, 0.06)))
        store.eval_scores["hallucination"].append(min(1.0, rn
g.gauss(0.92, 0.05)))
        store.request_count = 1420
        store.error_count = 12
    return store

metrics = get_metrics()

# — KPI row —————
k1, k2, k3, k4 = st.columns(4)
latencies = list(metrics.latencies)
costs = list(metrics.token_costs)

k1.metric("Total Requests", f"{metrics.request_count:,}")
k2.metric("Error Rate", f"{metrics.error_count / max(met
rics.request_count, 1):.1%}")
k3.metric("Avg Latency", f"{sum(latencies)/len(latencie
s):.0f}ms")
k4.metric("Avg Cost/Request", f"${sum(costs)/len(costs):.5f}")

st.divider()

# — Latency distribution —————
col_lat, col_eval = st.columns(2)

with col_lat:
    st.subheader("Latency Distribution (ms)")
    sorted_lat = sorted(latencies)
    n = len(sorted_lat)

```

```

    st.write(f"P50: **{sorted_lat[n//2]:.0f}ms** | P95: **{sorted_lat[int(n*0.95)]:.0f}ms** | P99: **{sorted_lat[int(n*0.99)]:.0f}ms**")
    import pandas as pd
    st.bar_chart(pd.DataFrame({"latency_ms": sorted_lat}).assign(bucket=lambda df: (df["latency_ms"] // 50) * 50).groupby("bucket").size())

# — Eval scores —————
with col_eval:
    st.subheader("Evaluation Scores (LLM-as-judge)")
    for metric, values in metrics.eval_scores.items():
        avg = sum(values) / len(values)
        colour = "🟢" if avg >= 0.8 else "🟡" if avg >= 0.6 else "🔴"
        st.progress(avg, text=f"{colour}{metric}:{avg:.3f}")

st.divider()

# — Cost over time —————
st.subheader("Cost per Request (rolling)")
import pandas as pd
cost_df = pd.DataFrame({"cost_usd": costs}).rolling(20).mean().dropna()
st.line_chart(cost_df)

# — Live refresh —————
if st.button("Refresh metrics"):
    st.rerun()

st.caption(f"Last updated:{time.strftime('%H:%M:%S')}")

```

20.7 Production Checklist

```
# production_checklist.py
# Run this as part of your CI/CD pipeline before deploying

CHECKLIST = [
    # Security
    ("No hardcoded API keys", "Scan all .py files for 'sk-' and 'sk-ant-' patterns"),
    ("All secrets via environment variables", "Check os.environ usage, no direct string keys"),
    ("API key validation on all endpoints", "Every route uses Depends(verify_api_key)"),
    ("Rate limiting enabled", "RateLimiter attached to all public endpoints"),
    ("CORS origin whitelist set", "allow_origins is not ['*'] in production"),

    # Reliability
    ("Retry with exponential backoff", "LLM calls wrapped in retry decorator"),
    ("Timeout on all API calls", "httpx/anthropic client has timeout set"),
    ("Circuit breaker for external deps", "Check for tenacity or similar"),
    ("Health endpoint returns 200", "GET /health returns {status: 'ok'}"),
    ("Graceful shutdown in lifespan", "Resources closed in lifespan yield block"),

    # Observability
    ("Structured JSON logging", "All loggers use JSONFormatter"),
    ("Request tracing", "trace_id propagated through all log entries"),
    ("Metrics endpoint", "GET /metrics returns request counts")
]
```

```

and latencies"),
    ("Error rates alerted", "Alert if error rate > 1% in any
5-minute window"),

    # Evaluation
    ("Baseline eval scores recorded", "baselines/eval_baselin
e.json exists"),
    ("Regression tests in CI", "pytest tests/eval/ runs on ev
ery PR"),
    ("Hallucination check enabled", "eval_hallucination calle
d on sampled responses"),

    # Performance
    ("LLM clients are async", "Using AsyncAnthropic / AsyncOp
enAI"),
    ("Embeddings cached", "Identical texts not re-embedded"),
    ("Vector DB indices built", "HNSW indices not rebuilt on
every query"),
    ("Response streaming enabled", "StreamingResponse used fo
r chat endpoints"),
]

def print_checklist() -> None:
    print("=" * 60)
    print("PRODUCTION READINESS CHECKLIST")
    print("=" * 60)
    for i, (item, detail) in enumerate(CHECKLIST, 1):
        print(f"  [{i:02d}].{item}")
        print(f"          →{detail}")
    print("=" * 60)
    print(f"  Total checks:{len(CHECKLIST)}")

print_checklist()

```

20.8 Module 20 Exercises

1. Integrate the `Tracer` from section 20.3 into the FastAPI RAG endpoint from Module 19. Every request should produce a trace with separate spans for embedding, retrieval, reranking, and generation. Log all spans as JSON.
2. Build an automated nightly evaluation job: a Python script that loads 50 stored conversation pairs from SQLite, runs the full eval suite (faithfulness, answer relevance, hallucination), saves results to a dated JSON file, and sends a Slack webhook notification if any score drops below 0.75.
3. Implement a `HallucinationMonitor` that: (a) samples 10% of all production responses randomly, (b) runs `eval_hallucination` on each sample, (c) stores results in SQLite, and (d) exposes a `GET /monitoring/hallucination-rate` endpoint returning the 7-day rolling rate.
4. Build a complete cost attribution system: instrument every LLM call to record model, input tokens, output tokens, and session ID. Expose a `GET /analytics/cost?session_id=X` endpoint returning total cost for that session, and a `GET /analytics/cost/daily` returning per-day totals for the last 30 days.

Phase 5 Summary

What You Learned

Module	Core Skills
M18 - Streamlit	<code>st.chat_message</code> / <code>st.chat_input</code> chat UI, <code>st.session_state</code> for history, <code>st.write_stream</code> for streaming, <code>@st.cache_resource</code> for clients, file upload with indexing, multi-page app structure, Streamlit Cloud deployment
M19 - FastAPI	Lifespan startup/shutdown, Pydantic request validation with <code>field_validator</code> , async LLM endpoints, SSE streaming with <code>StreamingResponse</code> , API key auth with <code>Depends</code> , token-bucket rate limiting, <code>BackgroundTasks</code> for non-blocking logging, <code>TestClient</code> for unit tests
M20 - Observability	<code>JSONFormatter</code> structured logging, <code>Tracer</code> with spans and parent-child relationships, LLM-as-judge eval (faithfulness, relevance, hallucination, conciseness), <code>RegressionSuite</code> with baseline comparison, Streamlit metrics dashboard, production readiness checklist

Patterns to Carry Forward

- Use `@st.cache_resource` for any object with a network connection - creating an `Anthropic` client on every rerun wastes time and may exhaust connection pools
 - Streamlit re-runs the entire script on every interaction - anything outside `st.session_state` is reset; always initialise state with `if "key" not in st.session_state`
 - FastAPI middleware runs in reverse order of registration - add timing middleware last so it wraps everything
 - SSE streaming requires `Cache-Control: no-cache` and `X-Accel-Buffering: no` headers when behind nginx; without them, nginx buffers the entire response before sending
 - Always use `async` LLM clients (`AsyncAnthropic` , `AsyncOpenAI`) in FastAPI - sync clients block the event loop and kill concurrency
 - LLM-as-judge scores drift if you change the judge model - pin the judge model version in your eval config and treat it like a dependency
 - A regression suite is only useful if it runs automatically - wire it into CI before merging any change to prompts, models, or retrieval parameters
-

What Is Next - Phase 6

Phase 6 is the final phase: three **end-to-end portfolio projects**, each one production-grade and fully runnable:

- **P01** - AI Research Assistant: RAG over uploaded PDFs, streaming answers with citations, Streamlit UI
- **P02** - Autonomous AI Agent: LangGraph ReAct agent with web search, code execution, and persistent memory, served via FastAPI
- **P03** - Multi-Agent Content Studio: CrewAI pipeline - researcher → writer → editor → SEO - with a Streamlit UI and FastAPI backend

No additional installs needed for Phase 6 - all libraries were covered in Phases 1–5.

Phase 6 - Portfolio Projects

All libraries covered in Phases 1–5 - no new installs required. Each project is fully runnable end-to-end with your API keys.

Project 01 - AI Research Assistant

RAG over uploaded documents · Streaming answers with inline citations · Streamlit UI

What You Are Building

A research assistant that ingests PDF and text documents, indexes them into a vector store, and answers questions with streaming responses that include inline `[Source N]` citations linking back to the original document chunks. Users interact through a polished Streamlit chat UI.

Stack: `anthropic` · `openai` (embeddings) · `chromadb` · `pdfplumber` · `streamlit`

Skills demonstrated: Document ingestion pipeline, chunking with metadata, vector search, RAG prompt construction, streaming with citations, session state management, persistent index.

Project Structure

```
p01_research_assistant/  
  app.py           ← Streamlit entry point  
  src/  
    ingestion.py   ← PDF/text loading and chunking  
    indexer.py      ← ChromaDB index management  
    retriever.py    ← Semantic search + reranking  
    generator.py    ← RAG prompt + Claude streaming  
    citations.py    ← Citation extraction and formatting  
  data/  
    chroma_db/      ← Persistent vector store (auto-created)  
  requirements.txt  
  .env
```

src/ingestion.py

```
# src/ingestion.py
import re
from dataclasses import dataclass, field
from pathlib import Path
from typing import Optional
import pdfplumber

@dataclass
class DocumentChunk:
    text: str
    source: str # filename
    page: Optional[int]
    chunk_index: int
    char_start: int
    char_end: int
    doc_id: str = field(init=False)

    def __post_init__(self):
        import hashlib
        self.doc_id = hashlib.md5(
            f"{self.source}_{self.chunk_index}".encode()
        ).hexdigest()[:12]

def load_pdf(path: str | Path) -> list[tuple[str, int]]:
    """Extract (text, page_number) tuples from a PDF."""
    pages = []
    with pdfplumber.open(str(path)) as pdf:
        for i, page in enumerate(pdf.pages, start=1):
            text = page.extract_text() or ""
            text = text.strip()
            if text:
                pages.append((text, i))
    return pages
```

```

def load_text(path: str | Path) -> list[tuple[str, int]]:
    """Load a plain-text or markdown file as a single page
    e."""
    text = Path(path).read_text(encoding="utf-8", errors="ignore").strip()
    return [(text, 1)] if text else []

def chunk_page(
    text: str,
    source: str,
    page: int,
    start_index: int = 0,
    chunk_size: int = 800,
    overlap: int = 100,
) -> list[DocumentChunk]:
    """Split a page's text into overlapping chunks."""
    chunks = []
    start = 0
    idx = start_index

    # Prefer paragraph boundaries
    paragraphs = [p.strip() for p in re.split(r'\n{2,}', text) if p.strip()]

    current = ""
    current_start = 0

    for para in paragraphs:
        candidate = (current + "\n\n" + para).strip() if current else para
        if len(candidate) > chunk_size and current:
            chunks.append(DocumentChunk(
                text=current.strip(),

```

```

        source=source,
        page=page,
        chunk_index=idx,
        char_start=current_start,
        char_end=current_start + len(current),
    ))
    idx += 1
    # Start next chunk with overlap from end of current
    overlap_text = current[-overlap:] if len(current)
> overlap else current
    current = (overlap_text + "\n\n" + para).strip()
    current_start += len(current) - len(overlap_text)
else:
    current = candidate

if current.strip():
    chunks.append(DocumentChunk(
        text=current.strip(),
        source=source,
        page=page,
        chunk_index=idx,
        char_start=current_start,
        char_end=current_start + len(current),
    ))

return chunks

def ingest_file(path: str | Path, chunk_size: int = 800, overlap: int = 100) -> list[DocumentChunk]:
    """Load and chunk any supported file type."""
    path = Path(path)
    suffix = path.suffix.lower()

```

```

if suffix == ".pdf":
    pages = load_pdf(path)
elif suffix in (".txt", ".md"):
    pages = load_text(path)
else:
    raise ValueError(f"Unsupported file type:{suffix}. Use .pdf, .txt, or .md")

all_chunks: list[DocumentChunk] = []
for text, page_num in pages:
    new_chunks = chunk_page(
        text=text,
        source=path.name,
        page=page_num,
        start_index=len(all_chunks),
        chunk_size=chunk_size,
        overlap=overlap,
    )
    all_chunks.extend(new_chunks)

return all_chunks

```

src/indexer.py

```

# src/indexer.py
import chromadb
import os
from openai import OpenAI
from pathlib import Path
from src.ingestion import DocumentChunk, ingest_file
from dotenv import load_dotenv

load_dotenv()

EMBED_MODEL = "text-embedding-3-small"

```

```

CHROMA_DIR = "data/chroma_db"

def get_embed_client() -> OpenAI:
    return OpenAI(api_key=os.environ["OPENAI_API_KEY"])

def embed_texts(texts: list[str], client: OpenAI) -> list[list[float]]:
    """Embed in batches of 100."""
    embeddings: list[list[float]] = []
    for i in range(0, len(texts), 100):
        batch = texts[i : i + 100]
        resp = client.embeddings.create(model=EMBED_MODEL, input=batch)
        embeddings.extend(e.embedding for e in sorted(resp.data, key=lambda x: x.index))
    return embeddings

class ResearchIndex:
    """Manages a persistent ChromaDB collection for the research assistant."""

    def __init__(self, persist_dir: str = CHROMA_DIR):
        Path(persist_dir).mkdir(parents=True, exist_ok=True)
        self._chroma = chromadb.PersistentClient(path=persist_dir)
        self._collection = self._chroma.get_or_create_collection(
            name="research_docs",
            metadata={"hnsw:space": "cosine"},
        )
        self._embed_client = get_embed_client()

    def indexed_sources(self) -> list[str]:

```

```

x."""
    """Return list of unique source filenames in the index."""
    if self._collection.count() == 0:
        return []
    results = self._collection.get(include=["metadatas"])
    return sorted({m["source"] for m in results["metadatas"]})

    def is_indexed(self, filename: str) -> bool:
        return filename in self.indexed_sources()

    def add_file(self, path: str, chunk_size: int = 800, overlap: int = 100) -> int:
        """Ingest, embed, and index a file. Returns number of chunks added."""
        chunks = ingest_file(path, chunk_size=chunk_size, overlap=overlap)
        if not chunks:
            return 0

        # Skip chunks already in the index
        existing_ids = set(self._collection.get(include=[]))
        ["ids"])
        new_chunks = [c for c in chunks if c.doc_id not in existing_ids]
        if not new_chunks:
            return 0

        texts = [c.text for c in new_chunks]
        embeddings = embed_texts(texts, self._embed_client)
        ids = [c.doc_id for c in new_chunks]
        metadatas = [
            {
                "source": c.source,
                "page": c.page or 0,
                "chunk_index": c.chunk_index,

```

```

        "char_start": c.char_start,
        "char_end": c.char_end,
    }
    for c in new_chunks
]

self._collection.add(
    ids=ids,
    documents=texts,
    embeddings=embeddings,
    metadatas=metadatas,
)
return len(new_chunks)

def remove_file(self, filename: str) -> int:
    """Remove all chunks for a given source file."""
    results = self._collection.get(
        where={"source": {"$eq": filename}},
        include=[],
    )
    if results["ids"]:
        self._collection.delete(ids=results["ids"])
    return len(results["ids"])

@property
def total_chunks(self) -> int:
    return self._collection.count()

```

src/retriever.py

```

# src/retriever.py
import os, json
from dataclasses import dataclass
from openai import OpenAI
from dotenv import load_dotenv

```



```

load_dotenv()

EMBED_MODEL = "text-embedding-3-small"

@dataclass
class RetrievedChunk:
    text: str
    source: str
    page: int
    chunk_index: int
    score: float

class Retriever:
    """Semantic search over the ResearchIndex."""

    def __init__(self, index: "ResearchIndex"):
        self._index = index
        self._embed_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

    def _embed(self, text: str) -> list[float]:
        resp = self._embed_client.embeddings.create(model=EMBED_MODEL, input=[text])
        return resp.data[0].embedding

    def retrieve(
        self,
        query: str,
        k: int = 6,
        source_filter: list[str] | None = None,
        min_score: float = 0.25,
    ) -> list[RetrievedChunk]:
        """Return top-k relevant chunks, optionally filtered

```

```

by source filename."""
    if self._index.total_chunks == 0:
        return []

    q_emb = self._embed(query)
    n = min(k * 2, self._index.total_chunks) # over
-fetch then filter

    query_kwargs: dict = {
        "query_embeddings": [q_emb],
        "n_results": n,
        "include": ["documents", "distances", "m
etadatas"],
    }
    if source_filter:
        query_kwargs["where"] = {"source": {"$in": source
_filter}}

    results = self._index._collection.query(**query_kwarg
s)

    chunks = []

    for text, dist, meta in zip(
        results["documents"][0],
        results["distances"][0],
        results["metadatas"][0],
    ):
        score = round(1.0 - dist / 2.0, 4) # cosine dis
tance → similarity
        if score >= min_score:
            chunks.append(RetrievedChunk(
                text=text,
                source=meta["source"],
                page=meta.get("page", 0),
                chunk_index=meta.get("chunk_index", 0),
                score=score,

```

```

    ))

    return sorted(chunks, key=lambda c: c.score, reverse=
True)[:k]

```

src/generator.py

```

# src/generator.py
import os
from typing import Generator
import anthropic
from src.retriever import RetrievedChunk
from dotenv import load_dotenv

load_dotenv()

SYSTEM_PROMPT = """You are a research assistant. Answer the u
ser's question using ONLY
the provided source excerpts. Every factual claim must be fol
lowed by a citation in the
format [Source N] corresponding to the numbered sources belo
w.

Rules:
- Only use information from the provided sources.
- If the sources don't contain the answer, say: "I don't have
enough information to answer that from the provided document
s."
- Never fabricate citations or facts.
- Be thorough but concise."""

def build_context(chunks: list[RetrievedChunk]) -> str:
    """Format retrieved chunks as numbered sources for the pr
ompt."""

```

```

parts = []
for i, chunk in enumerate(chunks, start=1):
    header = f"[Source{i}]{chunk.source}"
    if chunk.page:
        header += f", page{chunk.page}"
    parts.append(f"{header}\n{chunk.text}")
return "\n\n---\n\n".join(parts)

def stream_answer(
    question: str,
    chunks: list[RetrievedChunk],
    history: list[dict],
    model: str = "claude-sonnet-4-5",
    max_tokens: int = 1536,
    temperature: float = 0.3,
) -> Generator[str, None, None]:
    """
    Stream the LLM answer token by token.
    Yields text strings. Caller accumulates them.
    """
    client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])
    context = build_context(chunks)

    user_content = f"""Sources:
{context}

---

Question:{question}"""

    messages = list(history) + [{"role": "user", "content": user_content}]

    with client.messages.stream(

```

```

        model=model,
        max_tokens=max_tokens,
        system=SYSTEM_PROMPT,
        messages=messages,
    ) as stream:
        for text in stream.text_stream:
            yield text

```

src/citations.py

```

# src/citations.py
import re
from src.retriever import RetrievedChunk

def extract_cited_sources(answer: str, chunks: list[Retrieved
Chunk]) -> list[dict]:
    """
    Parse [Source N] markers from the answer and return the r
eferenced chunks.
    Returns a list of unique cited sources with their metadat
a.
    """
    cited_indices: list[int] = []
    for match in re.finditer(r'\[Source(\d+)\]', answer):
        idx = int(match.group(1)) - 1
        if 0 <= idx < len(chunks) and idx not in cited_indice
s:
            cited_indices.append(idx)

    return [
        {
            "number": i + 1,
            "source": chunks[idx].source,
            "page": chunks[idx].page,

```

```

        "score": chunks[idx].score,
        "excerpt": chunks[idx].text[:200] + ("..." if len
(chunks[idx].text) > 200 else ""),
    }
    for i, idx in enumerate(cited_indices)
]

def format_sources_for_display(cited: list[dict]) -> str:
    """Format cited sources as a readable markdown list."""
    if not cited:
        return ""
    lines = ["\n\n**Sources used:**"]
    for c in cited:
        page_info = f", p.{c['page']}" if c.get("page") else ""
        lines.append(f"- **[Source{c['number']}]**{c['source']}{page_info} (relevance:{c['score']:.2f})")
    return "\n".join(lines)

```

app.py - The Streamlit Interface

```

# app.py
import streamlit as st
import tempfile, os
from pathlib import Path
from src.indexer import ResearchIndex
from src.retriever import Retriever
from src.generator import stream_answer
from src.citations import extract_cited_sources, format_sources_for_display

st.set_page_config(
    page_title="AI Research Assistant",
    page_icon="🔬",

```

```

        layout="wide",
    )

# — Cached resources —————
@st.cache_resource
def get_index() -> ResearchIndex:
    return ResearchIndex(persist_dir="data/chroma_db")

@st.cache_resource
def get_retriever(_index: ResearchIndex) -> Retriever:
    return Retriever(_index)

index = get_index()
retriever = get_retriever(index)

# — Session state —————
if "messages" not in st.session_state: st.session_state.messages = []
if "source_filter" not in st.session_state: st.session_state.source_filter = []

# — Sidebar —————
with st.sidebar:
    st.title("🔬 Research Assistant")
    st.caption("Upload documents, ask questions, get cited answers.")

    st.subheader("Upload Documents")
    uploaded = st.file_uploader(
        "PDF, TXT, or MD files",
        type=["pdf", "txt", "md"],
        accept_multiple_files=True,
        help="Documents are chunked, embedded, and stored per

```

```

sistently.",
    )

    if uploaded:
        for file in uploaded:
            if not index.is_indexed(file.name):
                with st.spinner(f"Indexing{file.name}..."):
                    suffix = Path(file.name).suffix
                    with tempfile.NamedTemporaryFile(delete=False,
else, suffix=suffix) as tmp:
                        tmp.write(file.read())
                        tmp_path = tmp.name
                        try:
                            n = index.add_file(tmp_path)
                            st.success(f"✓{file.name}:{n} chunks
indexed")
                        except Exception as e:
                            st.error(f"Failed to index{file.name
e}:{e}")
                        finally:
                            os.unlink(tmp_path)
                    else:
                        st.info(f"Already indexed:{file.name}")

indexed_sources = index.indexed_sources()
if indexed_sources:
    st.subheader("Indexed Documents")
    for src in indexed_sources:
        col1, col2 = st.columns([4, 1])
        col1.caption(f"📄{src}")
        if col2.button("✕", key=f"del_{src}", help=f"Remo
ve{src}"):
            n = index.remove_file(src)
            st.success(f"Removed{n} chunks for{src}")
            st.rerun()

```



```

st.subheader("Filter Sources")
st.session_state.source_filter = st.multiselect(
    "Search only these documents",
    options=indexed_sources,
    default=[],
    help="Leave empty to search all documents.",
)

st.divider()
with st.expander("Settings"):
    model = st.selectbox("Model", ["claude-sonnet-4-5", "claude-haiku-4-5"])
    k = st.slider("Chunks to retrieve", 3, 10, 6)
    min_score = st.slider("Min relevance score", 0.1, 0.9, 0.25, step=0.05)
    show_chunks = st.checkbox("Show retrieved chunks", value=False)
    temperature = st.slider("Temperature", 0.0, 1.0, 0.3, step=0.05)

    if st.button("Clear conversation"):
        st.session_state.messages = []
        st.rerun()

# — Main chat area —
st.title("AI Research Assistant")

if not indexed_sources:
    st.info("👉 Upload one or more documents in the sidebar to get started.")
else:
    st.caption(f"Searching across **{len(indexed_sources)}** document(s) · {index.total_chunks:,} total chunks")

```

```

for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])

if question := st.chat_input("Ask a research question...", disabled=not indexed_sources):
    st.session_state.messages.append({"role": "user", "content": question})
    with st.chat_message("user"):
        st.markdown(question)

    # Retrieve
    source_filter = st.session_state.source_filter or None
    chunks = retriever.retrieve(question, k=k, source_filter=source_filter, min_score=min_score)

    if show_chunks:
        with st.expander(f"🔍 Retrieved {len(chunks)} chunk(s)"):
            for i, c in enumerate(chunks, 1):
                st.markdown(f"***[Source {i}]** `{c.source}` p. {c.page} · score {c.score:.3f}")
                st.caption(c.text[:300] + "...")
                st.divider()

    if not chunks:
        answer = "No relevant content found in the indexed documents for that question."
        with st.chat_message("assistant"):
            st.markdown(answer)
        st.session_state.messages.append({"role": "assistant", "content": answer})
    else:
        # Build history for context (last 6 turns, exclude source blocks)
        history = [

```

```

        {"role": m["role"], "content": m["content"]}
    for m in st.session_state.messages[-6:]
    if m["role"] in ("user", "assistant")
]

# Stream answer
with st.chat_message("assistant"):
    full_text = st.write_stream(
        stream_answer(
            question=question,
            chunks=chunks,
            history=history[:-1], # exclude current
question (already in chunks prompt)
            model=model,
            temperature=temperature,
        )
    )

    # Append citations section
    cited = extract_cited_sources(full_text, chunk
s)

    src_text = format_sources_for_display(cited)
    if src_text:
        st.markdown(src_text)

    final_content = full_text + src_text
    st.session_state.messages.append({"role": "assistan
t", "content": final_content})

```

Running P01

```

# 1. Clone/create the project folder
mkdir p01_research_assistant && cd p01_research_assistant

# 2. Create .env

```

```
echo "ANTHROPIC_API_KEY=sk-ant-..." > .env
echo "OPENAI_API_KEY=sk-..." >> .env

# 3. Install dependencies
pip install anthropic openai chromadb pdfplumber streamlit py
thon-dotenv

# 4. Run
streamlit run app.py

# 5. Open http://localhost:8501
# Upload a PDF → ask a question → get a cited answer
```

What This Demonstrates

- **Ingestion pipeline** - PDF and text loading, paragraph-aware chunking with overlap, deduplication by content hash
- **Persistent vector store** - ChromaDB survives restarts; documents are not re-embedded on relaunch
- **Metadata filtering** - users can restrict search to specific uploaded files
- **Cited streaming answers** - `[Source N]` markers extracted and formatted as a source list after streaming completes
- **Session memory** - last 6 turns passed as context so follow-up questions work correctly
- **Production patterns** - `@st.cache_resource` for the index and retriever, progress feedback, error handling on upload

Project 02 - Autonomous AI Agent

LangGraph ReAct agent · Web search · Code execution · Persistent memory · FastAPI backend

What You Are Building

An autonomous agent that can plan multi-step tasks, search the web, execute Python code, look up information, and remember context across sessions. A FastAPI backend serves the agent; a Streamlit frontend displays the agent's reasoning steps and streams the final answer.

Stack: `langgraph` · `anthropic` · `fastapi` · `uvicorn` · `streamlit` · `sqlite3`

Skills demonstrated: LangGraph `StateGraph`, ReAct loop, tool design, persistent `MemorySaver` with thread IDs, streaming SSE from FastAPI, Streamlit consuming SSE, background task logging.

Project Structure

```
p02_autonomous_agent/  
  backend/  
    main.py          ← FastAPI app  
    agent.py         ← LangGraph ReAct agent  
    tools.py         ← Tool implementations  
    memory.py        ← SQLite session persistence  
    models.py        ← Pydantic request/response models  
  frontend/  
    app.py           ← Streamlit UI  
  data/  
    sessions.db      ← SQLite memory store (auto-created)  
  .env
```

backend/tools.py

```
# backend/tools.py  
import math, json, subprocess, tempfile, os, re  
from datetime import datetime  
  
# — Tool registry —  
  
TOOL_SCHEMAS = [  
    {  
        "type": "function",  
        "function": {  
            "name": "calculator",
```

```

        "description": "Evaluate mathematical expressions. Supports Python math syntax including math.sqrt(), math.pi, etc.",
        "parameters": {
            "type": "object",
            "properties": {
                "expression": {"type": "string", "description": "Python math expression to evaluate"}
            },
            "required": ["expression"],
        },
    },
    {
        "type": "function",
        "function": {
            "name": "python_repl",
            "description": "Execute Python code and return stdout/stderr. Use for data analysis, string processing, and computations too complex for the calculator.",
            "parameters": {
                "type": "object",
                "properties": {
                    "code": {"type": "string", "description": "Python code to execute"}
                },
                "required": ["code"],
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "web_search",
            "description": "Search the web for current information. Returns top 3 results with titles and snippets.",

```

```

        "parameters": {
            "type": "object",
            "properties": {
                "query": {"type": "string", "description": "Search query"}
            },
            "required": ["query"],
        },
    },
    {
        "type": "function",
        "function": {
            "name": "get_datetime",
            "description": "Get the current date and time.",
            "parameters": {"type": "object", "properties": {}},
            "required": [],
        },
    },
    {
        "type": "function",
        "function": {
            "name": "save_note",
            "description": "Save a note or result to the session memory for later reference.",
            "parameters": {
                "type": "object",
                "properties": {
                    "title": {"type": "string", "description": "Short title for the note"},
                    "content": {"type": "string", "description": "Note content"}
                },
                "required": ["title", "content"],
            },
        },
    },

```

```

    },
]

# — Tool implementations —————

def calculator(expression: str) -> str:
    try:
        safe_env = {k: v for k, v in math.__dict__.items() if
not k.startswith("_")}
        safe_env.update({"abs": abs, "round": round, "pow": p
ow, "math": math})
        result = eval(expression, {"__builtins__": {}}, safe_
env)

        return str(round(float(result), 8))
    except Exception as e:
        return f"Error:{e}"

def python_repl(code: str) -> str:
    with tempfile.NamedTemporaryFile(mode="w", suffix=".py",
delete=False) as f:
        f.write(code)
        fname = f.name
    try:
        result = subprocess.run(
            ["python3", fname],
            capture_output=True,
            text=True,
            timeout=15,
        )
        output = (result.stdout + result.stderr).strip()
        return output[:3000] if output else "(no output)"
    except subprocess.TimeoutExpired:
        return "Error: execution timed out (15 second limit)"

```



```

except Exception as e:
    return f"Error:{e}"
finally:
    os.unlink(fname)

def web_search(query: str) -> str:
    """
    Mock web search. Replace with a real search API (Brave, S
    erper, Tavily)
    by swapping this implementation - the schema and agent lo
    op stay the same.
    """
    mock_results = [
        {
            "title": f"Understanding{query} - Comprehensive
Guide",
            "url": f"https://example.com/{query.replace('
', '-').lower()}",
            "snippet": f"{query} involves several key concept
s. Modern approaches use neural networks and attention mechan
isms to process information effectively.",
        },
        {
            "title": f"{query} - Latest Research 2024",
            "url": f"https://research.example.com/{query.
replace(' ', '-').lower()}",
            "snippet": f"Recent advances in{query} have shown
significant improvements in accuracy and efficiency across mu
ltiple benchmarks.",
        },
        {
            "title": f"Practical{query} Implementation Guid
e",
            "url": f"https://docs.example.com/{query.repl
ace(' ', '-').lower()}",

```

```

        "snippet": f"This guide covers the practical aspects of implementing{query}, including setup, configuration, and best practices.",
    },
]
return json.dumps(mock_results, indent=2)

def get_datetime() -> str:
    return datetime.now().strftime("%A, %B%d %Y at %H:%M:%S")

# Notes store (in-memory per session - persisted to SQLite by memory.py)
_session_notes: dict[str, list[dict]] = {}

def save_note(title: str, content: str, session_id: str = "default") -> str:
    if session_id not in _session_notes:
        _session_notes[session_id] = []
    _session_notes[session_id].append({"title": title, "content": content})
    return f>Note '{title}' saved successfully."

def get_notes(session_id: str = "default") -> list[dict]:
    return _session_notes.get(session_id, [])

TOOL_FUNCTIONS = {
    "calculator": calculator,
    "python_repl": python_repl,
    "web_search": web_search,
    "get_datetime": get_datetime,
    "save_note": save_note,
}

```

backend/agent.py

```
# backend/agent.py
import json, os
from typing import TypedDict, Annotated, AsyncGenerator
import operator
import anthropic
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from langchain_core.messages import HumanMessage, AIMessage,
ToolMessage, SystemMessage
from backend.tools import TOOL_SCHEMAS, TOOL_FUNCTIONS
from dotenv import load_dotenv

load_dotenv()

SYSTEM_PROMPT = """You are an autonomous AI agent with access
to tools: calculator, python_repl, web_search, get_datetime,
and save_note.

Use tools whenever they will improve accuracy:
- calculator: for any math
- python_repl: for complex data processing or code generation
- web_search: for current events or facts you are uncertain a
bout
- get_datetime: when the current time is needed
- save_note: to record important findings mid-task

Think step by step. Always verify numerical results with the
calculator. When done, give a clear, complete final answe
r."""

MAX_ITERATIONS = 15
```

```

class AgentState(TypedDict):
    messages: Annotated[list, operator.add]
    iteration: int
    session_id: str

def make_agent(model: str = "claude-sonnet-4-5") -> tuple:
    """Build and compile the LangGraph ReAct agent. Returns
    (app, checkpoint)."""

    client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

# — Nodes —

def llm_node(state: AgentState) -> dict:
    """Call Claude with tool definitions."""
    response = client.messages.create(
        model=model,
        max_tokens=2048,
        system=SYSTEM_PROMPT,
        tools=TOOL_SCHEMAS,
        messages=[
            {"role": m["role"], "content": m["content"]}
            for m in state["messages"]
        ],
    )

    # Convert to a unified message format
    content_blocks = []
    for block in response.content:
        if block.type == "text":
            content_blocks.append({"type": "text", "text": block.text})
        elif block.type == "tool_use":

```

```

        content_blocks.append({
            "type": "tool_use",
            "id": block.id,
            "name": block.name,
            "input": block.input,
        })

    return {
        "messages": [{"role": "assistant", "content": content_blocks}],
        "iteration": state["iteration"] + 1,
    }

def tool_node(state: AgentState) -> dict:
    """Execute all tool calls requested by the LLM."""
    last_msg = state["messages"][-1]
    content = last_msg["content"]
    tool_results = []

    for block in content:
        if isinstance(block, dict) and block.get("type") == "tool_use":
            fn_name = block["name"]
            fn_args = dict(block["input"])

            # Inject session_id for save_note
            if fn_name == "save_note":
                fn_args["session_id"] = state.get("session_id", "default")

            fn = TOOL_FUNCTIONS.get(fn_name)
            result = fn(*fn_args) if fn else f"Unknown tool: {fn_name}"

            tool_results.append({
                "type": "tool_result",

```

```

        "tool_use_id": block["id"],
        "content": str(result),
    })

    return {"messages": [{"role": "user", "content": tool
_results}]]}

# — Routing —————

def route(state: AgentState) -> str:
    if state["iteration"] >= MAX_ITERATIONS:
        return END
    last = state["messages"][-1]
    content = last.get("content", [])
    if isinstance(content, list):
        has_tools = any(
            isinstance(b, dict) and b.get("type") == "tool
_use"
            for b in content
        )
        return "tools" if has_tools else END
    return END

# — Graph —————

graph = StateGraph(AgentState)
graph.add_node("llm", llm_node)
graph.add_node("tools", tool_node)

graph.add_edge(START, "llm")
graph.add_conditional_edges("llm", route, {"tools": "tool
s", END: END})
graph.add_edge("tools", "llm")

```

```

    checkpointer = MemorySaver()
    app = graph.compile(checkpointer=checkpointer)
    return app, checkpointer

def extract_final_answer(messages: list[dict]) -> str:
    """Extract the last text-only assistant message as the final answer."""
    for msg in reversed(messages):
        if msg.get("role") != "assistant":
            continue
        content = msg.get("content", [])
        if isinstance(content, str):
            return content
        if isinstance(content, list):
            texts = [b["text"] for b in content if isinstance(b, dict) and b.get("type") == "text"]
            if texts:
                return "\n".join(texts)
    return "No final answer produced."

def extract_tool_calls(messages: list[dict]) -> list[dict]:
    """Extract tool call records for display in the UI."""
    calls = []
    for msg in messages:
        if msg.get("role") != "assistant":
            continue
        for block in (msg.get("content") or []):
            if isinstance(block, dict) and block.get("type") == "tool_use":
                calls.append({
                    "tool": block["name"],
                    "input": block["input"],
                })

```

```
    })  
    return calls
```

backend/main.py

```
# backend/main.py  
import json, time, os, asyncio  
from fastapi import FastAPI, HTTPException  
from fastapi.responses import StreamingResponse  
from fastapi.middleware.cors import CORSMiddleware  
from pydantic import BaseModel, Field  
from typing import Optional, AsyncGenerator  
from contextlib import asynccontextmanager  
from dotenv import load_dotenv  
  
load_dotenv()  
  
from backend.agent import make_agent, extract_final_answer, e  
xtract_tool_calls  
  
@asynccontextmanager  
async def lifespan(app: FastAPI):  
    app.state.agent, app.state.checkpointer = make_agent()  
    app.state.start_time = time.time()  
    yield  
  
app = FastAPI(title="Autonomous Agent API", version="1.0.0",  
lifespan=lifespan)  
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow  
_methods=["*"], allow_headers=["*"])  
  
class QueryRequest(BaseModel):  
    query: str = Field(..., min_length=1, max_l  
ength=10_000)
```



```

    session_id: str = Field(default="default", max_length=128)
    model: str = Field(default="claude-sonnet-4-5")

```

```

class QueryResponse(BaseModel):

```

```

    answer: str
    tool_calls: list[dict]
    iterations: int
    session_id: str
    latency_ms: float

```

```

@app.post("/agent/query", response_model=QueryResponse)

```

```

async def agent_query(req: QueryRequest):

```

```

    """Run the agent and return the complete response."""

```

```

    start = time.perf_counter()

```

```

    agent = app.state.agent

```

```

    config = {"configurable": {"thread_id": req.session_id}}

```

```

    initial_state = {

```

```

        "messages": [{ "role": "user", "content": req.query }],

```

```

        "iteration": 0,

```

```

        "session_id": req.session_id,

```

```

    }

```

```

    result = agent.invoke(initial_state, config=config)

```

```

    return QueryResponse(

```

```

        answer = extract_final_answer(result["messages"]),

```

```

        tool_calls = extract_tool_calls(result["messages"]),

```

```

        iterations = result["iteration"],

```

```

        session_id = req.session_id,

```

```

        latency_ms = round((time.perf_counter() - start) * 10
00, 2),
    )

async def run_agent_stream(query: str, session_id: str) -> As
yncGenerator[str, None]:
    """Run the agent and stream progress events as SSE."""
    agent = app.state.agent
    config = {"configurable": {"thread_id": session_id}}

    # Stream each step event
    initial_state = {
        "messages": [{"role": "user", "content": query}],
        "iteration": 0,
        "session_id": session_id,
    }

    # Use astream for async streaming of graph events
    async for event in agent.astream(initial_state, config=co
nfig, stream_mode="updates"):
        for node_name, node_output in event.items():
            if node_name == "llm":
                # Check for tool calls in progress
                messages = node_output.get("messages", [])
                for msg in messages:
                    content = msg.get("content", [])
                    if isinstance(content, list):
                        for block in content:
                            if isinstance(block, dict) and bl
ock.get("type") == "tool_use":
                                yield f"data:{json.dumps({'ty
pe': 'tool_call', 'tool': block['name'], 'input': block['inpu
t']})}\n\n"
                            elif isinstance(block, dict) and
block.get("type") == "text" and block.get("text"):

```

```

                                yield f"data:{json.dumps({'type': 'thinking', 'text': block['text'][:200]})}\n\n"

        elif node_name == "tools":
            messages = node_output.get("messages", [])
            for msg in messages:
                for result_block in (msg.get("content") or
r []):
                    if isinstance(result_block, dict) and
result_block.get("type") == "tool_result":
                        yield f"data:{json.dumps({'type':
'tool_result', 'content': str(result_block.get('content',
''))[:300]})}\n\n"

        # Final answer
        final_state = agent.get_state(config)
        answer      = extract_final_answer(final_state.values.get
("messages", []))
        yield f"data:{json.dumps({'type': 'done', 'answer': answe
r})}\n\n"

@app.post("/agent/query/stream")
async def agent_stream(req: QueryRequest) -> StreamingRespons
e:
    """Stream agent progress events as Server-Sent Events."""
    return StreamingResponse(
        run_agent_stream(req.query, req.session_id),
        media_type="text/event-stream",
        headers={"Cache-Control": "no-cache", "X-Accel-Buffer
ing": "no"},
    )

@app.get("/agent/sessions/{session_id}/notes")
async def get_session_notes(session_id: str) -> dict:

```

```

    """Return notes saved by the agent in this session."""
    from backend.tools import get_notes
    return {"session_id": session_id, "notes": get_notes(session_id)}

@app.get("/health")
async def health():
    return {"status": "ok", "uptime_s": round(time.time() - app.state.start_time, 1)}

```

frontend/app.py

```

# frontend/app.py
import streamlit as st
import requests, json, time, os

st.set_page_config(page_title="AI Agent", page_icon="🤖", layout="wide")
st.title("🤖 Autonomous AI Agent")

BACKEND = os.environ.get("AGENT_BACKEND_URL", "http://localhost:8000")

if "messages" not in st.session_state: st.session_state.messages = []
if "session_id" not in st.session_state:
    import uuid
    st.session_state.session_id = str(uuid.uuid4())[:8]

# — Sidebar —
with st.sidebar:
    st.header("Session")
    st.caption(f"Session ID: `{st.session_state.session_id}`")

```

```

`)

if st.button("New session"):
    import uuid
    st.session_state.session_id = str(uuid.uuid4())[:8]
    st.session_state.messages = []
    st.rerun()

st.divider()
st.header("Example Tasks")
examples = [
    "What is 15% of 47,832? Then write Python code to generate the first 20 Fibonacci numbers and sum them.",
    "Search for information about LangGraph and summarise the key features in bullet points.",
    "Calculate the compound interest on $10,000 at 7% annual rate for 20 years, then save a note with the result.",
    "What is today's date and day of the week? Then calculate how many days until December 31st.",
]
for ex in examples:
    if st.button(ex[:60] + "...", use_container_width=True):
        st.session_state.pending_query = ex

# — Render message history —————
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])
        if "tool_calls" in msg and msg["tool_calls"]:
            with st.expander(f"

```

```

# — Handle pending query from sidebar —————
query = st.chat_input("Ask the agent anything...")
if not query and hasattr(st.session_state, "pending_query"):
    query = st.session_state.pending_query
    del st.session_state.pending_query

if query:
    st.session_state.messages.append({"role": "user", "content": query})
    with st.chat_message("user"):
        st.markdown(query)

    with st.chat_message("assistant"):
        status = st.empty()
        tools_log = st.container()
        answer_ph = st.empty()

        tool_calls_seen: list[dict] = []
        final_answer = ""

        try:
            with requests.post(
                f"{BACKEND}/agent/query/stream",
                json={"query": query, "session_id": st.session_state.session_id},
                stream=True,
                timeout=120,
            ) as resp:
                resp.raise_for_status()
                for line in resp.iter_lines():
                    if not line:
                        continue
                    line = line.decode("utf-8") if isinstance(
                        (line, bytes) else line

```

```

        if not line.startswith("data: "):
            continue
        data = json.loads(line[6:])
        event_type = data.get("type")

        if event_type == "thinking":
            status.caption(f" Thinking:{data.get('text', '')[:80]}...")

        elif event_type == "tool_call":
            tool_name = data.get("tool", "unknown")

            status.caption(f" Using tool: `{tool_name}`")

            tool_calls_seen.append({"tool": tool_name, "input": data.get("input", {})})
            with tools_log.expander(f" {tool_name} {data.get('input', {})}", expanded=False):
                st.json(data.get("input", {}))

        elif event_type == "tool_result":
            status.caption("✓ Tool completed")

        elif event_type == "done":
            final_answer = data.get("answer", "")
            status.empty()
            answer_ph.markdown(final_answer)

    except requests.exceptions.ConnectionError:
        final_answer = "
```

```
st.session_state.messages.append({
    "role": "assistant",
    "content": final_answer,
    "tool_calls": tool_calls_seen,
})
```

Running P02

```
# Terminal 1 - start the backend
cd p02_autonomous_agent
uvicorn backend.main:app --reload --port 8000

# Terminal 2 - start the frontend
streamlit run frontend/app.py

# Open:
# - Frontend: http://localhost:8501
# - API docs: http://localhost:8000/docs
```

What This Demonstrates

- **LangGraph ReAct loop** - `llm` node requests tools, `tools` node executes them, conditional edge routes back until the model gives a final text answer
- **Persistent memory** - `MemorySaver` keyed by `session_id`; the agent remembers context across follow-up questions in the same session
- **Server-Sent Events** - the backend streams `tool_call`, `thinking`, `tool_result`, and `done` events; the frontend renders each in real time
- **Tool safety** - `python_repl` uses `subprocess` with a 15-second timeout in a temp file; `calculator` uses `eval` with a restricted namespace
- **Separation of concerns** - tools, agent logic, API, and UI are all in separate modules

Project 03 - Multi-Agent Content Studio

CrewAI pipeline · Researcher → Writer → Editor → SEO · Streamlit UI · FastAPI backend

What You Are Building

A content production pipeline where four specialised AI agents collaborate to produce a complete, publication-ready article. A Streamlit UI lets users configure the topic, audience, and style. A FastAPI backend runs the crew asynchronously and streams progress updates back to the frontend.

Stack: `crewai` · `langchain-openai` · `fastapi` · `streamlit`

Skills demonstrated: CrewAI `Agent` / `Task` / `Crew`, custom `BaseTool`, sequential process with task context, async job queue, SSE streaming, real-time progress tracking.

Project Structure

```
p03_content_studio/  
  backend/  
    main.py      ← FastAPI app with job queue  
    crew.py      ← CrewAI agents, tasks, crew  
    tools.py     ← Custom CrewAI tools  
    jobs.py      ← Async job manager  
  frontend/  
    app.py       ← Streamlit UI  
  .env
```

backend/tools.py

```
# backend/tools.py  
import json, re, time  
from pydantic import BaseModel, Field  
from crewai.tools import BaseTool  
from typing import Type
```

```

class WordCountInput(BaseModel):
    text: str = Field(description="Text to analyse")

class WordCountTool(BaseTool):
    name: str = "Word and Readability Analyser"
    description: str = "Count words, sentences, and estimate reading time for a piece of text."
    args_schema: Type[BaseModel] = WordCountInput

    def _run(self, text: str) -> str:
        words = len(text.split())
        sentences = max(1, len(re.findall(r'[.!?]+' , text)))
        paragraphs = len([p for p in text.split('\n\n') if p.strip()])
        avg_words_per_sentence = round(words / sentences, 1)
        read_time_min = round(words / 200, 1) # 200 wpm average

        return json.dumps({
            "word_count": words,
            "sentence_count": sentences,
            "paragraph_count": paragraphs,
            "avg_words_per_sentence": avg_words_per_sentence,
            "estimated_read_time_min": read_time_min,
            "assessment": "good" if 15 <= avg_words_per_sentence <= 25 else "needs adjustment",
        })

class SEOAnalyserInput(BaseModel):
    text: str = Field(description="Article content to analyse")
    primary_keyword: str = Field(description="Primary target keyword or phrase")

class SEOAnalyserTool(BaseTool):
    name: str = "SEO Keyword Analyser"

```

```

description: str = "Analyse keyword density, heading structure, and SEO readiness of content."
args_schema: Type[BaseModel] = SEOAnalyserInput

def _run(self, text: str, primary_keyword: str) -> str:
    words = text.lower().split()
    kw = primary_keyword.lower()
    # Count keyword occurrences (whole phrase)
    kw_count = len(re.findall(re.escape(kw), text.lower
    )))
    density = round(kw_count / max(len(words), 1) * 100,
    2)

    # Check heading structure
    h1_count = len(re.findall(r'^#\s', text, re.MULTILINE))
    h2_count = len(re.findall(r'^##\s', text, re.MULTILINE))
    h3_count = len(re.findall(r'^###\s', text, re.MULTILINE))

    # Check meta elements
    has_intro = len(text.split('\n\n')) >= 2
    has_bullets = bool(re.search(r'^[-*]\s', text, re.MULTILINE))
    has_numbered = bool(re.search(r'^\d+\.\s', text, re.MULTILINE))

    score = 0
    score += 25 if 0.5 <= density <= 3.0 else 0
    score += 20 if h1_count == 1 else 0
    score += 20 if h2_count >= 2 else (10 if h2_count >=
    1 else 0)
    score += 15 if has_intro else 0
    score += 10 if has_bullets or has_numbered else 0
    score += 10 if len(words) >= 500 else 5

```

```

    return json.dumps({
        "primary_keyword": primary_keyword,
        "occurrences": kw_count,
        "keyword_density_pct": density,
        "h1_count": h1_count,
        "h2_count": h2_count,
        "h3_count": h3_count,
        "has_lists": has_bullets or has_numbered,
        "word_count": len(words),
        "seo_score_out_of_100": score,
        "recommendation": "publish-ready" if score >= 70
    else "needs improvement",
    })

```

```

class ResearchSummaryInput(BaseModel):
    topic: str = Field(description="Topic to research")
    audience: str = Field(description="Target audience for the content")

```

```

class ResearchSummaryTool(BaseTool):
    name: str = "Research Topic Summariser"
    description: str = "Gather key facts, statistics, and talking points about a topic for content creation."
    args_schema: Type[BaseModel] = ResearchSummaryInput

```

```

    def _run(self, topic: str, audience: str) -> str:
        """
        Mock research tool. In production, replace this with a real web search API call (Brave, Serper, Tavily) and scraping.
        """
        return json.dumps({

```

```

        "topic":            topic,
        "audience":        audience,
        "key_facts":        [
            f"{topic} is a rapidly evolving area with sig
nificant practical applications.",
            f"Recent surveys show growing adoption of{top
ic} across enterprise and startup sectors.",
            f"Common challenges include implementation co
mplexity and measuring ROI.",
            f"Best practices emphasise iterative developm
ent and continuous evaluation.",
        ],
        "statistics":        [
            "78% of organisations report productivity gai
ns from AI implementations (Gartner 2024)",
            "Average time-to-value for AI projects: 6–18
months depending on scope",
            "Top barrier: data quality and availability
(cited by 64% of practitioners)",
        ],
        "recommended_angle": f"Practical guide for{audien
ce} implementing{topic} - focus on actionable steps over theo
ry.",
        "competitor_gaps":   "Most existing content is ei
ther too theoretical or too vendor-specific. Opportunity for
a balanced, practitioner-focused guide.",
    })

```

backend/crew.py

```

# backend/crew.py
import os
from dataclasses import dataclass
from crewai import Agent, Task, Crew, Process
from langchain_openai import ChatOpenAI

```

```

from backend.tools import WordCountTool, SEOAnalyserTool, ResearchSummaryTool
from dotenv import load_dotenv

load_dotenv()

@dataclass
class ContentBrief:
    topic: str
    audience: str
    tone: str # "professional", "conversational", "technical"
    target_words: int
    primary_keyword: str

def build_crew(brief: ContentBrief, model: str = "gpt-4o") -> Crew:
    """Assemble a CrewAI crew for a given content brief."""
    llm = ChatOpenAI(model=model, api_key=os.environ["OPENAI_API_KEY"], temperature=0.7)

    # — Tools —
    word_count_tool = WordCountTool()
    seo_tool = SEOAnalyserTool()
    research_tool = ResearchSummaryTool()

    # — Agents —
    researcher = Agent(
        role="Senior Content Researcher",
        goal=f"Research '{brief.topic}' thoroughly for a{brief.audience} audience and produce a structured research brief.",

```

```

        backstory="""You are a meticulous researcher with a talent for finding the most relevant facts, statistics, and insights. You understand what different audiences care about and tailor your research brief accordingly. You always cite specific data points.""",
        llm=llm,
        tools=[research_tool],
        verbose=False,
        allow_delegation=False,
        max_iter=4,
    )

    writer = Agent(
        role="Expert Content Writer",
        goal=f"Write a{brief.target_words}-word{brief.tone} article about '{brief.topic}' for{brief.audience}.",
        backstory="""You are an experienced technical and content writer who transforms research into compelling, well-structured articles. You write in active voice, use concrete examples, and structure content with clear headings and logical flow. You always hit word count targets.""",
        llm=llm,
        tools=[word_count_tool],
        verbose=False,
        allow_delegation=False,
        max_iter=5,
    )

    editor = Agent(
        role="Senior Content Editor",
        goal="Edit the article for clarity, accuracy, flow, and audience fit. Produce a polished final draft.",
        backstory="""You are a sharp editor who improves content without losing the author's voice.

```

```

        You fix logical gaps, improve transitions, eliminate
        redundancy, and ensure every paragraph
        earns its place. You flag factual claims that need ve
        rification.""",
        llm=llm,
        tools=[word_count_tool],
        verbose=False,
        allow_delegation=False,
        max_iter=4,
    )

```

```

seo_specialist = Agent(
    role="SEO Content Specialist",
    goal=f"Optimise the article for search engines target
    ing the keyword '{brief.primary_keyword}'.",
    backstory="""You are an SEO specialist who improves c
    ontent discoverability without
    compromising readability. You optimise title tags, ad
    d keyword variations, improve heading
    structure, and add internal linking suggestions. You
    never keyword-stuff.""",
    llm=llm,
    tools=[seo_tool, word_count_tool],
    verbose=False,
    allow_delegation=False,
    max_iter=4,
)

```

— Tasks

```

research_task = Task(
    description=f"""Research the topic: "{brief.topic}"

    Target audience:{brief.audience}
    Primary keyword:{brief.primary_keyword}

```


Use the Research Topic Summariser tool, then expand on its output with your own analysis.

Produce a structured research brief including:

1. Core concepts and definitions the audience needs to understand
2. Key facts and statistics (with sources where possible)
3. Common misconceptions to address
4. Practical angles most relevant to {brief.audience}
5. Recommended article structure (outline with H2 sections)"""

expected_output="""A comprehensive research brief in markdown format with:

- Executive summary (3 sentences)
- Core concepts section
- Statistics and data points
- Recommended article outline with 4-6 H2 sections
- Notes on audience expectations"""

agent=researcher,

)

writing_task = Task(

description=f"""Write a complete article based on the research brief provided.

Topic:{brief.topic}

Audience:{brief.audience}

Tone:{brief.tone}

Target word count:{brief.target_words} words ($\pm 10\%$)

Primary keyword:{brief.primary_keyword}

Requirements:

- Use the research brief's recommended outline
- Write in {brief.tone} tone throughout
- Include a compelling H1 title and introduction that hooks the reader

```

        - Use H2 and H3 headings to structure content
        - Include at least 2 practical examples or case studies

        - End with a clear conclusion and call to action
        - Use the Word and Readability Analyser tool to verify word count before submitting"""
        expected_output=f"A complete, well-structured article of{brief.target_words} words in markdown format.",
        agent=writer,
        context=[research_task],
    )

    editing_task = Task(
        description="""Edit the article for publication quality.

        Review and improve:
        1. Opening hook - does it immediately engage the target audience?
        2. Logical flow - does each section lead naturally to the next?
        3. Clarity - remove jargon, simplify complex sentences where possible
        4. Accuracy - flag any claims that seem unsupported
        5. Conciseness - cut any padding or repetition
        6. Consistency - uniform terminology and formatting throughout

        Use the Word and Readability Analyser tool to verify readability metrics.

        Preserve the author's voice - improve, don't rewrite."""
        expected_output="""The polished, edited article in markdown format plus a short editor's note
        (3-5 bullet points) summarising the key changes made."""

```

```

        agent=editor,
        context=[writing_task],
    )

    seo_task = Task(
        description=f"""Optimise the edited article for SEO.

        Primary keyword: "{brief.primary_keyword}"

        Tasks:
        1. Run the SEO Keyword Analyser tool on the article
        2. Optimise the H1 title to include the primary keyword naturally
        3. Ensure the primary keyword appears in the first 100 words
        4. Add 2-3 semantic keyword variations throughout (do not keyword-stuff)
        5. Suggest a meta description (150-160 characters) that includes the keyword
        6. Add a "Key Takeaways" section at the top if not already present
        7. Ensure all H2 headings are descriptive and scannable
        8. Add a final SEO report showing score and remaining recommendations""",
        expected_output="""The SEO-optimised article in markdown format, followed by:
        - Suggested meta description
        - SEO analysis report from the tool
        - List of any remaining improvement suggestions""",
        agent=seo_specialist,
        context=[editing_task],
    )

```

— Crew

```

crew = Crew(
    agents=[researcher, writer, editor, seo_specialist],
    tasks=[research_task, writing_task, editing_task, seo
_task],
    process=Process.sequential,
    verbose=False,
    memory=False,
)

return crew

```

backend/jobs.py

```

# backend/jobs.py
import asyncio, uuid, time, json
from dataclasses import dataclass, field
from enum import Enum
from typing import Optional
from backend.crew import ContentBrief, build_crew

class JobStatus(str, Enum):
    PENDING = "pending"
    RUNNING = "running"
    COMPLETED = "completed"
    FAILED = "failed"

@dataclass
class Job:
    id: str
    brief: ContentBrief
    status: JobStatus = JobStatus.PENDING
    progress: list[str] = field(default_factory=list)
    result: Optional[str] = None

```

```

error: Optional[str] = None
created_at: float = field(default_factory=time.time)
ended_at: Optional[float] = None

class JobManager:
    """In-memory async job manager. Replace with Redis/Celery
    for multi-server deployments."""

    def __init__(self):
        self._jobs: dict[str, Job] = {}

    def create_job(self, brief: ContentBrief) -> str:
        job_id = str(uuid.uuid4())[:8]
        self._jobs[job_id] = Job(id=job_id, brief=brief)
        return job_id

    def get_job(self, job_id: str) -> Optional[Job]:
        return self._jobs.get(job_id)

    async def run_job(self, job_id: str) -> None:
        """Run a CrewAI crew in the background."""
        job = self._jobs.get(job_id)
        if not job:
            return

        job.status = JobStatus.RUNNING
        job.progress.append(f"[{self._ts()}] Starting content
        pipeline for:{job.brief.topic}")

        stages = [
            "🔍 Researcher gathering facts and statistics...",
            "✍️ Writer drafting the article...",
            "📝 Editor polishing content and improving flo
W...",
            "🔍 SEO Specialist optimising for search...",

```

```

    ]

    try:
        # Run in a thread pool to avoid blocking the event
        loop = asyncio.get_event_loop()

        # Simulate stage progress in background
        async def progress_updater():
            for stage in stages:
                await asyncio.sleep(3)
                job.progress.append(f"[{self._ts()}] {stage}")

        progress_task = asyncio.create_task(progress_updater())

        crew = build_crew(job.brief)
        result = await loop.run_in_executor(None, crew.kickoff())

        await progress_task
        job.status = JobStatus.COMPLETED
        job.result = str(result)
        job.ended_at = time.time()
        job.progress.append(f"[{self._ts()}] ✓ Content pipeline completed.")

    except Exception as e:
        job.status = JobStatus.FAILED
        job.error = str(e)
        job.ended_at = time.time()
        job.progress.append(f"[{self._ts()}] ✗ Failed: {e}")

```

```
def _ts(self) -> str:
    return time.strftime("%H:%M:%S")
```

backend/main.py

```
# backend/main.py
import asyncio, json, time
from fastapi import FastAPI, HTTPException, BackgroundTasks
from fastapi.responses import StreamingResponse
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, Field
from contextlib import asynccontextmanager
from typing import Optional
from backend.crew import ContentBrief
from backend.jobs import JobManager, JobStatus

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.jobs = JobManager()
    yield

app = FastAPI(title="Content Studio API", version="1.0.0", lifespan=lifespan)
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])

class ContentRequest(BaseModel):
    topic: str = Field(..., min_length=3, max_length=200)
    audience: str = Field(default="software developers")
    tone: str = Field(default="professional")
    target_words: int = Field(default=800, ge=300, le=3000)
```

```

00)
    primary_keyword: str = Field(..., min_length=2, max_length=100)
    model: str = Field(default="gpt-4o")

@app.post("/jobs", status_code=202)
async def create_job(req: ContentRequest, background_tasks: BackgroundTasks) -> dict:
    """Start a content generation job. Returns job_id immediately."""
    brief = ContentBrief(
        topic=req.topic, audience=req.audience, tone=req.tone,
        target_words=req.target_words, primary_keyword=req.primary_keyword,
    )
    job_id = app.state.jobs.create_job(brief)
    background_tasks.add_task(app.state.jobs.run_job, job_id)
    return {"job_id": job_id, "status": "pending", "message": "Job started."}

@app.get("/jobs/{job_id}")
async def get_job(job_id: str) -> dict:
    """Poll job status and result."""
    job = app.state.jobs.get_job(job_id)
    if not job:
        raise HTTPException(status_code=404, detail="Job not found")
    return {
        "job_id": job.id,
        "status": job.status,
        "progress": job.progress,
        "result": job.result,
        "error": job.error,
    }

```



```

        "elapsed_s": round((job.ended_at or time.time()) - job.created_at, 1),
    }

@app.get("/jobs/{job_id}/stream")
async def stream_job(job_id: str) -> StreamingResponse:
    """Stream job progress events until completion."""

    async def event_generator():
        job = app.state.jobs.get_job(job_id)
        if not job:
            yield f"data:{json.dumps({'type': 'error', 'message': 'Job not found'})}\n\n"
            return

        sent_lines = 0
        while True:
            progress = job.progress
            for line in progress[sent_lines:]:
                yield f"data:{json.dumps({'type': 'progress', 'message': line})}\n\n"
                sent_lines += 1

            if job.status == JobStatus.COMPLETED:
                yield f"data:{json.dumps({'type': 'done', 'result': job.result})}\n\n"
                break
            elif job.status == JobStatus.FAILED:
                yield f"data:{json.dumps({'type': 'error', 'message': job.error})}\n\n"
                break

            await asyncio.sleep(1)

    return StreamingResponse(

```

```

        event_generator(),
        media_type="text/event-stream",
        headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
    )

@app.get("/health")
async def health() -> dict:
    return {"status": "ok"}

```

frontend/app.py

```

# frontend/app.py
import streamlit as st
import requests, json, time, os

st.set_page_config(page_title="Content Studio", page_icon="📄", layout="wide")
st.title("📄 Multi-Agent Content Studio")

BACKEND = os.environ.get("CONTENT_BACKEND_URL", "http://localhost:8000")

if "job_id" not in st.session_state: st.session_state.job_id = None
if "articles" not in st.session_state: st.session_state.articles = []

# — Sidebar - content brief —————
with st.sidebar:
    st.header("Content Brief")
    topic = st.text_input("Topic", placeholder="e.g. Building RAG Systems in Production")

```

```

keyword = st.text_input("Primary Keyword", placeholder
="e.g. RAG pipeline")
audience = st.selectbox("Target Audience", [
    "software developers", "data scientists", "product ma
nagers",
    "business executives", "technical beginners",
])
tone = st.selectbox("Tone", ["professional", "conversatio
nal", "technical", "educational"])
words = st.slider("Target Word Count", 400, 2000, 800, st
ep=100)
model = st.selectbox("Model", ["gpt-4o", "gpt-4o-mini"])

generate = st.button(
    "🚀 Generate Article",
    disabled=not (topic and keyword),
    use_container_width=True,
)

if st.session_state.articles:
    st.divider()
    st.header("Past Articles")
    for i, art in enumerate(reversed(st.session_state.art
icles)):
        st.caption(f"📄 {art['topic'][:40]}...")

# — Main area —
col_progress, col_article = st.columns([1, 2])

with col_progress:
    st.subheader("Pipeline Progress")
    pipeline_steps = {
        "Researcher": "🔍",
        "Writer": "✍️",
        "Editor": "📝",
    }

```

```

        "SEO":          "🎯",
    }
    step_placeholders = {name: st.empty() for name in pipeline_steps}
    for name, icon in pipeline_steps.items():
        step_placeholders[name].markdown(f"{icon} **{name}** - waiting...")

    log_placeholder = st.empty()

with col_article:
    st.subheader("Generated Article")
    article_placeholder = st.empty()
    article_placeholder.info("Your article will appear here once generation is complete.")

# — Trigger generation —————
if generate and topic and keyword:
    st.session_state.job_id = None

    try:
        resp = requests.post(
            f"{BACKEND}/jobs",
            json={
                "topic":          topic,
                "audience":       audience,
                "tone":            tone,
                "target_words":    words,
                "primary_keyword": keyword,
                "model":           model,
            },
            timeout=10,
        )
        resp.raise_for_status()
        job_id = resp.json()["job_id"]

```

```

st.session_state.job_id = job_id

# Stream progress
logs: list[str] = []
active_step = None

with requests.get(
    f"{BACKEND}/jobs/{job_id}/stream", stream=True, timeout=300
) as stream:
    for line in stream.iter_lines():
        if not line:
            continue
        line = line.decode() if isinstance(line, bytes) else line
        if not line.startswith("data: "):
            continue
        data = json.loads(line[6:])
        etype = data.get("type")

        if etype == "progress":
            msg = data.get("message", "")
            logs.append(msg)
            log_placeholder.markdown("\n\n".join(f"• {l}" for l in logs[-6:]))

            # Update step indicators
            for name, icon in pipeline_steps.items():
                if name.lower() in msg.lower():
                    active_step = name
                    step_placeholders[name].markdown(
                        f"{icon} **{name}** - running..."
                    )

        elif etype == "done":
            # Mark all steps complete
            for name, icon in pipeline_steps.items():

```

```

                                step_placeholders[name].markdown(f"✅
**{name}** - complete")

                                result = data.get("result", "")
                                article_placeholder.markdown(result)

                                st.session_state.articles.append({
                                    "topic":    topic,
                                    "keyword": keyword,
                                    "content":  result,
                                })

                                with col_article:
                                    st.download_button(
                                        "↓ Download Article (.md)",
                                        data=result,
                                        file_name=f"{topic.replace(' ',
'_' ).lower()}.md",
                                        mime="text/markdown",
                                    )
                                    break

                                elif etype == "error":
                                    st.error(f"Pipeline failed:{data.get('mes
sage')}")
                                    break

                                except requests.exceptions.ConnectionError:
                                    st.error("❌ Cannot connect to backend. Start it wit
h: `uvicorn backend.main:app --reload`")
                                except Exception as e:
                                    st.error(f"❌ Unexpected error:{e}")

```

Running P03

```
# Terminal 1 - backend
cd p03_content_studio
uvicorn backend.main:app --reload --port 8000

# Terminal 2 - frontend
streamlit run frontend/app.py

# Open http://localhost:8501
# Fill in the content brief → click Generate Article
# Watch the four agents collaborate in real time
```

What This Demonstrates

- **Multi-agent coordination** - four specialist agents with clearly defined roles and task context dependencies
- **Custom tools** - `WordCountTool`, `SEOAnalyserTool`, and `ResearchSummaryTool` implement `BaseTool` with Pydantic schemas
- **Async job queue** - POST creates a job immediately (202 Accepted), background task runs the crew, GET polls status, SSE streams progress
- **Real-time UI** - Streamlit consumes SSE progress events and updates step indicators and the log pane in real time
- **Download support** - completed article offered as a `.md` file download

Phase 6 Summary

What You Built

Project	Core Skills Demonstrated
P01 - Research Assistant	PDF ingestion with <code>pdfplumber</code> , paragraph-aware chunking, persistent ChromaDB, metadata filtering by source, inline citation extraction from streamed output, Streamlit document management UI

Project	Core Skills Demonstrated
P02 - Autonomous Agent	LangGraph ReAct graph with 5 real tools, persistent <code>MemorySaver</code> by session, SSE streaming of tool-call events from FastAPI, Streamlit consuming a live event stream, safe <code>python_repl</code> via subprocess
P03 - Content Studio	Four-agent CrewAI pipeline with task context dependencies, custom <code>BaseTool</code> implementations, async <code>BackgroundTasks</code> job queue, SSE progress streaming, Streamlit real-time step tracker and article download

Architecture Patterns Across All Three Projects

Separation of concerns - ingestion, retrieval, generation, and UI are always in separate files. This makes each piece independently testable and replaceable.

Streaming everywhere - all three projects stream responses (tokens, events, progress) rather than blocking until completion. Users see activity immediately.

Persistent state - P01 uses ChromaDB on disk, P02 uses `MemorySaver` with thread IDs, P03 uses an in-memory job manager. In all cases, state outlives a single request.

Mock → real swap - the web search in P02 and the research tool in P03 are deliberately mocked with a clear comment showing exactly where to swap in a real API (Brave, Serper, Tavily). The rest of the system does not change.

Error handling at the boundary - every Streamlit frontend wraps backend calls in `try/except requests.exceptions.ConnectionError` and shows a clear “start the backend” message. Users never see a raw traceback.

Completing the Guide

You have now worked through all six phases:

Phase	Content
Phase 1	Python Core for AI - foundations, OOP, data structures, NumPy, Pandas
Phase 2	LLM APIs and Prompting - Anthropic/OpenAI SDKs, prompt patterns, embeddings
Phase 3	GenAI Libraries - LangChain, LlamaIndex, ChromaDB, Hugging Face, LangGraph, CrewAI

Phase	Content
Phase 4	RAG and Memory - chunking, hybrid search, HyDE, multi-hop, memory systems
Phase 5	Deployment - Streamlit UIs, FastAPI backends, structured logging, evaluation
Phase 6	Portfolio Projects - three end-to-end production-grade applications

Next steps:

- Deploy P01 to Streamlit Cloud (free tier)
- Deploy P02 and P03 backends to Railway, Render, or Fly.io
- Replace mock tools with real APIs (Brave Search, Serper, Tavily)
- Add authentication (Clerk, Auth0) to any public-facing deployment
- Add observability (LangSmith for tracing, Grafana for metrics)
- Fine-tune an open-source model on your domain data using the LoRA pattern from Phase 3